

**UNIVERSIDAD DE LA CUENCA DEL PLATA**

Facultad de Ingeniería y Tecnología

Ingeniería en Sistemas de Información

**TRABAJO FINAL INTEGRADOR**

# StockFlow: diseño, implementación y programación de una base de datos para la gestión de stock y ventas multidepósito

*Documentación integral del proyecto (AE1 → AE2 → AE4)*

| | |
|---|---|
| Asignatura | Base de Datos (ISI17BAD28) |
| Actividad | Actividad de Evaluación N.º 4 (AE4) — Trabajo Final Integrador (ISI17BAD28V26AE4V26) |
| Carrera | Ingeniería en Sistemas de Información |
| Facultad | Facultad de Ingeniería y Tecnología |
| Sede | Posadas |
| Docente | Ing. Gladys Daniela Neuendorf |
| Estudiantes | Leandro Acosta — DNI 46.715.865<br>Nahuel Villegas — DNI 41.878.359 |
| Modalidad | Pares · Presencial · Sincrónica |
| Sistema gestor | Microsoft SQL Server 2022 (Transact-SQL) |
| Entrega y coloquio | Semana 15: del 16/11 al 21/11/2026 |

Posadas, Misiones — noviembre de 2026

## Resumen

El presente trabajo integra las actividades evaluativas de la asignatura Base de Datos en torno a un mismo dominio: una empresa comercializadora de productos de tecnología, informática y electrodomésticos que opera varias sucursales y depósitos. A partir del modelo conceptual elaborado en la AE1 y de su revisión, ampliación y normalización en la AE2, se implementa en Microsoft SQL Server 2022 una base de datos de 23 tablas que cubre el ciclo completo de stock y ventas: catálogo con historial de precios, existencias por depósito con kardex de movimientos, compras a proveedores, pedidos, facturación y cobranzas.

La implementación se completa con un juego de datos de prueba representativo, operaciones de manipulación con transacciones explícitas, veinte consultas que responden a requerimientos concretos, nueve vistas y una capa de programación compuesta por tres funciones, cinco procedimientos almacenados transaccionales y cinco triggers de validación, automatización y auditoría. Las reglas de negocio que el lenguaje de definición de datos no permite expresar —coherencia entre stock y kardex, capacidad de los depósitos, ciclo de vida del pedido, cobro máximo por factura e inalterabilidad del historial de precios— quedan garantizadas por el propio motor. La solución se verificó ejecutando un script integral sobre un servidor sin la base: 21 pruebas negativas y 5 positivas de integridad declarativa, y 48 pruebas de los objetos programados, todas con el resultado esperado.

**Palabras clave:** modelo entidad-relación, modelo relacional, normalización, SQL Server, procedimientos almacenados, triggers, transacciones, trazabilidad.

# Índice

1. Introducción ..... 5
1.1. Objetivos ..... 5
1.2. Metodología ..... 5
1.3. Estructura del documento y entregables ..... 6
2. Evolución del proyecto y justificación de los cambios ..... 7
2.1. AE1: relevamiento y modelo conceptual inicial ..... 7
2.2. AE2: auditoría, ampliación, normalización e implementación ..... 7
2.3. AE3 ..... 8
2.4. Borrador inicial de la AE4 y decisión de consolidación ..... 8
2.5. Elección del SGBD ..... 9
3. Relevamiento del dominio ..... 11
3.1. Contexto de la organización ..... 11
3.2. Problemática ..... 11
3.3. Alcance ..... 11
3.4. Usuarios del sistema ..... 12
3.5. Supuestos ..... 12
4. Requerimientos y reglas de negocio ..... 13
4.1. Requerimientos de información ..... 13
4.2. Requerimientos funcionales ..... 13
4.3. Reglas de negocio ..... 14
5. Modelo conceptual ..... 16
5.1. Convenciones de notación ..... 16
5.2. Entidades y relaciones ..... 16
5.3. Diagramas ..... 17
5.4. Incorporación de la AE4: auditoría de estados del pedido ..... 19
6. Modelo lógico relacional ..... 20
6.1. Transformación del modelo conceptual al relacional ..... 20
6.2. Esquema relacional ..... 21
6.3. Diagrama lógico ..... 22
7. Dependencias funcionales y normalización ..... 23
7.1. Relación universal de ventas y dependencias funcionales ..... 23
7.2. Primera, segunda y tercera forma normal ..... 23
7.3. Relación universal de stock y compras ..... 24
7.4. Verificación de las descomposiciones ..... 24
8. Diccionario de datos ..... 26
9. Implementación de la base de datos ..... 34

9.1. Entorno ..... 34
9.2. Organización y ejecución del script ..... 34
9.3. Objetos creados ..... 35
9.4. Restricciones declarativas destacadas ..... 35
10. Carga y manipulación de datos ..... 37
10.1. Datos de prueba ..... 37
10.2. Operaciones de manipulación ..... 37
10.3. Transacciones explícitas ..... 38
11. Consultas SQL y vistas ..... 40
11.1. Catálogo de consultas ..... 40
11.2. Ejemplos comentados ..... 40
11.3. Vistas ..... 42
12. Programación en la base de datos ..... 44
12.1. Criterios de diseño ..... 44
12.2. Funciones ..... 44
12.3. Procedimientos almacenados ..... 45
12.4. Triggers ..... 46
12.5. Transacciones ..... 47
13. Pruebas y evidencias de funcionamiento ..... 49
13.1. Estrategia ..... 49
13.2. Ejecución del script integral ..... 49
13.3. Pruebas de integridad declarativa ..... 49
13.4. Pruebas de funciones, procedimientos y triggers ..... 50
13.5. Defectos detectados durante las pruebas ..... 52
14. Matriz de trazabilidad ..... 53
15. Conclusiones ..... 55
16. Bitácora del equipo ..... 56
17. Reflexiones individuales ..... 57
17.1. Leandro Acosta ..... 57
17.2. Nahuel Villegas ..... 57
Referencias ..... 58
Anexo A. Aplicación web de demostración ..... 59
Anexo B. Consultas: código y resultados ..... 67

# 1. Introducción

La Actividad de Evaluación N.º 4 constituye la instancia integradora de la asignatura. Su propósito es demostrar la correspondencia entre el análisis de requerimientos, el modelado conceptual, el diseño lógico y la implementación de la solución en un Sistema Gestor de Bases de Datos (SGBD), así como documentar la evolución del proyecto desde la primera actividad y fundamentar las decisiones adoptadas en cada etapa.

## 1.1. Objetivos

**Objetivo general.** Implementar y poner en funcionamiento, sobre un SGBD relacional, la base de datos del sistema de gestión de stock y ventas StockFlow, garantizando la coherencia entre los requerimientos, las reglas de negocio, el modelo conceptual, el esquema relacional normalizado y los objetos programados.

**Objetivos específicos.**

- Consolidar la documentación de las actividades AE1 y AE2 e incorporar las mejoras y correcciones surgidas durante el proceso, justificando cada modificación.
- Implementar el esquema relacional mediante SQL DDL con sus tipos de datos, claves primarias y foráneas y restricciones de integridad, en un script ejecutable desde una base vacía.
- Cargar datos de prueba representativos del dominio y ejercitar operaciones de inserción, actualización y eliminación.
- Formular consultas y vistas que respondan a requerimientos concretos del sistema.
- Programar funciones, procedimientos almacenados, triggers y transacciones que automaticen operaciones y hagan cumplir las reglas de negocio no expresables de forma declarativa.
- Verificar el funcionamiento de la solución con pruebas de situaciones válidas y rechazadas, y presentar las evidencias de ejecución.

## 1.2. Metodología

El proyecto se desarrolló de manera incremental, en etapas coincidentes con las actividades evaluativas. En la AE1 se relevó el dominio y se construyó el diagrama entidad-relación (DER). En la AE2 se auditó ese diagrama, se corrigieron los errores de notación y de cardinalidad detectados, se amplió el modelo al ciclo completo de stock y ventas, se lo transformó al modelo relacional, se lo normalizó hasta la tercera forma normal (3FN) y se lo implementó en SQL. En la AE4 se consolidó el modelo final, se incorporaron los objetos programados y se ejecutó una batería de pruebas automatizadas que registra, para cada caso, el resultado esperado y el obtenido.

Para la implementación se adoptó un criterio de **defensa en profundidad**: toda regla que el motor puede verificar de forma declarativa (claves, CHECK, índices únicos filtrados) se declara en el DDL; las reglas que involucran varias filas o tablas se programan en triggers, de modo que rijan incluso ante sentencias ejecutadas fuera de la aplicación; y las operaciones de negocio compuestas por varios pasos se encapsulan en procedimientos almacenados transaccionales.

## 1.3. Estructura del documento y entregables

Las secciones 2 a 8 reúnen la documentación integral del proyecto (evolución, dominio, requerimientos, reglas de negocio, modelos conceptual y relacional, normalización y diccionario de datos). Las secciones 9 a 13 describen la implementación, la carga y manipulación de datos, las consultas y vistas, la programación en la base de datos y las pruebas. La sección 14 presenta la matriz de trazabilidad, y las secciones 15 a 17 contienen las conclusiones, la bitácora del equipo y las reflexiones individuales. Los entregables digitales se detallan en la Tabla 1.

**Tabla 1.** *Entregables digitales del trabajo final integrador.*

| Archivo | Contenido |
|---|---|
| `AE4/AE4_StockFlow_script_integral.sql` | Script integral, ordenado y ejecutable desde una base vacía (3.250 líneas). Se genera concatenando los ocho módulos de `AE4/sql/`. |
| `AE4/sql/01_ddl.sql` | Creación de la base `StockFlow`: 23 tablas, claves, restricciones, índices y 6 vistas de atributos derivados. |
| `AE4/sql/02_datos_prueba.sql` | Datos de prueba sobre las 23 tablas. |
| `AE4/sql/03_pruebas_integridad.sql` | 26 pruebas de las restricciones declarativas y 8 consultas de verificación del modelo (heredadas de la AE2). |
| `AE4/sql/04_programacion.sql` | 3 funciones, 5 procedimientos almacenados y 5 triggers. |
| `AE4/sql/05_vistas.sql` | 3 vistas de explotación para la aplicación y los reportes. |
| `AE4/sql/06_manipulacion_dml.sql` | Operaciones INSERT, UPDATE y DELETE, transacciones explícitas y ventas completas mediante procedimientos. |
| `AE4/sql/07_consultas.sql` | 20 consultas (C01–C20) que responden a requerimientos del sistema. |
| `AE4/sql/08_pruebas_programacion.sql` | 48 pruebas automatizadas de funciones, procedimientos y triggers. |
| `AE4/evidencias/ejecucion_script_integral.log` | Registro completo de la ejecución del script integral (evidencia de funcionamiento). |
| `stockflow/` | Aplicación web de demostración (HTML + PHP) adaptada al modelo final (Anexo A). |

# 2. Evolución del proyecto y justificación de los cambios

La consigna exige que la documentación permita reconocer la evolución y la trazabilidad del proyecto. Esta sección resume qué produjo cada etapa, qué problemas se detectaron y cómo se resolvieron, de modo que cada decisión del modelo final pueda rastrearse hasta su origen.

## 2.1. AE1: relevamiento y modelo conceptual inicial

La AE1 (entregada el 05/09/2026) relevó el dominio de una comercializadora de productos de tecnología e informática, definió ocho requerimientos de información (RI01–RI08) y siete reglas de negocio (RN01–RN07), y construyó un DER con siete conjuntos de entidades: CATEGORIAS, DEPOSITOS, PROVEEDORES, CLIENTES, PRODUCTOS, PEDIDOS y la entidad débil asociativa DETALLE_PEDIDO. Como decisiones de modelado se destacan la separación de las categorías en una entidad propia, la resolución de la relación N:M entre pedidos y productos mediante DETALLE_PEDIDO y la conservación del precio pactado en cada línea de venta (atributo PrecioUnitarioHistorico).

El propio informe de la AE1 dejó explicitadas sus limitaciones: cada producto se almacenaba en un único depósito (RN02) y era provisto por un único proveedor; el stock mínimo exigido por RI08 no estaba representado; la regla de capacidad de los depósitos (RN07) no era expresable en el modelo conceptual; y quedaban fuera del alcance los estados del pedido, las direcciones de entrega y la facturación.

## 2.2. AE2: auditoría, ampliación, normalización e implementación

La AE2 (01/10/2026) auditó el DER de la AE1 y distinguió dos clases de defectos. Los **errores de notación** (atributos dentro del rectángulo, relaciones como rótulos de aristas, claves foráneas en el modelo conceptual, falta de distinción entre participación total y parcial) impedían considerar el diagrama como notación de Chen. Los **errores de modelado** habrían producido un esquema incapaz de representar el negocio; se resumen en la Tabla 2.

**Tabla 2.** *Errores de modelado del DER de la AE1 detectados y corregidos en la AE2.*

| # | Hallazgo en la AE1 | Corrección adoptada |
|---|---|---|
| M1 | ALMACENA modelada 1:N: cada producto en un único depósito. | Relación N:M con atributo Cantidad; el stock vive en la relación (tabla Stock). |
| M2 | StockTotal derivado sin fuente de la cual derivarse. | Se deriva como SUM(Stock.Cantidad) sobre los depósitos del producto. |
| M3 | SUMINISTRA 1:N: un único proveedor por producto. | Relación N:M con PrecioCompra, PlazoEntregaDias, CodigoProveedor y EsPreferido. |
| M4 | Precio como atributo simple de PRODUCTOS: se pierde el valor anterior al actualizarlo. | Entidad débil PRECIO_PRODUCTO con vigencia desde/hasta. |
| M5 | Total derivado en PEDIDOS y ausencia de comprobante fiscal. | Entidad FACTURA (1:1 con PEDIDO), PAGO y MEDIO_PAGO. |
| M6 | CUIT y Email sin restricción de unicidad. | UNIQUE sobre CUIT, (TipoDoc, NroDoc) y Email. |
| M7 | Sin registro de movimientos de stock. | Entidad MOVIMIENTO_STOCK (kardex) con tipo, cantidad, fecha, empleado y documento de origen. |

El modelo conceptual se amplió de 7 a 16 entidades (se incorporaron SUCURSAL, EMPLEADO, MARCA, PRECIO_PRODUCTO, la agregación STOCK, MOVIMIENTO_STOCK, ORDEN_COMPRA, FACTURA, PAGO, MEDIO_PAGO y DIRECCION_CLIENTE), se transformó en un esquema de 22 tablas, se normalizó hasta 3FN —verificando además FNBC y 4FN— y se implementó en SQL Server 2022. La AE2 dejó documentadas tres reglas que el DDL no podía expresar y que requerían programación: que la suma de pagos no supere el total de la factura, que el neto facturado coincida con el detalle del pedido y que el stock sea coherente con el saldo del kardex (AE2, §11.6).

## 2.3. AE3

La AE3 fue entregada y aprobada con calificación 8 (ocho). No se recibieron devoluciones escritas ni orales sobre esa entrega, por lo que no hay observaciones de la cátedra que incorporar; las mejoras realizadas en la versión final surgen de la revisión propia del equipo, documentada en las secciones 2.2, 2.4 y 13.5. Los productos de la AE3 no forman parte del repositorio digital del proyecto, por lo que esta documentación integra las entregas AE1 y AE2, el borrador inicial de la AE4 y la versión final descripta en este informe.

## 2.4. Borrador inicial de la AE4 y decisión de consolidación

El primer borrador de la AE4 implementó en MySQL 8 una base denominada TiendaSimpleDB de diez tablas, con un procedimiento de alta de pedidos, una función de total gastado por cliente, dos triggers (asignación de precio histórico y auditoría de cambios de precio) y una vista de resumen comercial. Al contrastarlo con la AE2 se constató que el borrador **retrocedía al modelo de la AE1**: reintroducía las relaciones 1:N corregidas en M1 y M3, el precio como atributo actualizable de PRODUCTOS (M4) y la ausencia de kardex (M7), y dejaba fuera la facturación y la cobranza. Además, su procedimiento sp_RegistrarPedido creaba un pedido sin renglones, en contradicción con la regla de participación total de PEDIDOS en COMPONE (todo pedido contiene al menos una línea).

Por esos motivos el equipo decidió consolidar la versión final sobre el modelo completo de la AE2, implementado en SQL Server 2022, e incorporar sobre él los objetos programados. Las ideas válidas del borrador no se descartaron sino que se reformularon sobre el modelo corregido, como muestra la Tabla 3.

**Tabla 3.** *Correspondencia entre el borrador inicial de la AE4 y la versión final.*

| Elemento del borrador (MySQL) | Versión final (SQL Server) | Motivo del cambio |
|---|---|---|
| Base TiendaSimpleDB, 10 tablas | Base StockFlow, 23 tablas (22 de la AE2 + PedidoEstadoHistorial) | El borrador reintroducía los errores M1, M3, M4 y M7 ya corregidos en la AE2. |
| Tabla AUDITORIA_PRECIOS + trigger AFTER UPDATE sobre PRODUCTOS | Historial ProductoPrecio (AE2) + sp_ActualizarPrecioProducto + trigger trg_ProductoPrecio_ProtegerHistorial | Con vigencias desde/hasta el historial ya existe; lo necesario es impedir que se altere (RN10). |

| Elemento del borrador (MySQL) | Versión final (SQL Server) | Motivo del cambio |
|---|---|---|
| Trigger BEFORE INSERT que copia el precio de catálogo al detalle | sp_RegistrarPedido asigna el precio con fn_PrecioVigente; CHECK Cantidad > 0 | SQL Server no tiene triggers BEFORE; el precio vigente se obtiene del historial. |
| sp_RegistrarPedido: inserta solo la cabecera | sp_RegistrarPedido: cabecera, renglones, control de stock y kardex en una transacción | Un pedido sin renglones viola RN08; la venta debe descontar stock de forma atómica. |
| fn_CalcularTotalGastoCliente | fn_TotalCompradoCliente | Considera el descuento de cada renglón y excluye pedidos cancelados. |
| vw_ResumenVentasClientes | vw_ResumenVentasCliente | Agrega documento, última compra y saldo pendiente de cobro. |
| RN08 (capacidad) y RN10 (stock crítico) delegadas a la aplicación | Trigger trg_Stock_ControlarExistencias y vista vw_StockBajoMinimo | Lo que el motor puede garantizar no debe depender del código de la aplicación. |
| Clientes de prueba con los nombres de los integrantes | Datos ficticios representativos del dominio (AE2 + casos límite AE4) | Los datos deben ejercitar los casos límite del modelo. |

## 2.5. Elección del SGBD

La consigna requiere implementar el modelo «en el SGBD». La implementación se realizó en **Microsoft SQL Server 2022 Express**, el sistema gestor utilizado durante toda la cursada y en la AE2, que ya había sido implementada y verificada en ese motor. El borrador inicial de la AE4 se había escrito en MySQL porque era el entorno disponible en ese momento; al consolidar el trabajo se migró a SQL Server, de modo que la entrega final es ejecutable y verificable en ese motor. La migración no presentó obstáculos porque las construcciones exigidas tienen equivalencia directa entre ambos productos (Tabla 4). La elección permitió además aprovechar dos capacidades que simplifican el modelo: los índices únicos filtrados —que resuelven reglas como «un único precio vigente por producto» sin triggers— y el procesamiento nativo de JSON para recibir el detalle de un pedido como un único parámetro.

**Tabla 4.** *Equivalencia de las construcciones utilizadas entre MySQL 8 y SQL Server 2022.*

| Construcción | MySQL 8 | SQL Server 2022 (utilizado) |
|---|---|---|
| Autonumérico | AUTO_INCREMENT | IDENTITY(1,1) |
| Último identificador generado | LAST_INSERT_ID() | SCOPE_IDENTITY() |
| Delimitación de rutinas | DELIMITER // … END // | Lotes separados por GO |
| Señalar un error | SIGNAL SQLSTATE '45000' | THROW número, mensaje, estado |
| Manejo de excepciones | DECLARE EXIT HANDLER FOR SQLEXCEPTION | BEGIN TRY … END TRY BEGIN CATCH … END CATCH |
| Transacción | START TRANSACTION / COMMIT / ROLLBACK | BEGIN TRANSACTION / COMMIT / ROLLBACK, SET XACT_ABORT ON |
| Triggers | BEFORE/AFTER, por fila (NEW/OLD) | AFTER/INSTEAD OF, por sentencia (tablas inserted/deleted) |
| Unicidad condicional | Columna generada o trigger | Índice único filtrado (WHERE …) |
| Atributo derivado de la fila | Columna GENERATED | Columna calculada PERSISTED |

La diferencia de mayor impacto en el diseño de los triggers es que SQL Server los ejecuta **una vez por sentencia** y no por fila: todos los triggers del proyecto se escribieron orientados a

conjuntos, operando sobre las tablas virtuales `inserted` y `deleted`, de modo que funcionan correctamente aunque una sentencia afecte muchas filas.

# 3. Relevamiento del dominio

## 3.1. Contexto de la organización

La organización analizada comercializa productos de tecnología, informática y electrodomésticos en la provincia de Misiones. Opera tres sucursales (Casa Central y Sucursal Norte, en Posadas, y Sucursal Oberá), cada una con uno o más depósitos, y vende por tres canales: mostrador, web y teléfono. Su operatoria comprende la administración del catálogo y de las listas de precios, el control de existencias por depósito, la reposición mediante órdenes de compra a proveedores, la venta a clientes registrados, la emisión de facturas y la cobranza, que puede ser parcial y con distintos medios de pago.

## 3.2. Problemática

La situación relevada en la AE1 presentaba tres deficiencias: duplicación de datos de clientes y proveedores, incoherencia en los precios registrados e insuficiente control de los niveles de stock. Ello impedía responder con confiabilidad consultas operativas básicas, como la disponibilidad real de un artículo en un depósito, el importe efectivamente pactado en una venta anterior o el saldo adeudado por un cliente. Al ampliar el análisis en la AE2 se sumaron dos problemas: la existencia en stock era un número sin historia —no se podía auditar un faltante ni saber quién movió la mercadería— y no existía separación entre el pedido comercial y el comprobante fiscal.

## 3.3. Alcance

**Incluido en el alcance:**

- Organización: sucursales, depósitos y empleados con su rol y jerarquía de supervisión.
- Catálogo: categorías jerárquicas, marcas, productos e historial de precios con vigencia.
- Stock: existencias por producto y depósito con mínimos, máximos y ubicación interna; kardex de movimientos (ingresos, egresos, transferencias y ajustes).
- Compras: proveedores con teléfonos múltiples, condiciones de suministro por producto y órdenes de compra con recepción parcial.
- Ventas: clientes con teléfonos y direcciones múltiples, pedidos por canal, detalle con precio histórico y descuento, facturación A/B/C, pagos parciales con distintos medios y auditoría de estados del pedido.

**Fuera del alcance:** notas de crédito y débito, liquidación de impuestos distintos del IVA, logística de despacho y seguimiento de envíos, devoluciones de proveedores, administración de usuarios y permisos del SGBD, y la interfaz de usuario definitiva (la aplicación web incluida es una demostración).

## 3.4. Usuarios del sistema

Los usuarios se identifican por el rol registrado en la tabla Empleado, que además condiciona las operaciones que pueden realizar a través de los procedimientos almacenados (Tabla 5).

**Tabla 5.** *Usuarios del sistema y operaciones que realizan.*

| Rol | Descripción | Operaciones principales |
|---|---|---|
| VENDEDOR | Atiende clientes en una sucursal. | Registra pedidos de su sucursal (sp_RegistrarPedido), consulta catálogo y stock, cancela pedidos no facturados. |
| DEPOSITO | Responsable físico de la mercadería. | Registra recepciones, transferencias y ajustes de inventario (movimientos de kardex). |
| COMPRAS | Gestiona la reposición. | Emite órdenes de compra, consulta productos bajo mínimo y proveedores preferidos. |
| GERENTE | Conduce una sucursal; supervisa empleados. | Registra ventas, actualiza precios (sp_ActualizarPrecioProducto) y consulta indicadores. |
| ADMIN | Administración y facturación. | Emite facturas (sp_FacturarPedido), registra cobros (sp_RegistrarPago) y consulta la cartera de clientes. |
| Cliente (externo) | Persona humana o jurídica que compra. | No accede a la base; es sujeto de los datos. Se identifica por tipo y número de documento. |

## 3.5. Supuestos

- Un producto puede almacenarse en varios depósitos y ser provisto por varios proveedores, uno de los cuales es el preferido.
- El precio de venta tiene un único valor vigente por día; los cambios rigen desde la fecha indicada y no se admiten vigencias futuras.
- Un pedido se despacha desde un único depósito de la sucursal que lo registra; los pedidos de mostrador no llevan dirección de envío.
- Cada pedido genera a lo sumo una factura; la factura A corresponde a clientes con CUIT y la B al resto.
- Las cantidades admiten decimales porque existen productos que se venden por kilogramo, litro o metro.

# 4. Requerimientos y reglas de negocio

## 4.1. Requerimientos de información

Los requerimientos de información (RI) expresan los datos que el sistema debe registrar y las consultas que debe satisfacer. Se conserva la numeración de la AE1 para RI01–RI08 y se agregan los surgidos en las etapas posteriores. La columna «Origen» permite reconstruir la evolución de cada uno.

**Tabla 6.** *Requerimientos de información.*

| Código | Requerimiento | Origen |
|---|---|---|
| RI01 | Registrar y consultar los datos personales y de contacto de los clientes (teléfonos y direcciones múltiples). | AE1 (ampliado AE2) |
| RI02 | Mantener el catálogo de productos con categoría, marca, precio vigente y stock. | AE1 (ampliado AE2) |
| RI03 | Registrar los depósitos con su ubicación, sucursal y capacidad máxima. | AE1 |
| RI04 | Consultar la existencia de cada producto en cada depósito. | AE1 (modificado AE2) |
| RI05 | Registrar los proveedores, sus teléfonos y las condiciones en que proveen cada producto. | AE1 (ampliado AE2) |
| RI06 | Registrar los pedidos de los clientes con fecha, hora, canal, vendedor y sucursal. | AE1 (ampliado AE2) |
| RI07 | Almacenar el detalle de cada pedido con cantidad, precio pactado y descuento. | AE1 |
| RI08 | Consultar los productos cuyo stock está por debajo del mínimo configurado. | AE1 (resuelto AE2) |
| RI09 | Conservar el historial de precios de venta y costo de cada producto. | AE2 |
| RI10 | Registrar todos los movimientos de stock con responsable y documento de origen (kardex). | AE2 |
| RI11 | Registrar las órdenes de compra a proveedores y la recepción de la mercadería. | AE2 |
| RI12 | Emitir facturas, registrar pagos y conocer el saldo pendiente de cada factura y cliente. | AE2 |
| RI13 | Registrar la organización: sucursales, empleados, roles y jerarquía de supervisión. | AE2 |
| RI14 | Auditar los cambios de estado de los pedidos (qué cambió, cuándo y quién). | AE4 |
| RI15 | Obtener indicadores de ventas por sucursal, canal, categoría, producto y cliente. | AE4 |

## 4.2. Requerimientos funcionales

Los requerimientos funcionales (RF) describen las operaciones que el sistema debe ejecutar. Cada uno se implementa en un objeto de la base de datos, lo que permite que las reglas se apliquen de forma uniforme cualquiera sea la aplicación cliente.

**Tabla 7.** *Requerimientos funcionales y objeto que los implementa.*

| Código | Requerimiento funcional | Implementación |
|---|---|---|
| RF01 | Registrar una venta validando cliente, vendedor, sucursal, depósito, canal, precio vigente y stock, y descontar las existencias. | sp_RegistrarPedido |

| Código | Requerimiento funcional | Implementación |
|---|---|---|
| RF02 | Facturar un pedido con numeración correlativa y neto igual al total del detalle. | sp_FacturarPedido |
| RF03 | Registrar cobros parciales o totales sin exceder el saldo de la factura. | sp_RegistrarPago, trg_Pago_ControlarSaldo |
| RF04 | Actualizar el precio de un producto conservando el historial. | sp_ActualizarPrecioProducto, trg_ProductoPrecio_ProtegerHistorial |
| RF05 | Anular un pedido no facturado y reponer el stock egresado. | sp_CancelarPedido |
| RF06 | Registrar la recepción de una orden de compra e ingresar la mercadería. | Transacción M2, trg_MovimientoStock_AplicarKardex |
| RF07 | Transferir mercadería entre depósitos. | Transacción M3, trg_MovimientoStock_AplicarKardex |
| RF08 | Consultar el catálogo con precio vigente, stock total y proveedor preferido. | vw_CatalogoProductos, C01, C04 |
| RF09 | Detectar productos bajo mínimo y sugerir a qué proveedor reponer. | vw_StockBajoMinimo, vw_StockPorDeposito, C11 |
| RF10 | Consultar la cuenta corriente de los clientes y las facturas con saldo. | vw_FacturaSaldo, vw_ResumenVentasCliente, C13 |
| RF11 | Obtener indicadores de ventas. | C05–C08, C16 |
| RF12 | Consultar la trazabilidad de un pedido (estados y movimientos). | PedidoEstadoHistorial, C17 |
| RF13 | Mantener los datos maestros (clientes, productos, proveedores, contactos). | DML M1, C19, C20 |

## 4.3. Reglas de negocio

Las reglas de negocio (RN) son restricciones semánticas del dominio que la base de datos debe garantizar. RN01 a RN07 conservan la numeración y el sentido de la AE1 (RN02 con la modificación introducida en la AE2); el resto se incorporó al ampliar el modelo o al implementar la capa de programación. La última columna indica el mecanismo concreto que hace cumplir cada regla; entre paréntesis se consigna el código de error que devuelve el motor cuando la regla se viola.

**Tabla 8.** *Catálogo consolidado de reglas de negocio.*

| Código | Regla de negocio | Origen | Mecanismo de verificación |
|---|---|---|---|
| RN01 | Todo producto pertenece a una única categoría existente. | AE1 | FK_Producto_Categoria (NOT NULL) |
| RN02 | La existencia se registra por par (producto, depósito) y el par no se repite. *Modificada en AE2:* un producto puede estar en varios depósitos. | AE1 → AE2 | PK_Stock (ProductoId, DepositoId) |
| RN03 | Un pedido solo puede generarse para un cliente registrado y activo. | AE1 | FK_Pedido_Cliente; sp_RegistrarPedido (50001) |
| RN04 | Cada renglón del pedido refiere a un único producto, que no se repite en el mismo pedido. | AE1 | PK_PedidoDetalle; sp_RegistrarPedido (50007) |
| RN05 | La existencia de un producto en un depósito no puede ser negativa. | AE1 | CK_Stock_Cantidad; trg_MovimientoStock_AplicarKardex (50102); sp_RegistrarPedido (50010) |
| RN06 | El precio del renglón se fija al momento de la venta y no cambia aunque cambie la lista. | AE1 | PedidoDetalle.PrecioUnitarioHistorico asignado por sp_RegistrarPedido con fn_PrecioVigente |
| RN07 | La suma de existencias de un depósito no puede superar su capacidad máxima. | AE1 (diferida) | trg_Stock_ControlarExistencias (50112) |

| Código | Regla de negocio | Origen | Mecanismo de verificación |
|---|---|---|---|
| RN08 | Todo pedido contiene al menos un renglón. | AE1 | sp_RegistrarPedido (50006) |
| RN09 | Cada producto tiene a lo sumo un precio vigente y los períodos de vigencia no se superponen. | AE2 | UX_ProductoPrecio_Vigente; CK_ProductoPrecio_Periodo; sp_ActualizarPrecioProducto |
| RN10 | El historial de precios es inalterable: solo puede cerrarse el período vigente. | AE4 | trg_ProductoPrecio_ProtegerHistorial (50141, 50142) |
| RN11 | La existencia es el saldo del kardex: solo cambia mediante movimientos de stock. | AE2 (pendiente) → AE4 | trg_MovimientoStock_AplicarKardex; trg_Stock_ControlarExistencias (50111) |
| RN12 | Los movimientos de stock no se modifican ni se eliminan; se corrigen con un movimiento compensatorio. | AE4 | trg_MovimientoStock_AplicarKardex (50101) |
| RN13 | Ciclo de vida del pedido: PENDIENTE → CONFIRMADO \| CANCELADO; CONFIRMADO → ENTREGADO \| CANCELADO. | AE4 | CK_Pedido_Estado; trg_Pedido_ControlarEstado (50121) |
| RN14 | Un pedido solo se entrega facturado; un pedido facturado no se cancela; la cancelación repone el stock. | AE2 (pendiente) → AE4 | trg_Pedido_ControlarEstado (50122–50124); sp_CancelarPedido; sp_FacturarPedido (50022) |
| RN15 | Cada alta y cambio de estado de un pedido queda auditado con fecha, hora y usuario. | AE4 | Tabla PedidoEstadoHistorial; trg_Pedido_ControlarEstado |
| RN16 | La suma de los pagos de una factura no puede superar su total. | AE2 (pendiente) → AE4 | trg_Pago_ControlarSaldo (50131) |
| RN17 | Cada pedido tiene a lo sumo una factura, con numeración única por tipo y punto de venta, y neto igual al total del detalle. | AE2 → AE4 | UQ_Factura_Pedido; UQ_Factura_Numeracion; sp_FacturarPedido (50023) |
| RN18 | Los pedidos de mostrador no llevan dirección; los web y telefónicos se envían a una dirección del propio cliente. | AE2 | CK_Pedido_Envio; FK_Pedido_ClienteDireccion (compuesta); sp_RegistrarPedido (50005) |
| RN19 | Un vendedor solo vende en su sucursal y despacha desde un depósito de ella. | AE4 | sp_RegistrarPedido (50002–50004) |
| RN20 | SKU, CUIT, documento de cliente y correos electrónicos no se repiten. | AE1 → AE2 | UQ_Producto_Sku; UQ_Proveedor_Cuit; UQ_Cliente_Doc; UX_Cliente_Email; UX_Proveedor_Email |

**Clasificación de los mecanismos**

**Estructurales (DDL):** RN01, RN02, RN04, RN05, RN09, RN17, RN18 y RN20 se garantizan con claves primarias, foráneas, UNIQUE, CHECK e índices únicos filtrados.

**Programadas (triggers):** RN05, RN07, RN10–RN16 comparan una fila con un agregado o con otra tabla, o necesitan conocer el valor anterior; un CHECK no puede hacerlo.

**Operativas (procedimientos):** RN03, RN06, RN08, RN14, RN17 y RN19 dependen del contexto de la operación (quién vende, desde qué depósito, a qué precio) y se validan antes de abrir la transacción.

# 5. Modelo conceptual

El modelo conceptual definitivo es el elaborado en la AE2 en notación de Peter Chen (Chen, 1976; Elmasri y Navathe, 2016). Comprende 16 conjuntos de entidades y 24 conjuntos de relaciones, organizados en cuatro módulos: catálogo, stock, compras y ventas-cobranzas. En la AE4 se incorpora la auditoría de estados del pedido (§5.4).

## 5.1. Convenciones de notación

**Tabla 9.** *Símbolos de la notación de Chen empleados en los diagramas.*

| Símbolo | Significado |
|---|---|
| Rectángulo / rectángulo de doble borde | Entidad fuerte / entidad débil |
| Rombo / rombo de doble borde | Relación / relación identificadora |
| Elipse; nombre subrayado | Atributo; atributo identificador |
| Nombre subrayado punteado | Discriminante de entidad débil (clave parcial) |
| Elipse de doble borde / borde punteado | Atributo multivaluado / atributo derivado |
| Elipse con elipses hijas | Atributo compuesto |
| Línea simple / línea doble | Participación parcial / participación total |
| Rectángulo punteado envolvente | Agregación (una relación tratada como entidad) |

## 5.2. Entidades y relaciones

**Tabla 10.** *Entidades del modelo conceptual y problema que resuelven.*

| Entidad | Tipo | Propósito |
|---|---|---|
| SUCURSAL | Fuerte | Unidad de negocio a la que pertenecen depósitos, empleados y ventas. |
| EMPLEADO | Fuerte | Responsable de cada operación; autorrelación SUPERVISA. |
| CATEGORIA | Fuerte | Clasificación jerárquica (autorrelación SUBCATEGORIA DE). |
| MARCA | Fuerte | Dimensión de análisis independiente de la categoría. |
| PRODUCTO | Fuerte | Artículo comercializado; centro del modelo (seis relaciones). |
| PRECIO_PRODUCTO | Débil de PRODUCTO | Historial de precios con vigencia (discriminante FechaDesde). |
| DEPOSITO | Fuerte | Almacén físico con capacidad máxima. |
| STOCK | Agregación de ALMACENA | Existencia de un producto en un depósito, con mínimo, máximo y ubicación. |
| MOVIMIENTO_STOCK | Fuerte | Kardex: cada ingreso, egreso, transferencia o ajuste. |
| PROVEEDOR | Fuerte | Firma que suministra productos; teléfono multivaluado. |
| ORDEN_COMPRA | Fuerte | Pedido de reposición a un proveedor (DETALLA productos). |
| CLIENTE | Fuerte | Comprador; teléfono multivaluado y nombre compuesto. |
| DIRECCION_CLIENTE | Débil de CLIENTE | Domicilios del cliente (multivaluado y compuesto). |
| PEDIDO | Fuerte | Venta; COMPONE productos con precio histórico y descuento. |
| FACTURA | Fuerte | Comprobante fiscal; GENERA 1:1 con PEDIDO. |

| Entidad | Tipo | Propósito |
|---|---|---|
| PAGO / MEDIO_PAGO | Débil de FACTURA / Fuerte | Cobranza parcial en distintos medios. |

Las relaciones N:M del modelo —ALMACENA (producto-depósito), SUMINISTRA (proveedor-producto), DETALLA (orden de compra-producto) y COMPONE (pedido-producto)— poseen atributos propios que no pertenecen a ninguna de las entidades que vinculan. El caso más significativo es PrecioUnitarioHistorico en COMPONE: depende del par (pedido, producto) y no del producto solo, por lo que no constituye una redundancia respecto del historial de precios.

Un movimiento de stock no afecta a un producto ni a un depósito por separado, sino a la existencia de ese producto en ese depósito, que es la relación ALMACENA. Como la notación de Chen no permite conectar un rombo con otro, se utilizó una **agregación**: ALMACENA y sus dos entidades se tratan como la entidad STOCK, con la que se relaciona MOVIMIENTO_STOCK.

## 5.3. Diagramas

La Figura 1 presenta el mapa general sin atributos; las Figuras 2 a 5 detallan cada módulo con todos sus atributos. Las entidades que aparecen en más de un módulo se repiten sin atributos fuera de su módulo propio.

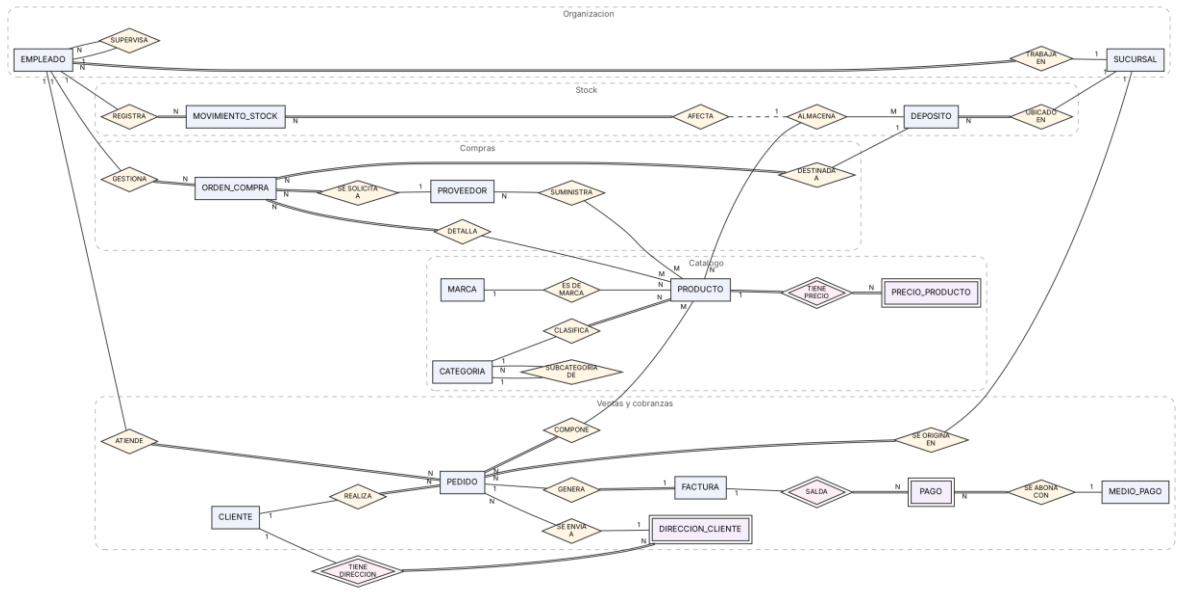



**Figura 1.** *Mapa general del modelo conceptual en notación de Chen (AE2).*

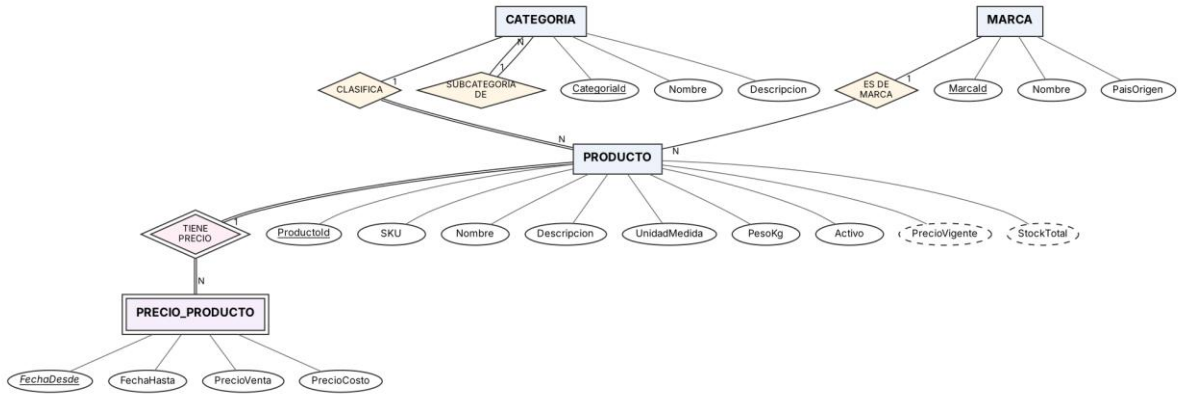



**Figura 2.** *Módulo de catálogo: categorías, marcas, productos e historial de precios.*

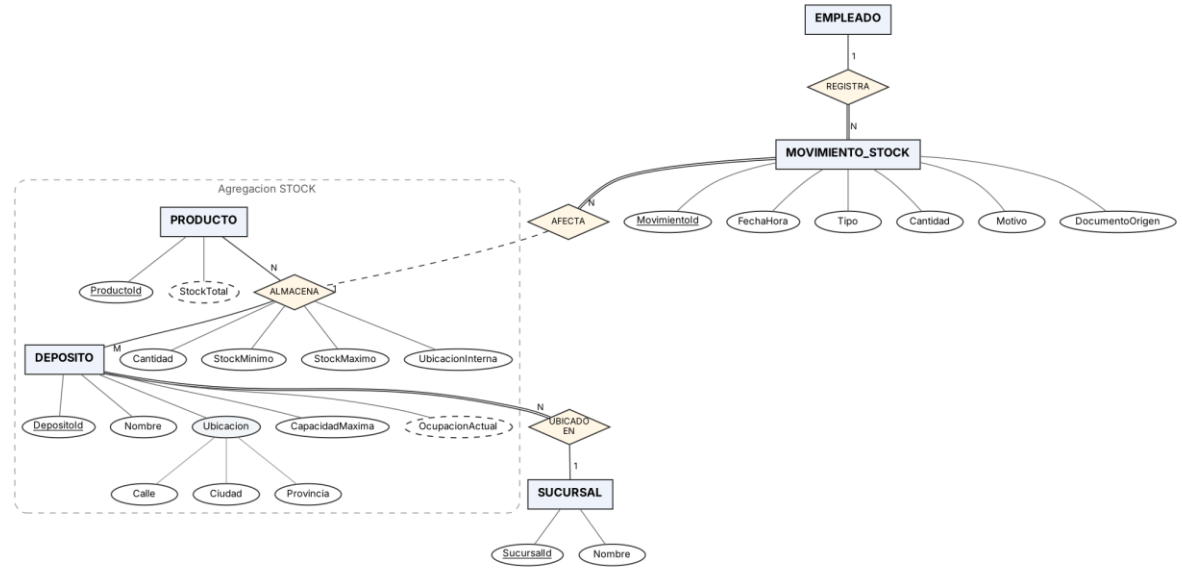



**Figura 3.** *Módulo de stock: agregación STOCK y kardex de movimientos.*

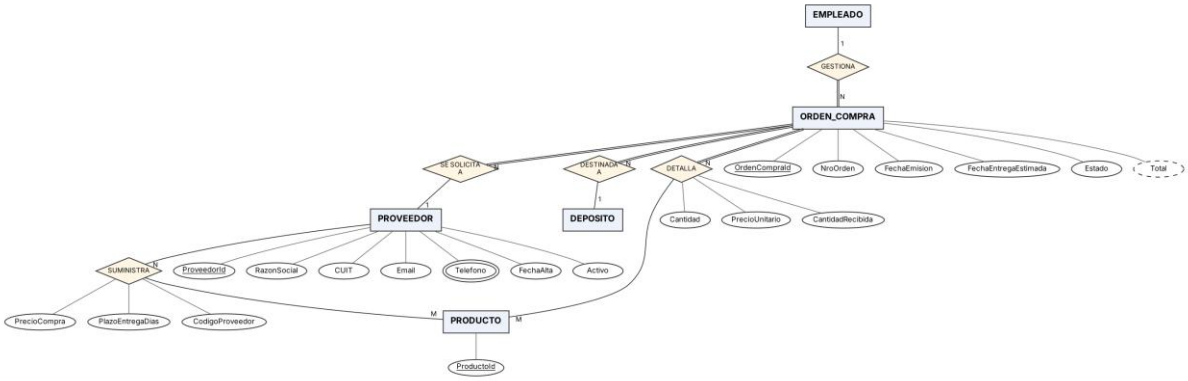



**Figura 4.** *Módulo de compras: proveedores, condiciones de suministro y órdenes de compra.*

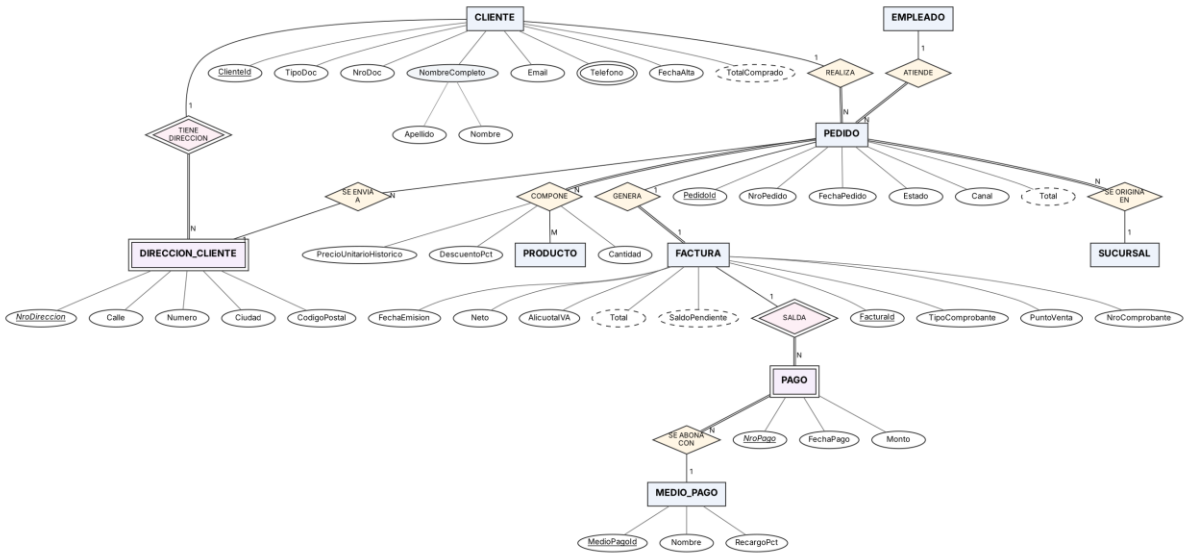



**Figura 5.** *Módulo de ventas y cobranzas: clientes, pedidos, facturas y pagos.*

## 5.4. Incorporación de la AE4: auditoría de estados del pedido

Para satisfacer RI14 y RN15 se incorpora la entidad débil PEDIDO_ESTADO_HISTORIAL, dependiente de PEDIDO, con los atributos EstadoAnterior, EstadoNuevo, FechaHora y Usuario. Cada pedido participa en ella con cardinalidad 1:N; la participación del historial es total (no existe registro de auditoría sin pedido). En la implementación se le asignó una clave sustituta (HistorialId) en lugar de la clave (PedidoId, FechaHora), porque dos cambios de estado pueden producirse dentro del mismo segundo —por ejemplo, el alta y la confirmación dentro de una misma transacción— y la clave natural no sería única. Esta es la única modificación del modelo conceptual respecto de la AE2.

# 6. Modelo lógico relacional

## 6.1. Transformación del modelo conceptual al relacional

La transformación aplica las reglas clásicas de mapeo entidad-relación a relacional (Elmasri y Navathe, 2016, cap. 9). La tabla siguiente indica dónde se aplica cada una.

**Tabla 11.** *Reglas de transformación aplicadas.*

| Regla | Enunciado | Aplicación en el proyecto |
|---|---|---|
| R1 | Entidad fuerte → tabla; su identificador → clave primaria. | Sucursal, Empleado, Categoria, Marca, Producto, Deposito, MovimientoStock, Proveedor, OrdenCompra, Cliente, Pedido, Factura, MedioPago (13 tablas). |
| R2 | Entidad débil → tabla; PK = PK del propietario + discriminante. | ProductoPrecio (ProductoId, FechaDesde), ClienteDireccion (ClienteId, NroDireccion), Pago (FacturaId, NroPago). PedidoEstadoHistorial usa clave sustituta (§5.4). |
| R3 | Relación 1:N → clave foránea en el lado N. | Producto.CategoriaId, Producto.MarcaId, Deposito.SucursalId, Empleado.SucursalId, Pedido.ClienteId, Pago.MedioPagoId, entre otras. |
| R4 | Relación 1:1 → clave foránea con UNIQUE en el lado de participación total. | Factura.PedidoId con UQ_Factura_Pedido. |
| R5 | Relación N:M → tabla propia; PK = unión de las PK; atributos de la relación como columnas. | Stock, ProveedorProducto, OrdenCompraDetalle, PedidoDetalle. |
| R6 | Atributo multivaluado → tabla propia; PK = PK de la entidad + discriminante. | ClienteTelefono, ProveedorTelefono. |
| R7 | Atributo compuesto → una columna por componente. | NombreCompleto → Nombre, Apellido; Ubicacion → Calle, Ciudad, Provincia. |

**Casos especiales.** Las autorrelaciones SUPERVISA y SUBCATEGORIA DE generan claves foráneas nulables hacia la misma tabla (Empleado.SupervisorId, Categoria.CategoriaPadreId) con ON DELETE NO ACTION. La agregación STOCK se materializa como la tabla de ALMACENA y MovimientoStock la referencia mediante una **clave foránea compuesta** sobre (ProductoId, DepositoId), lo que impide registrar movimientos sobre combinaciones inexistentes. Pedido referencia ClienteDireccion con una **clave foránea compuesta solapada** (ClienteId, NroDireccion) que reutiliza la columna ClienteId, por lo que es imposible por construcción enviar un pedido a la dirección de otro cliente.

**Atributos derivados.** Solo Factura.TotalFactura se materializa, como columna calculada PERSISTED, porque depende exclusivamente de columnas de su propia fila. El stock total, el precio vigente, el total del pedido, el saldo de la factura y la ocupación del depósito se obtienen mediante vistas (vw_ProductoStock, vw_ProductoPrecioVigente, vw_PedidoTotal, vw_FacturaSaldo, vw_DepositoOcupacion), ya que materializarlos exigiría mantener datos redundantes.

## 6.2. Esquema relacional

Notación: los atributos en cursiva forman la clave primaria, * indica clave foránea y ° indica unicidad.

### Organización y catálogo

Sucursal(*SucursalId*, Nombre°, Calle, Numero, Ciudad, Provincia, Telefono, Activa)

Empleado(*EmpleadoId*, Legajo°, Nombre, Apellido, Dni°, Email°, FechaIngreso, Rol, SucursalId*, SupervisorId*, Activo)

Categoria(*CategoriaId*, Nombre°, Descripcion, CategoriaPadreId*)

Marca(*MarcaId*, Nombre°, PaisOrigen)

Producto(*ProductoId*, Sku°, Nombre, Descripcion, CategoriaId*, MarcaId*, UnidadMedida, PesoKg, Activo)

ProductoPrecio(*ProductoId*, FechaDesde*, FechaHasta, PrecioVenta, PrecioCosto)

### Stock

Deposito(*DepositoId*, Nombre, SucursalId*, Calle, Ciudad, Provincia, CapacidadMaxima) °(SucursalId, Nombre)

Stock(*ProductoId*, DepositoId**, Cantidad, StockMinimo, StockMaximo, UbicacionInterna)

MovimientoStock(*MovimientoId*, FechaHora, ProductoId*, DepositoId*, TipoMovimiento, Cantidad, EmpleadoId*, Motivo, DocumentoTipo, DocumentoId) (ProductoId, DepositoId) → Stock

### Compras

Proveedor(*ProveedorId*, RazonSocial, Cuit°, Email°, FechaAlta, Activo)

ProveedorTelefono(*ProveedorId*, NroTelefono*, Telefono, Tipo)

ProveedorProducto(*ProveedorId*, ProductoId**, PrecioCompra, PlazoEntregaDias, CodigoProveedor, EsPreferido)

OrdenCompra(*OrdenCompraId*, NroOrden°, ProveedorId*, EmpleadoId*, DepositoId*, FechaEmision, FechaEntregaEstimada, Estado)

OrdenCompraDetalle(*OrdenCompraId*, ProductoId**, Cantidad, PrecioUnitario, CantidadRecibida)

### Ventas, cobranzas y auditoría

Cliente(*ClienteId*, TipoDoc, NroDoc, Nombre, Apellido, Email°, FechaAlta, Activo) °(TipoDoc, NroDoc)

ClienteTelefono(*ClienteId*, NroTelefono*, Telefono, Tipo)

ClienteDireccion(*ClienteId*, NroDireccion*, Calle, Numero, Ciudad, Provincia, CodigoPostal, EsPrincipal)

Pedido(*PedidoId*, NroPedido°, ClienteId*, NroDireccion*, EmpleadoId*, SucursalId*, FechaPedido, Estado, Canal) (ClienteId, NroDireccion) → ClienteDireccion

PedidoDetalle(*PedidoId*, ProductoId**, Cantidad, PrecioUnitarioHistorico, DescuentoPct)

Factura(*FacturaId*, PedidoId*°, TipoComprobante, PuntoVenta, NroComprobante, FechaEmision, Neto, AlicuotaIva, TotalFactura) °(TipoComprobante, PuntoVenta, NroComprobante)

MedioPago(*MedioPagoId*, Nombre°, RecargoPct, Activo)

Pago(*FacturaId*, NroPago*, FechaPago, Monto, MedioPagoId*, Referencia)

PedidoEstadoHistorial(*HistorialId*, PedidoId*, EstadoAnterior, EstadoNuevo, FechaHora, Usuario) [AE4]

El esquema resultante tiene **23 tablas y 31 claves foráneas**, de las cuales 9 propagan la eliminación en cascada (siempre de un detalle hacia su cabecera) y 22 son NO ACTION. SQL Server rechaza los esquemas con más de un camino de cascada hacia la misma tabla (error 1785), lo que obligó a revisar cada clave foránea; la decisión es también conceptualmente correcta: eliminar un cliente no debe arrastrar su historial de ventas.

## 6.3. Diagrama lógico

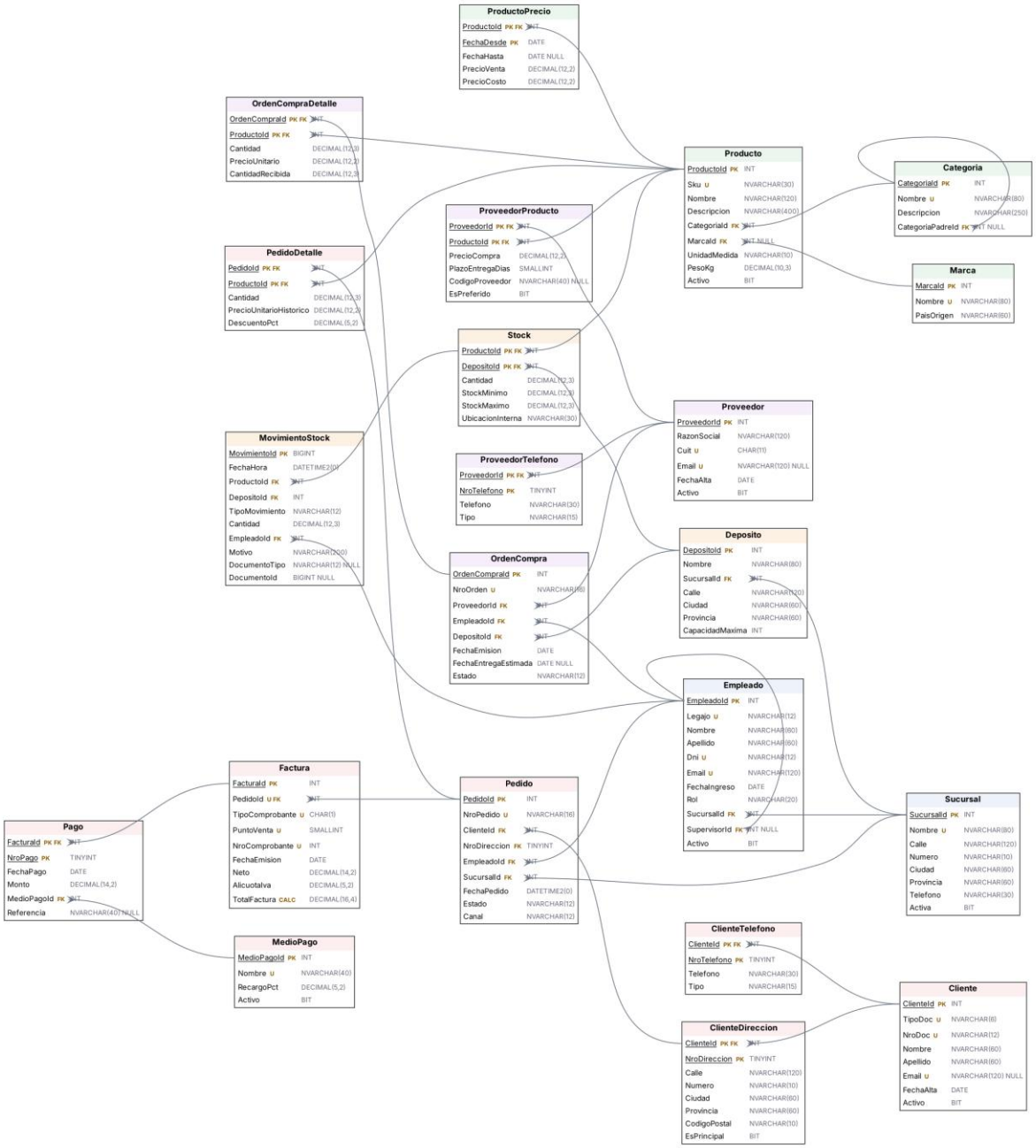



**Figura 6.** *Diagrama lógico del esquema relacional (AE2). En la AE4 se agrega PedidoEstadoHistorial, referenciada desde Pedido.*

# 7. Dependencias funcionales y normalización

Se normalizan dos relaciones universales —la de ventas y la de stock y compras— en lugar de afirmar que el esquema «ya está normalizado». Partir de la relación universal permite mostrar las dependencias funcionales (DF), las anomalías que producen y la descomposición que las elimina (Date, 2019; Silberschatz et al., 2019).

## 7.1. Relación universal de ventas y dependencias funcionales

R_VENTA(NroPedido, FechaPedido, Estado, Canal, ClienteId, ClienteNombre, ClienteApellido, ClienteEmail, ClienteTelefonos, ClienteDireccion, EmpleadoId, EmpleadoNombre, SucursalId, SucursalNombre, SucursalCiudad, ProductoId, Sku, ProductoNombre, CategoriaId, CategoriaNombre, MarcaId, MarcaNombre, UnidadMedida, Cantidad, PrecioUnitarioHistorico, DescuentoPct, TipoComprobante, PuntoVenta, NroComprobante, FechaEmision, Neto, AlicuotaIva, NroPago, FechaPago, MontoPago, MedioPagoNombre)

Clave candidata: (NroPedido, ProductoId).

**Tabla 12.** *Dependencias funcionales de la relación universal de ventas.*

| DF | Dependencia |
|---|---|
| f1 | NroPedido → FechaPedido, Estado, Canal, ClienteId, EmpleadoId, SucursalId |
| f2 | ClienteId → ClienteNombre, ClienteApellido, ClienteEmail |
| f3 | EmpleadoId → EmpleadoNombre, SucursalId |
| f4 | SucursalId → SucursalNombre, SucursalCiudad |
| f5 | ProductoId → Sku, ProductoNombre, CategoriaId, MarcaId, UnidadMedida |
| f6 | Sku → ProductoId (clave alternativa) |
| f7 | CategoriaId → CategoriaNombre |
| f8 | MarcaId → MarcaNombre |
| f9 | (NroPedido, ProductoId) → Cantidad, PrecioUnitarioHistorico, DescuentoPct |
| f10 | NroPedido → TipoComprobante, PuntoVenta, NroComprobante, FechaEmision, Neto, AlicuotaIva |
| f11 | (TipoComprobante, PuntoVenta, NroComprobante) → NroPedido |
| f12 | (NroPedido, NroPago) → FechaPago, MontoPago, MedioPagoNombre |

**Anomalías.** De inserción: no puede darse de alta un producto o un cliente hasta que intervengan en un pedido, porque NroPedido forma parte de la clave. De actualización: la corrección del correo de un cliente obliga a modificar una fila por cada renglón de cada pedido suyo, con riesgo de inconsistencia si la actualización es parcial. De eliminación: al borrar el único pedido que contenía un producto desaparecen del sistema su nombre, SKU y categoría.

## 7.2. Primera, segunda y tercera forma normal

**1FN.** ClienteTelefonos es multivaluado y ClienteDireccion es multivaluado y compuesto: se extraen a ClienteTelefono(ClienteId, NroTelefono, …) y ClienteDireccion(ClienteId, NroDireccion, …). Los compuestos no multivaluados (NombreCompleto) se descomponen en columnas atómicas sin tabla nueva.

**2FN.** Se eliminan las dependencias parciales respecto de la clave (NroPedido, ProductoId): f1 y f10 dependen solo de NroPedido (cabecera y comprobante), f5 solo de ProductoId y f12 de (NroPedido, NroPago). La descomposición produce PEDIDO_2, PRODUCTO_2, DETALLE_2(NroPedido, ProductoId, Cantidad, PrecioUnitarioHistorico, DescuentoPct) y PAGO_2. PrecioUnitarioHistorico permanece en DETALLE_2 porque depende de la clave completa: es el precio de ese producto en ese pedido.

**3FN.** Se eliminan las dependencias transitivas NroPedido → ClienteId → datos del cliente (Cliente), NroPedido → EmpleadoId → datos del empleado (Empleado), EmpleadoId → SucursalId → datos de la sucursal (Sucursal), ProductoId → CategoriaId → CategoriaNombre (Categoria) y ProductoId → MarcaId → MarcaNombre (Marca). MedioPagoNombre depende de MedioPagoId y se separa en MedioPago. Los datos del comprobante, aunque dependen de NroPedido, describen una entidad con su propia clave candidata (f11) y se separan en Factura con UNIQUE(PedidoId), lo que preserva la cardinalidad 1:1.

## 7.3. Relación universal de stock y compras

R_STOCK(ProductoId, DepositoId, Cantidad, StockMinimo, UbicacionInterna, ProductoNombre, Sku, DepositoNombre, DepositoCiudad, SucursalId, SucursalNombre, CapacidadMaxima, ProveedorId, ProveedorRazonSocial, ProveedorCuit, PrecioCompra, PlazoEntregaDias) tiene clave (ProductoId, DepositoId, ProveedorId): una clave de tres atributos que ya revela dos hechos independientes mezclados. En 2FN, Cantidad, StockMinimo y UbicacionInterna dependen de (ProductoId, DepositoId) y PrecioCompra y PlazoEntregaDias de (ProveedorId, ProductoId), lo que produce Stock y ProveedorProducto, además de Producto, Deposito y Proveedor. En 3FN, DepositoId → SucursalId → SucursalNombre genera Sucursal. La misma mecánica aplicada a la cabecera de compra produce OrdenCompra y OrdenCompraDetalle.

El precio como atributo simple de Producto no viola ninguna forma normal, pero produce una anomalía de actualización: al modificarlo se pierde el valor anterior. Su solución es temporal y no estructural —la tabla ProductoPrecio con período de vigencia—, lo que muestra que no toda anomalía se resuelve normalizando.

## 7.4. Verificación de las descomposiciones

**Unión sin pérdida.** En cada descomposición binaria la intersección de los esquemas es la clave primaria de uno de ellos (NroPedido para PEDIDO_2, ProductoId para PRODUCTO_2, ClienteId para Cliente, PedidoId para Factura), condición suficiente para que la reunión natural reconstruya la relación original.

**Preservación de dependencias.** Las doce DF sobreviven en una única tabla cada una: f1 en Pedido, f2 en Cliente, f3 en Empleado, f4 en Sucursal, f5 y f6 en Producto (f6 como UNIQUE(Sku)), f7 en Categoria, f8 en Marca, f9 en PedidoDetalle, f10 y f11 en Factura (f11 como UNIQUE(TipoComprobante, PuntoVenta, NroComprobante)) y f12 en Pago.

**Más allá de 3FN.** El esquema cumple FNBC: en Factura y en Producto, que tienen dos claves candidatas, todo determinante es superclave. También cumple 4FN, porque las dependencias multivaluadas independientes NroPedido →→ renglones y NroPedido →→ pagos quedaron separadas en PedidoDetalle y Pago.

**Redundancia controlada.** PedidoDetalle.PrecioUnitarioHistorico y Factura.Neto parecen redundantes respecto del historial de precios y de la suma del detalle, pero son datos de hecho congelados al momento de la operación por requisitos de auditoría. La normalización elimina la redundancia que puede volverse inconsistente, no la que registra un valor en un instante. En la AE4, la coherencia de Factura.Neto con el detalle queda garantizada por sp_FacturarPedido, que lo calcula desde vw_PedidoTotal (prueba P2-04).

# 8. Diccionario de datos

El diccionario se obtuvo consultando el catálogo del sistema de SQL Server (sys.columns, sys.key_constraints, sys.foreign_keys, sys.check_constraints, sys.indexes y sys.default_constraints) sobre la base efectivamente creada por el script, de modo que refleja con exactitud lo implementado. Convenciones: **PK** clave primaria, **FK** clave foránea (→ tabla referenciada), **UQ** única, **UX** índice único filtrado, **DF** valor por defecto, **CK** restricción de dominio. En la columna «Nulo», «No» indica NOT NULL.

Todas las tablas pertenecen al esquema dbo. Los textos se almacenan como NVARCHAR (Unicode), los importes como DECIMAL —nunca FLOAT, que introduce errores de redondeo—, las cantidades como DECIMAL(12,3) porque existen productos que se venden por kilo, litro o metro, las fechas como DATE y los instantes como DATETIME2(0).

## *8.1. Sucursal*

Unidades de negocio de la empresa. Filas en el juego de datos final: 3.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| SucursalId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Nombre | NVARCHAR(80) | No | | UQ | Denominación. |
| Calle | NVARCHAR(120) | No | | — | Calle del domicilio. |
| Numero | NVARCHAR(10) | No | | — | Altura de la calle. |
| Ciudad | NVARCHAR(60) | No | | — | Ciudad. |
| Provincia | NVARCHAR(60) | No | | — | Provincia. |
| Telefono | NVARCHAR(30) | Sí | | — | Número telefónico. |
| Activa | BIT | No | | DF 1 | Indicador de vigencia (1 = activa). |

## *8.2. Empleado*

Personal de la empresa, con su rol, sucursal y supervisor. Filas en el juego de datos final: 8.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| EmpleadoId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Legajo | NVARCHAR(12) | No | | UQ | Número de legajo interno. |
| Nombre | NVARCHAR(60) | No | | — | Denominación. |
| Apellido | NVARCHAR(60) | No | | — | Apellido. |
| Dni | NVARCHAR(12) | No | | UQ<br>CK NOT Dni like '%[^0-9]%' AND (len(Dni)>=7 AND len(Dni)<=9) | Documento nacional de identidad. |
| Email | NVARCHAR(120) | No | | UQ<br>CK Email like '%_@_%._%' | Correo electrónico. |
| FechaIngreso | DATE | No | | — | Fecha de ingreso a la empresa. |
| Rol | NVARCHAR(20) | No | | CK Rol IN ('VENDEDOR', 'DEPOSITO', 'COMPRAS', 'ADMIN', 'GERENTE') | Rol funcional; condiciona las operaciones permitidas. |
| SucursalId | INT | No | FK | FK → Sucursal | Sucursal donde trabaja. |

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| SupervisorId | INT | Sí | FK | FK → Empleado<br>CK SupervisorId<>EmpleadoId | Supervisor directo (autorrelación SUPERVISA); NULL si no reporta a nadie. |
| Activo | BIT | No | | DF 1 | Indicador de vigencia (1 = activo); se usa en lugar de borrar. |

### *8.3. Categoria*

Clasificación jerárquica de los productos. Filas en el juego de datos final: 7.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| CategoriaId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Nombre | NVARCHAR(80) | No | | UQ | Denominación. |
| Descripcion | NVARCHAR(250) | Sí | | — | Texto descriptivo. |
| CategoriaPadreId | INT | Sí | FK | FK → Categoria<br>CK CategoriaPadreId<>CategoriaId | Categoría padre (autorrelación); NULL en las categorías raíz. |

### *8.4. Marca*

Marcas comerciales de los productos. Filas en el juego de datos final: 6.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| MarcaId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Nombre | NVARCHAR(80) | No | | UQ | Denominación. |
| PaisOrigen | NVARCHAR(60) | Sí | | — | País de origen de la marca. |

### *8.5. Producto*

Artículos comercializados. Filas en el juego de datos final: 14.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProductoId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Sku | NVARCHAR(30) | No | | UQ | Código de artículo (clave alternativa). |
| Nombre | NVARCHAR(120) | No | | — | Denominación. |
| Descripcion | NVARCHAR(400) | Sí | | — | Texto descriptivo. |
| CategoriaId | INT | No | FK | FK → Categoria | Categoría del producto (RN01). |
| MarcaId | INT | Sí | FK | FK → Marca | Marca del producto. |
| UnidadMedida | NVARCHAR(10) | No | | DF 'UN'<br>CK UnidadMedida IN ('UN', 'KG', 'LT', 'MT', 'CJ') | Unidad de venta: unidad, kilo, litro, metro o caja. |
| PesoKg | DECIMAL(10,3) | Sí | | CK PesoKg IS NULL OR PesoKg>=0 | Peso unitario en kilogramos. |
| Activo | BIT | No | | DF 1 | Indicador de vigencia (1 = activo); se usa en lugar de borrar. |

### 8.6. ProductoPrecio

Historial de precios de venta y costo con período de vigencia (entidad débil de Producto). Filas en el juego de datos final: 19.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProductoId | INT | No | PK, FK | FK → Producto CASCADE<br>UX donde FechaHasta IS NULL | Producto propietario. |
| FechaDesde | DATE | No | PK | — | Inicio de la vigencia (discriminante). |
| FechaHasta | DATE | Sí | | CK FechaHasta IS NULL OR FechaHasta>FechaDesde | Fin de la vigencia, inclusivo; NULL = precio vigente. |
| PrecioVenta | DECIMAL(12,2) | No | | CK PrecioVenta>0 | Precio de venta sin IVA. |
| PrecioCosto | DECIMAL(12,2) | Sí | | CK PrecioCosto IS NULL OR PrecioCosto>=0 | Costo de reposición de referencia. |

### 8.7. Deposito

Almacenes físicos de cada sucursal. Filas en el juego de datos final: 4.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| DepositoId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Nombre | NVARCHAR(80) | No | | — | Denominación. |
| SucursalId | INT | No | FK | FK → Sucursal<br>UQ (SucursalId, Nombre) | Sucursal a la que pertenece. |
| Calle | NVARCHAR(120) | Sí | | — | Calle del domicilio. |
| Ciudad | NVARCHAR(60) | Sí | | — | Ciudad. |
| Provincia | NVARCHAR(60) | Sí | | — | Provincia. |
| CapacidadMaxima | INT | Sí | | CK CapacidadMaxima IS NULL OR CapacidadMaxima>0 | Capacidad en unidades; NULL = sin límite (RN07). |

### 8.8. Stock

Existencia de cada producto en cada depósito (relación N:M ALMACENA). Filas en el juego de datos final: 21.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProductoId | INT | No | PK, FK | FK → Producto | Producto almacenado. |
| DepositoId | INT | No | PK, FK | FK → Deposito | Depósito donde se almacena. |
| Cantidad | DECIMAL(12,3) | No | | DF 0<br>CK Cantidad>=0 | Existencia actual; solo la modifica el kardex (RN11). |
| StockMinimo | DECIMAL(12,3) | No | | DF 0<br>CK StockMinimo>=0<br>CK StockMaximo IS NULL OR StockMaximo>=StockMinimo | Punto de reposición en ese depósito (RI08). |
| StockMaximo | DECIMAL(12,3) | Sí | | — | Existencia máxima deseada. |
| UbicacionInterna | NVARCHAR(30) | Sí | | — | Pasillo, estante o sector. |

### 8.9. MovimientoStock

Kardex: registro inalterable de ingresos, egresos, transferencias y ajustes. Filas en el juego de datos final: 26.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| MovimientoId | BIGINT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| FechaHora | DATETIME2(0) | No | | DF fecha y hora actual | Momento del movimiento. |
| ProductoId | INT | No | FK | FK → Stock (ProductoId, DepositoId) | Producto (parte de la FK compuesta a Stock). |
| DepositoId | INT | No | FK | FK → Stock (ProductoId, DepositoId) | Depósito (parte de la FK compuesta a Stock). |
| TipoMovimiento | NVARCHAR(12) | No | | CK TipoMovimiento IN ('INGRESO', 'EGRESO', 'AJUSTE', 'TRANSFER') | INGRESO suma; EGRESO, TRANSFER y AJUSTE restan. |
| Cantidad | DECIMAL(12,3) | No | | CK Cantidad>0 | Siempre positiva; el signo lo determina el tipo. |
| EmpleadoId | INT | No | FK | FK → Empleado | Responsable del movimiento. |
| Motivo | NVARCHAR(200) | Sí | | — | Explicación del movimiento. |
| DocumentoTipo | NVARCHAR(12) | Sí | | CK DocumentoTipo IS NULL OR DocumentoTipo IN ('OC', 'PEDIDO', 'AJUSTE')<br>CK DocumentoTipo IS NULL AND DocumentoId IS NULL OR DocumentoTipo IS NOT NULL AND DocumentoId IS NOT NULL | Documento que origina el movimiento (PEDIDO, OC, AJUSTE). |
| DocumentoId | BIGINT | Sí | | — | Identificador del documento de origen. |

### 8.10. Proveedor

Firmas que suministran productos. Filas en el juego de datos final: 4.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProveedorId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| RazonSocial | NVARCHAR(120) | No | | — | Nombre legal de la firma. |
| Cuit | CHAR(11) | No | | UQ<br>CK NOT Cuit like '%[^0-9]%' | CUIT de 11 dígitos sin guiones. |
| Email | NVARCHAR(120) | Sí | | UX donde Email IS NOT NULL<br>CK Email IS NULL OR Email like '%_@_%._%' | Correo electrónico. |
| FechaAlta | DATE | No | | DF fecha actual | Fecha de alta en el sistema. |
| Activo | BIT | No | | DF 1 | Indicador de vigencia (1 = activo); se usa en lugar de borrar. |

### 8.11. ProveedorTelefono

Teléfonos de los proveedores (atributo multivaluado). Filas en el juego de datos final: 6.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProveedorId | INT | No | PK, FK | FK → Proveedor CASCADE | Proveedor propietario. |
| NroTelefono | TINYINT | No | PK | — | Discriminante dentro del proveedor. |
| Telefono | NVARCHAR(30) | No | | — | Número telefónico. |
| Tipo | NVARCHAR(15) | No | | DF 'PRINCIPAL'<br>CK Tipo IN ('PRINCIPAL', 'ALTERNATIVO', 'WHATSAPP', 'FAX') | Clasificación del teléfono. |

### 8.12. ProveedorProducto

Condiciones en que cada proveedor suministra cada producto (relación N:M SUMINISTRA). Filas en el juego de datos final: 14.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ProveedorId | INT | No | PK, FK | FK → Proveedor CASCADE | Proveedor. |
| ProductoId | INT | No | PK, FK | FK → Producto<br>UX donde EsPreferido=1 | Producto suministrado. |
| PrecioCompra | DECIMAL(12,2) | No | | CK PrecioCompra>0 | Precio de compra acordado. |
| PlazoEntregaDias | SMALLINT | No | | DF 0<br>CK PlazoEntregaDias>=0 | Plazo de entrega en días. |
| CodigoProveedor | NVARCHAR(40) | Sí | | — | Código del artículo en el catálogo del proveedor. |
| EsPreferido | BIT | No | | DF 0 | Proveedor preferido del producto (a lo sumo uno). |

### 8.13. OrdenCompra

Cabecera de las órdenes de compra a proveedores. Filas en el juego de datos final: 4.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| OrdenCompraId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| NroOrden | NVARCHAR(16) | No | | UQ | Número visible de la orden. |
| ProveedorId | INT | No | FK | FK → Proveedor | Proveedor destinatario. |
| EmpleadoId | INT | No | FK | FK → Empleado | Empleado que emite la orden. |
| DepositoId | INT | No | FK | FK → Deposito | Depósito de recepción. |
| FechaEmision | DATE | No | | DF fecha actual<br>CK FechaEntregaEstimada IS NULL OR FechaEntregaEstimada>=FechaEmision | Fecha de emisión. |
| FechaEntregaEstimada | DATE | Sí | | — | Fecha de entrega comprometida. |
| Estado | NVARCHAR(12) | No | | DF 'BORRADOR'<br>CK Estado IN ('BORRADOR', 'EMITIDA', 'RECIBIDA', 'ANULADA') | Estado de la orden. |

### 8.14. OrdenCompraDetalle

Renglones de las órdenes de compra (relación N:M DETALLA). Filas en el juego de datos final: 8.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| OrdenCompraId | INT | No | PK, FK | FK → OrdenCompra CASCADE | Orden de compra. |
| ProductoId | INT | No | PK, FK | FK → Producto | Producto pedido. |
| Cantidad | DECIMAL(12,3) | No | | CK Cantidad>0<br>CK CantidadRecibida>=0 AND CantidadRecibida<=Cantidad | Cantidad en la unidad de medida del producto. |
| PrecioUnitario | DECIMAL(12,2) | No | | CK PrecioUnitario>=0 | Precio pactado en la orden. |
| CantidadRecibida | DECIMAL(12,3) | No | | DF 0 | Cantidad efectivamente recibida. |

### 8.15. Cliente

Compradores registrados, personas humanas o jurídicas. Filas en el juego de datos final: 7.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ClienteId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| TipoDoc | NVARCHAR(6) | No | | UQ (TipoDoc, NroDoc)<br>CK TipoDoc IN ('DNI', 'CUIT', 'CUIL', 'PAS') | Tipo de documento. |
| NroDoc | NVARCHAR(12) | No | | — | Número de documento. |
| Nombre | NVARCHAR(60) | No | | — | Nombre o denominación. |
| Apellido | NVARCHAR(60) | No | | — | Apellido o razón social. |
| Email | NVARCHAR(120) | Sí | | UX donde Email IS NOT NULL<br>CK Email IS NULL OR Email like '%_@_%._%' | Correo electrónico; opcional y único si existe. |
| FechaAlta | DATE | No | | DF fecha actual | Fecha de alta en el sistema. |
| Activo | BIT | No | | DF 1 | Indicador de vigencia (1 = activo); se usa en lugar de borrar. |

### 8.16. ClienteTelefono

Teléfonos de los clientes (atributo multivaluado). Filas en el juego de datos final: 7.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ClienteId | INT | No | PK, FK | FK → Cliente CASCADE | Cliente propietario. |
| NroTelefono | TINYINT | No | PK | — | Discriminante dentro del cliente. |
| Telefono | NVARCHAR(30) | No | | — | Número telefónico. |
| Tipo | NVARCHAR(15) | No | | DF 'PRINCIPAL'<br>CK Tipo IN ('PRINCIPAL', 'ALTERNATIVO', 'WHATSAPP', 'LABORAL') | Clasificación del teléfono. |

### 8.17. ClienteDireccion

Domicilios de los clientes (entidad débil; multivaluado y compuesto). Filas en el juego de datos final: 8.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| ClienteId | INT | No | PK, FK | FK → Cliente CASCADE<br>UX donde EsPrincipal=1 | Cliente propietario. |
| NroDireccion | TINYINT | No | PK | — | Discriminante dentro del cliente. |
| Calle | NVARCHAR(120) | No | | — | Calle del domicilio. |
| Numero | NVARCHAR(10) | No | | — | Altura de la calle. |
| Ciudad | NVARCHAR(60) | No | | — | Ciudad. |
| Provincia | NVARCHAR(60) | No | | — | Provincia. |
| CodigoPostal | NVARCHAR(10) | Sí | | — | Código postal. |
| EsPrincipal | BIT | No | | DF 0 | Dirección por defecto (a lo sumo una por cliente). |

## 8.18. Pedido

Cabecera de las ventas. Filas en el juego de datos final: 9.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| PedidoId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| NroPedido | NVARCHAR(16) | No | | UQ | Número visible PED-AAAA-NNNN. |
| ClienteId | INT | No | FK | FK → Cliente<br>FK → ClienteDireccion (ClienteId, NroDireccion) | Cliente comprador (RN03). |
| NroDireccion | TINYINT | Sí | FK | FK → ClienteDireccion (ClienteId, NroDireccion)<br>CK Canal='MOSTRADOR' AND NroDireccion IS NULL OR Canal<>'MOSTRADOR' AND NroDireccion IS NOT NULL | Dirección de envío del propio cliente; NULL en mostrador (RN18). |
| EmpleadoId | INT | No | FK | FK → Empleado | Vendedor que registra la venta. |
| SucursalId | INT | No | FK | FK → Sucursal | Sucursal de la venta. |
| FechaPedido | DATETIME2(0) | No | | DF fecha y hora actual | Fecha y hora del pedido. |
| Estado | NVARCHAR(12) | No | | DF 'PENDIENTE'<br>CK Estado IN ('PENDIENTE', 'CONFIRMADO', 'ENTREGADO', 'CANCELADO') | Estado del ciclo de vida (RN13). |
| Canal | NVARCHAR(12) | No | | CK Canal IN ('MOSTRADOR', 'WEB', 'TELEFONO') | Canal de venta. |

## 8.19. PedidoDetalle

Renglones de cada venta (relación N:M COMPONE). Filas en el juego de datos final: 19.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| PedidoId | INT | No | PK, FK | FK → Pedido CASCADE | Pedido. |
| ProductoId | INT | No | PK, FK | FK → Producto | Producto vendido. |
| Cantidad | DECIMAL(12,3) | No | | CK Cantidad>0 | Cantidad en la unidad de medida del producto. |
| PrecioUnitarioHistorico | DECIMAL(12,2) | No | | CK PrecioUnitarioHistorico>=0 | Precio vigente al momento de la venta (RN06). |
| DescuentoPct | DECIMAL(5,2) | No | | DF 0<br>CK DescuentoPct>=0 AND DescuentoPct<=100 | Descuento porcentual del renglón. |

## 8.20. Factura

Comprobante fiscal emitido por un pedido (relación 1:1 GENERA). Filas en el juego de datos final: 8.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| FacturaId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| PedidoId | INT | No | FK | FK → Pedido<br>UQ | Pedido facturado (1:1). |
| TipoComprobante | CHAR(1) | No | | UQ (TipoComprobante, PuntoVenta, NroComprobante)<br>CK TipoComprobante IN ('A', 'B', 'C') | Tipo A, B o C. |
| PuntoVenta | SMALLINT | No | | CK PuntoVenta>=1 AND PuntoVenta<=9999 | Punto de venta fiscal. |
| NroComprobante | INT | No | | CK NroComprobante>0 | Número correlativo por tipo y punto de venta. |
| FechaEmision | DATE | No | | DF fecha actual | Fecha de emisión. |

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| Neto | DECIMAL(14,2) | No | | CK Neto>=0 | Importe neto = total del detalle (RN17). |
| AlicuotaIva | DECIMAL(5,2) | No | | DF 21<br>CK Alicuotalva=21 OR Alicuotalva=10.5 OR Alicuotalva=0 | Alícuota de IVA vigente. |
| TotalFactura | DECIMAL(16,4) | Sí | | Calculada PERSISTED: Neto*(1+Alicuotalva/100.0) | Neto más IVA (columna calculada). |

## *8.21. MedioPago*

Medios de pago admitidos. Filas en el juego de datos final: 5.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| MedioPagoId | INT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| Nombre | NVARCHAR(40) | No | | UQ | Denominación. |
| RecargoPct | DECIMAL(5,2) | No | | DF 0<br>CK RecargoPct>=0 AND RecargoPct<=100 | Recargo porcentual del medio. |
| Activo | BIT | No | | DF 1 | Indicador de vigencia (1 = activo); se usa en lugar de borrar. |

## *8.22. Pago*

Pagos aplicados a una factura (entidad débil de Factura). Filas en el juego de datos final: 9.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| FacturaId | INT | No | PK, FK | FK → Factura CASCADE | Factura propietaria. |
| NroPago | TINYINT | No | PK | — | Número de pago dentro de la factura. |
| FechaPago | DATE | No | | DF fecha actual | Fecha del pago. |
| Monto | DECIMAL(14,2) | No | | CK Monto>0 | Importe abonado (RN16). |
| MedioPagoId | INT | No | FK | FK → MedioPago | Medio utilizado. |
| Referencia | NVARCHAR(40) | Sí | | — | Número de operación, cupón o cheque. |

## *8.23. PedidoEstadoHistorial*

Auditoría de altas y cambios de estado de los pedidos (incorporada en AE4). Filas en el juego de datos final: 9.

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| HistorialId | BIGINT | No | PK | IDENTITY(1,1) | Identificador sustituto. |
| PedidoId | INT | No | FK | FK → Pedido CASCADE | Pedido auditado. |
| EstadoAnterior | NVARCHAR(12) | Sí | | CK EstadoAnterior IS NULL OR EstadoAnterior IN ('PENDIENTE', 'CONFIRMADO', 'ENTREGADO', 'CANCELADO')<br>CK EstadoAnterior IS NULL OR EstadoAnterior<>EstadoNuevo | Estado previo; NULL en el alta. |
| EstadoNuevo | NVARCHAR(12) | No | | CK EstadoNuevo IN ('PENDIENTE', 'CONFIRMADO', 'ENTREGADO', 'CANCELADO') | Estado resultante. |
| FechaHora | DATETIME2(0) | No | | DF fecha y hora actual | Momento del cambio. |

| Atributo | Tipo | Nulo | Clave | Restricciones | Descripción |
|---|---|---|---|---|---|
| Usuario | NVARCHAR(128) | No | | DF login actual | Login del SGBD que realizó el cambio. |

# 9. Implementación de la base de datos

## 9.1. Entorno

La base se implementó en Microsoft SQL Server 2022 Express Edition (versión 16.0.1000.6, 64 bits) sobre Windows, con la intercalación Modern_Spanish_CI_AI a nivel de base: las búsquedas no distinguen mayúsculas ni acentos, de modo que «monitor», «MONITOR» y «mónitor» se consideran iguales. Los scripts pueden ejecutarse desde SQL Server Management Studio (SSMS) o desde la utilidad de línea de comandos sqlcmd.

## 9.2. Organización y ejecución del script

El código se organizó en ocho módulos numerados que se ejecutan en orden; el script integral se genera concatenándolos (`AE4/sql/build_script_integral.ps1`), lo que garantiza que el archivo entregado coincide con los módulos probados. El primer módulo elimina la base StockFlow si existe y la vuelve a crear, por lo que el script es **idempotente**: puede ejecutarse repetidamente y siempre parte de una base vacía.

**Tabla 13.** *Módulos del script integral y orden de ejecución.*

| Orden | Módulo | Contenido | Líneas |
|---|---|---|---|
| 1 | 01_ddl.sql | CREATE DATABASE, 23 tablas, restricciones, índices y 6 vistas de derivados | 617 |
| 2 | 02_datos_prueba.sql | Datos de prueba de las 23 tablas y recuento de filas | 329 |
| 3 | 03_pruebas_integridad.sql | Pruebas T01–T26 y consultas V1–V8 de la AE2 | 341 |
| 4 | 04_programacion.sql | Funciones F1–F3, triggers T1–T5 y procedimientos P1–P5 | 689 |
| 5 | 05_vistas.sql | Vistas de explotación de la AE4 | 84 |
| 6 | 06_manipulacion_dml.sql | Operaciones DML M1–M6 y transacciones | 250 |
| 7 | 07_consultas.sql | Consultas C01–C20 | 292 |
| 8 | 08_pruebas_programacion.sql | 48 pruebas de los objetos programados | 598 |

Las pruebas de integridad declarativa (módulo 3) se ejecutan antes de crear los triggers para que cada restricción se verifique de forma aislada, tal como en la AE2. Para ejecutar el script completo desde la línea de comandos se utiliza:

```
sqlcmd -S .\SQLEXPRESS -E -b -f 65001 -i AE4_StockFlow_script_integral.sql -o ejecucion.log
```

La opción `-b` detiene la ejecución ante el primer error y devuelve un código de salida distinto de cero; `-f 65001` fija la codificación UTF-8. Cada módulo comienza con `SET ANSI_NULLS ON` y `SET QUOTED_IDENTIFIER ON`, configuración que SQL Server exige para crear índices filtrados y columnas calculadas y que sqlcmd no activa por defecto (véase la Tabla 24).

## 9.3. Objetos creados

El recuento de la Tabla 14 se obtuvo del catálogo del sistema después de ejecutar el script integral.

**Tabla 14.** *Objetos de la base de datos StockFlow.*

| Objeto | Cantidad | Observación |
|---|---|---|
| Tablas | 23 | 22 del modelo de la AE2 y PedidoEstadoHistorial (AE4) |
| Claves primarias | 23 | 9 compuestas (entidades débiles y relaciones N:M) |
| Claves foráneas | 31 | 9 con ON DELETE CASCADE; 2 compuestas (Movimiento→Stock, Pedido→ClienteDireccion) |
| Restricciones UNIQUE | 15 | Claves alternativas: SKU, CUIT, documento, legajo, numeración fiscal, etc. |
| Índices únicos filtrados | 5 | Precio vigente, proveedor preferido, dirección principal y correos opcionales |
| Índices no únicos | 22 | Sobre claves foráneas y columnas de búsqueda frecuente |
| Restricciones CHECK | 47 | Dominios, estados, rangos y reglas multicolumna |
| Valores por defecto | 29 | Fechas de alta, estados iniciales, indicadores |
| Columnas calculadas | 1 | Factura.TotalFactura (PERSISTED) |
| Vistas | 9 | 6 de atributos derivados (AE2) y 3 de explotación (AE4) |
| Funciones escalares | 3 | fn_PrecioVigente, fn_StockDisponible, fn_TotalCompradoCliente |
| Procedimientos almacenados | 5 | Todos transaccionales, con parámetros y controles |
| Triggers | 5 | Validación, automatización y auditoría |

## 9.4. Restricciones declarativas destacadas

Cinco reglas que abarcan varias filas se resolvieron con **índices únicos filtrados**, sin necesidad de triggers. El caso del correo electrónico es ilustrativo: SQL Server trata los NULL como iguales en una restricción UNIQUE, por lo que `UNIQUE(Email)` sobre una columna opcional impediría tener dos clientes sin correo; el índice filtrado aplica la unicidad solo a las filas con valor.

```
CREATE UNIQUE INDEX UX_ProductoPrecio_Vigente
    ON dbo.ProductoPrecio (ProductoId)

CREATE UNIQUE INDEX UX_Cliente_Email ON dbo.Cliente (Email) WHERE Email IS NOT NULL;
```

La tabla Pedido concentra tres decisiones: la clave foránea compuesta solapada hacia ClienteDireccion, el CHECK condicional que completa esa clave cuando NroDireccion es nulo (comportamiento MATCH SIMPLE del estándar) y la restricción de estados.

```
CREATE TABLE dbo.Pedido (
    PedidoId     INT          NOT NULL IDENTITY(1,1),
    NroPedido    NVARCHAR(16) NOT NULL,
    ClienteId    INT          NOT NULL,
    NroDireccion TINYINT      NULL,
    EmpleadoId   INT          NOT NULL,
    SucursalId   INT          NOT NULL,
    FechaPedido  DATETIME2(0) NOT NULL CONSTRAINT DF_Pedido_Fecha DEFAULT (SYSDATETIME()),
    Estado       NVARCHAR(12) NOT NULL CONSTRAINT DF_Pedido_Estado DEFAULT ('PENDIENTE'),
    Canal        NVARCHAR(12) NOT NULL,
```

```
    CONSTRAINT PK_Pedido     PRIMARY KEY (PedidoId),
    CONSTRAINT UQ_Pedido_Nro UNIQUE (NroPedido),
    CONSTRAINT FK_Pedido_Cliente FOREIGN KEY (ClienteId)
        REFERENCES dbo.Cliente (ClienteId) ON DELETE NO ACTION,
    /* FK compuesta solapada: la direccion de envio tiene que ser una direccion
       DE ESE cliente. La columna ClienteId participa de las dos FK, de modo que
       el motor hace imposible enviar al domicilio de otro cliente. */
    CONSTRAINT FK_Pedido_ClienteDireccion FOREIGN KEY (ClienteId, NroDireccion)
        REFERENCES dbo.ClienteDireccion (ClienteId, NroDireccion) ON DELETE NO ACTION,
    CONSTRAINT FK_Pedido_Empleado FOREIGN KEY (EmpleadoId)
        REFERENCES dbo.Empleado (EmpleadoId) ON DELETE NO ACTION,
    CONSTRAINT FK_Pedido_Sucursal FOREIGN KEY (SucursalId)
        REFERENCES dbo.Sucursal (SucursalId) ON DELETE NO ACTION,
    CONSTRAINT CK_Pedido_Estado CHECK (Estado IN ('PENDIENTE','CONFIRMADO','ENTREGADO','CANCELADO')),
    CONSTRAINT CK_Pedido_Canal  CHECK (Canal IN ('MOSTRADOR','WEB','TELEFONO')),
    /* Un pedido de mostrador se retira; cualquier otro canal exige domicilio.
       Nota: con NroDireccion NULL la FK compuesta no se verifica (MATCH SIMPLE),
       por eso la regla se completa con este CHECK. */
    CONSTRAINT CK_Pedido_Envio CHECK (
        (Canal = 'MOSTRADOR' AND NroDireccion IS NULL)
     OR (Canal <> 'MOSTRADOR' AND NroDireccion IS NOT NULL))
);
```

La tabla de auditoría incorporada en la AE4 se define de la siguiente manera; los valores por defecto registran automáticamente el instante y el login del SGBD que provocó el cambio.

```
CREATE TABLE dbo.PedidoEstadoHistorial (
    HistorialId    BIGINT        NOT NULL IDENTITY(1,1),
    PedidoId       INT           NOT NULL,
    EstadoAnterior NVARCHAR(12)  NULL,
    EstadoNuevo    NVARCHAR(12)  NOT NULL,
    FechaHora      DATETIME2(0)  NOT NULL CONSTRAINT DF_PedHist_Fecha   DEFAULT (SYSDATETIME()),
    Usuario        NVARCHAR(128) NOT NULL CONSTRAINT DF_PedHist_Usuario DEFAULT (SUSER_SNAME()),
    CONSTRAINT PK_PedidoEstadoHistorial PRIMARY KEY (HistorialId),
    /* El historial es parte del pedido: si un pedido se elimina (solo posible
       si no esta facturado), su historial se elimina con el. */
    CONSTRAINT FK_PedHist_Pedido FOREIGN KEY (PedidoId)
        REFERENCES dbo.Pedido (PedidoId) ON DELETE CASCADE,
    CONSTRAINT CK_PedHist_EstadoAnt CHECK (EstadoAnterior IS NULL OR
        EstadoAnterior IN ('PENDIENTE','CONFIRMADO','ENTREGADO','CANCELADO')),
    CONSTRAINT CK_PedHist_EstadoNvo CHECK (
        EstadoNuevo IN ('PENDIENTE','CONFIRMADO','ENTREGADO','CANCELADO')),
    CONSTRAINT CK_PedHist_Cambio CHECK (EstadoAnterior IS NULL OR EstadoAnterior <> EstadoNuevo)
);
```

# 10. Carga y manipulación de datos

## 10.1. Datos de prueba

El juego de datos de la AE2 se conservó y se amplió con casos límite necesarios para ejercitar los objetos de la AE4: un producto discontinuado con precio vigente (IM-SAM-003), un producto activo sin precio ni stock (WC-LOG-004), una clienta sin correo, teléfono ni pedidos (Rocío Benítez) y una empresa con CUIT que permite verificar la emisión automática de factura A. Los importes del detalle, los netos de las facturas y los pagos se construyeron para que cierren entre sí. La Tabla 15 muestra el volumen después de la carga inicial y al finalizar las operaciones de manipulación.

**Tabla 15.** *Filas por tabla tras la carga (módulo 02) y al final de la ejecución (módulo 08).*

| Tabla | Carga inicial | Estado final | Tabla | Carga inicial | Estado final |
|---|---|---|---|---|---|
| Sucursal | 3 | 3 | OrdenCompra | 4 | 4 |
| Empleado | 8 | 8 | OrdenCompraDetalle | 8 | 8 |
| Categoria | 7 | 7 | Cliente | 7 | 7 |
| Marca | 6 | 6 | ClienteTelefono | 7 | 7 |
| Producto | 14 | 14 | ClienteDireccion | 7 | 8 |
| ProductoPrecio | 17 | 19 | Pedido | 5 | 9 |
| Deposito | 4 | 4 | PedidoDetalle | 10 | 19 |
| Stock | 21 | 21 | Factura | 4 | 8 |
| MovimientoStock | 10 | 26 | MedioPago | 5 | 5 |
| Proveedor | 4 | 4 | Pago | 4 | 9 |
| ProveedorTelefono | 7 | 6 | PedidoEstadoHistorial | 0 | 9 |
| ProveedorProducto | 15 | 14 | | | |

Entre los casos que el juego de datos permite verificar se destacan: un producto con dos proveedores a distintos precios (posible gracias a la relación N:M), productos bajo el mínimo en algún depósito, facturas cobradas en dos pagos con medios distintos, cobradas parcialmente o sin cobrar, pedidos de mostrador sin dirección y productos vendidos a un precio distinto del vigente actualmente.

## 10.2. Operaciones de manipulación

El módulo 06 aplica sobre los datos de prueba las operaciones de la Tabla 16. Las ventas se registran exclusivamente a través de los procedimientos almacenados, que es el modo en que operaría la aplicación.

**Tabla 16.** *Operaciones DML del módulo 06.*

| Id | Operación | Sentencias y objetos involucrados |
|---|---|---|
| M1.1 | Alta de teléfono y dirección de la clienta 6 | INSERT en ClienteTelefono y ClienteDireccion |
| M1.2 | Actualización del correo del cliente 4 y del mínimo/máximo de un producto en un depósito | UPDATE en Cliente y Stock (columnas no controladas por el kardex) |

| Id | Operación | Sentencias y objetos involucrados |
|---|---|---|
| M1.3 | Baja de un teléfono laboral y de un acuerdo de suministro vencido | DELETE en ClienteTelefono y ProveedorProducto |
| M2 | Recepción de la OC-2026-0003 en el depósito de Oberá | Transacción explícita: INGRESO en kardex (T1 suma stock), CantidadRecibida y estado RECIBIDA |
| M3 | Transferencia de 10 teclados del Depósito Central al Salón | Transacción explícita: TRANSFER (salida) + INGRESO (entrada) |
| M4 | Actualización de precios de dos productos | sp_ActualizarPrecioProducto (cierra el período vigente y abre uno nuevo) |
| M5.1 | Venta web: pedido, factura B, pago total y entrega | sp_RegistrarPedido, sp_FacturarPedido, sp_RegistrarPago, UPDATE Estado (T3) |
| M5.2 | Venta telefónica a empresa con descuento y cobranza parcial | Idem; factura A deducida del CUIT |
| M5.3 | Venta de mostrador en Oberá con pago en efectivo | Idem; punto de venta 3; deja un producto bajo el mínimo |
| M5.4 | Pedido telefónico anulado por el cliente | sp_RegistrarPedido y sp_CancelarPedido (reversión de stock) |
| M6 | Cierre de pedidos heredados de la AE2 | Confirmación, facturación, cobro y entrega con los procedimientos |

## 10.3. Transacciones explícitas

La recepción de una orden de compra modifica tres tablas que deben cambiar juntas. Se encapsula en una transacción con manejo de errores: si cualquiera de las sentencias falla, la instrucción ROLLBACK deja la base en su estado anterior.

```
BEGIN TRY
    BEGIN TRANSACTION;

    DECLARE @Oc INT = (SELECT OrdenCompraId FROM dbo.OrdenCompra WHERE NroOrden = N'OC-2026-0003');

    INSERT INTO dbo.MovimientoStock (ProductoId, DepositoId, TipoMovimiento, Cantidad, EmpleadoId,
                                     Motivo, DocumentoTipo, DocumentoId)
    SELECT d.ProductoId, oc.DepositoId, 'INGRESO', d.Cantidad, 6, N'Recepcion OC-2026-0003', 'OC',
oc.OrdenCompraId
    FROM dbo.OrdenCompraDetalle AS d
    JOIN dbo.OrdenCompra AS oc ON oc.OrdenCompraId = d.OrdenCompraId
    WHERE d.OrdenCompraId = @Oc;

    UPDATE dbo.OrdenCompraDetalle SET CantidadRecibida = Cantidad WHERE OrdenCompraId = @Oc;
    UPDATE dbo.OrdenCompra SET Estado = 'RECIBIDA' WHERE OrdenCompraId = @Oc;

    COMMIT TRANSACTION;
    PRINT 'M2 COMMIT: orden recibida y stock actualizado por el trigger de kardex.';
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    PRINT 'M2 ROLLBACK: ' + ERROR_MESSAGE();
END CATCH;
```

El efecto se verifica con el estado del stock antes y después de la transacción (registro de ejecución):

| Momento | ProductoId | DepositoId | Cantidad |
|---|---|---|---|
| M2 antes | 9 | 4 | 2 |
| M2 antes | 10 | 4 | 18 |

| Momento | ProductoId | DepositoId | Cantidad |
|---|---|---|---|
| M2 despues | 9 | 4 | 8 |
| M2 despues | 10 | 4 | 48 |

La anulación de un pedido (M5.4) muestra el ciclo completo del kardex: el stock disminuye al registrar el pedido y se repone al anularlo.

| ProductoId | Antes del pedido | Con el pedido registrado | Después de anular |
|---|---|---|---|
| 2 | 11 | 10 | 11 |
| 7 | 140 | 130 | 140 |
| 8 | 58 | 54 | 58 |

La actualización de precios (M4) conserva el período anterior cerrado y abre uno nuevo, sin sobrescribir valores:

| ProductoId | FechaDesde | FechaHasta | PrecioVenta | PrecioCosto |
|---|---|---|---|---|
| 3 | 2026-01-02 | 2026-11-11 | 11.500,00 | 7.200,00 |
| 3 | 2026-11-12 | NULL | 12.900,00 | 8.100,00 |
| 5 | 2026-01-02 | 2026-11-11 | 198.000,00 | 142.000,00 |
| 5 | 2026-11-12 | NULL | 214.000,00 | 142.000,00 |

# 11. Consultas SQL y vistas

## 11.1. Catálogo de consultas

Las veinte consultas del módulo 07 responden a requerimientos concretos del sistema y cubren las técnicas que exige la consigna. La Tabla 17 las relaciona con su requerimiento; el código completo y el resultado de cada una se presentan en el Anexo B.

**Tabla 17.** *Consultas, técnicas SQL utilizadas y requerimientos que satisfacen.*

| Id | Consulta | Técnica | Requerimiento |
|---|---|---|---|
| C01 | Catálogo de periféricos activos | Simple, subconsulta en WHERE, ORDER BY | RI02 / RF08 |
| C02 | Clientes sin correo electrónico | IS NULL, ISNULL | RI01 |
| C03 | Detalle de ventas: quién compró, qué, quién vendió y dónde | INNER JOIN de 6 tablas | RI06, RI07 |
| C04 | Productos sin precio vigente o sin stock | LEFT JOIN, COALESCE, NULL | RI02 / RF08 |
| C05 | Indicadores globales de ventas | COUNT, SUM, AVG, MIN, MAX | RI15 / RF11 |
| C06 | Ventas por sucursal y canal | GROUP BY de dos columnas | RI15 / RF11 |
| C07 | Categorías con venta neta superior a $1.000.000 | GROUP BY, HAVING, autorrelación | RI15 / RF11 |
| C08 | Clientes con más de un pedido | GROUP BY, HAVING COUNT | RI15 / RF11 |
| C09 | Productos con precio superior al promedio | Subconsulta escalar | RI02, RI09 |
| C10 | Productos activos nunca vendidos | NOT EXISTS, función | RI02, RI07 |
| C11 | Proveedor preferido de los productos bajo mínimo | Subconsulta IN sobre vista | RI08 / RF09 |
| C12 | Último pedido de cada cliente | Subconsulta correlacionada | RI06 |
| C13 | Cuentas a cobrar con teléfono de contacto | EXISTS, GROUP BY sobre vista | RI12 / RF10 |
| C14 | Precio cobrado frente a precio de lista | Función fn_PrecioVigente | RI07, RI09 |
| C15 | Disponibilidad por depósito | Función fn_StockDisponible, pivoteo | RI04 |
| C16 | Ranking de productos por venta neta | RANK() y SUM() OVER, CTE | RI15 / RF11 |
| C17 | Trazabilidad de un pedido anulado | JOIN con auditoría y kardex | RI10, RI14 / RF12 |
| C18 | Catálogo y cartera de clientes | Consulta sobre vistas | RF08, RF10 |
| C19 | Anular órdenes en borrador con más de 30 días | UPDATE con subconsulta | RI11 / RF13 |
| C20 | Eliminar faxes de proveedores con teléfono principal | DELETE con EXISTS | RI05 / RF13 |

## 11.2. Ejemplos comentados

### *Agrupamiento con condición sobre grupos (C07)*

HAVING filtra grupos después de agregar, mientras que WHERE filtra filas antes de hacerlo. La consulta suma la venta neta por categoría, excluye pedidos cancelados (WHERE) y conserva solo las categorías que superan el millón de pesos (HAVING). La autorrelación de Categoria permite informar el rubro padre.

```
SELECT 'C07' AS Consulta, ISNULL(cp.Nombre, c.Nombre) AS Rubro, c.Nombre AS Categoria,
       COUNT(DISTINCT pd.PedidoId) AS Pedidos, SUM(pd.Cantidad) AS Unidades,
```

```
       SUM(CAST(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0) AS DECIMAL(16,2)))
AS VentaNeta
FROM dbo.PedidoDetalle AS pd
JOIN dbo.Pedido    AS p  ON p.PedidoId    = pd.PedidoId
JOIN dbo.Producto  AS pr ON pr.ProductoId = pd.ProductoId
JOIN dbo.Categoria AS c  ON c.CategoriaId = pr.CategoriaId
LEFT JOIN dbo.Categoria AS cp ON cp.CategoriaId = c.CategoriaPadreId
WHERE p.Estado <> 'CANCELADO'
GROUP BY cp.Nombre, c.Nombre
HAVING SUM(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0)) > 1000000
ORDER BY VentaNeta DESC;
```

| Rubro | Categoria | Pedidos | Unidades | VentaNeta |
|---|---|---|---|---|
| Informatica | Notebooks | 2 | 3 | 3.150.000,00 |
| Informatica | Perifericos | 6 | 16 | 1.361.300,00 |
| Informatica | Almacenamiento | 3 | 10 | 1.041.000,00 |

### *Tratamiento de valores nulos (C04)*

El LEFT JOIN conserva los productos que no tienen precio vigente ni filas de stock; las columnas de la tabla externa llegan como NULL y COALESCE las presenta de forma legible. Con un INNER JOIN esos productos —justamente los que se buscan— desaparecerían del resultado.

```
SELECT 'C04' AS Consulta, p.Sku, p.Nombre,
       COALESCE(CONVERT(NVARCHAR(20), pv.PrecioVenta), N'SIN PRECIO') AS PrecioVigente,
       COALESCE(SUM(s.Cantidad), 0) AS StockTotal,
       CASE WHEN p.Activo = 1 THEN 'ACTIVO' ELSE 'DISCONTINUADO' END AS Situacion
FROM dbo.Producto AS p
LEFT JOIN dbo.vw_ProductoPrecioVigente AS pv ON pv.ProductoId = p.ProductoId
LEFT JOIN dbo.Stock AS s ON s.ProductoId = p.ProductoId
GROUP BY p.Sku, p.Nombre, pv.PrecioVenta, p.Activo
HAVING pv.PrecioVenta IS NULL OR COALESCE(SUM(s.Cantidad), 0) = 0;
```

| Sku | Nombre | PrecioVigente | StockTotal | Situacion |
|---|---|---|---|---|
| WC-LOG-004 | Webcam Logitech C920 | SIN PRECIO | 0 | ACTIVO |
| IM-SAM-003 | Impresora Samsung Xpress M2020 | 310.000,00 | 0 | DISCONTINUADO |

### *Uso de funciones del sistema (C14)*

La consulta compara el precio cobrado en cada renglón con el precio de lista del día de la venta y con el vigente hoy, ambos obtenidos con `fn_PrecioVigente`. Es la evidencia de RN06: los aumentos posteriores no alteran lo pactado.

```
SELECT 'C14' AS Consulta, p.NroPedido, CAST(p.FechaPedido AS DATE) AS Fecha, pr.Sku,
       pd.PrecioUnitarioHistorico AS PrecioCobrado,
       dbo.fn_PrecioVigente(pd.ProductoId, CAST(p.FechaPedido AS DATE)) AS PrecioListaEseDia,
       dbo.fn_PrecioVigente(pd.ProductoId, NULL) AS PrecioListaHoy
FROM dbo.PedidoDetalle AS pd
JOIN dbo.Pedido   AS p  ON p.PedidoId    = pd.PedidoId
JOIN dbo.Producto AS pr ON pr.ProductoId = pd.ProductoId
WHERE pd.PrecioUnitarioHistorico <> dbo.fn_PrecioVigente(pd.ProductoId, NULL)
ORDER BY p.NroPedido;
```

| NroPedido | Fecha | Sku | PrecioCobrado | PrecioListaEseDia | PrecioListaHoy |
|---|---|---|---|---|---|
| PED-2026-0001 | 2026-09-05 | MN-SAM-001 | 198.000,00 | 198.000,00 | 214.000,00 |
| PED-2026-0002 | 2026-09-08 | MO-LOG-001 | 11.500,00 | 11.500,00 | 12.900,00 |
| PED-2026-0003 | 2026-09-15 | MN-SAM-001 | 198.000,00 | 198.000,00 | 214.000,00 |
| PED-2026-0003 | 2026-09-15 | SS-KIN-001 | 112.000,00 | 112.000,00 | 129.000,00 |

### *Funciones de ventana (C16)*

```
SELECT 'C16' AS Consulta,
       RANK() OVER (ORDER BY v.VentaNeta DESC) AS Puesto,
       v.Sku, v.Unidades, v.VentaNeta,
       CAST(100.0 * v.VentaNeta / SUM(v.VentaNeta) OVER () AS DECIMAL(5,2)) AS PorcentajeDelTotal
FROM (SELECT pr.Sku, SUM(pd.Cantidad) AS Unidades,
             SUM(CAST(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0) AS
DECIMAL(16,2))) AS VentaNeta
      FROM dbo.PedidoDetalle AS pd
      JOIN dbo.Pedido   AS p  ON p.PedidoId    = pd.PedidoId
      JOIN dbo.Producto AS pr ON pr.ProductoId = pd.ProductoId
      WHERE p.Estado <> 'CANCELADO'
      GROUP BY pr.Sku) AS v
ORDER BY Puesto;
```

| Puesto | Sku | Unidades | VentaNeta | PorcentajeDelTotal |
|---|---|---|---|---|
| 1 | NB-LEN-001 | 3 | 3.150.000,00 | 50,63 |
| 2 | MN-SAM-001 | 6 | 1.196.400,00 | 19,23 |
| 3 | SS-KIN-001 | 7 | 835.000,00 | 13,42 |
| 4 | VT-LIL-001 | 7 | 584.800,00 | 9,40 |
| 5 | RM-KIN-003 | 2 | 148.000,00 | 2,38 |
| 6 | MO-LOG-001 | 9 | 106.850,00 | 1,72 |

*(... 3 filas más en el registro de ejecución)*

## 11.3. Vistas

Las vistas encapsulan cálculos y uniones de uso frecuente, de modo que la aplicación y los reportes consultan información derivada sin repetir la lógica ni almacenar datos redundantes.

**Tabla 18.** *Vistas de la base de datos.*

| Vista | Origen | Información que provee |
|---|---|---|
| vw_ProductoPrecioVigente | AE2 | Precio vigente de cada producto (atributo derivado PrecioVigente). |
| vw_ProductoStock | AE2 | Stock total por producto (atributo derivado StockTotal). |
| vw_DepositoOcupacion | AE2 | Ocupación de cada depósito frente a su capacidad. |
| vw_PedidoTotal | AE2 | Total neto de cada pedido con descuentos. |
| vw_FacturaSaldo | AE2 | Cobrado y saldo pendiente de cada factura. |
| vw_StockBajoMinimo | AE2 | Pares producto-depósito por debajo del mínimo (RI08). |
| vw_CatalogoProductos | AE4 | Catálogo con categoría, marca, precio vigente, stock total y proveedor preferido; base de la aplicación web. |
| vw_StockPorDeposito | AE4 | Existencias por depósito con semáforo SIN STOCK / BAJO MINIMO / OK. |
| vw_ResumenVentasCliente | AE4 | Cartera de clientes: pedidos, total comprado, última compra y saldo pendiente. |

La vista `vw_ResumenVentasCliente` evoluciona la vista del borrador: utiliza LEFT JOIN para incluir clientes sin pedidos, excluye los pedidos cancelados, emplea `fn_TotalCompradoCliente` y agrega el saldo pendiente a partir de `vw_FacturaSaldo`.

```
CREATE OR ALTER VIEW dbo.vw_ResumenVentasCliente AS
SELECT c.ClienteId,
       CONCAT(c.Apellido, N', ', c.Nombre)        AS Cliente,
       CONCAT(c.TipoDoc, N' ', c.NroDoc)          AS Documento,
       ISNULL(c.Email, N'(sin email)')            AS Email,
       COUNT(p.PedidoId)                          AS CantidadPedidos,
       dbo.fn_TotalCompradoCliente(c.ClienteId)   AS TotalComprado,
       MAX(p.FechaPedido)                         AS UltimaCompra,
       ISNULL((SELECT SUM(fs.SaldoPendiente)
               FROM dbo.vw_FacturaSaldo AS fs
               JOIN dbo.Factura AS f  ON f.FacturaId = fs.FacturaId
               JOIN dbo.Pedido  AS p2 ON p2.PedidoId = f.PedidoId
               WHERE p2.ClienteId = c.ClienteId), 0) AS SaldoPendiente
FROM dbo.Cliente AS c
LEFT JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId AND p.Estado <> 'CANCELADO'
GROUP BY c.ClienteId, c.Apellido, c.Nombre, c.TipoDoc, c.NroDoc, c.Email;
GO

PRINT 'Vistas AE4 creadas: vw_CatalogoProductos, vw_StockPorDeposito, vw_ResumenVentasCliente.';
```

| Cliente | Documento | Email | CantidadPedidos | TotalComprado | UltimaCompra | SaldoPendiente |
|---|---|---|---|---|---|---|
| Gomez, Martin | DNI 32145678 | mgomez@mail.com | 2 | 2.700.900,00 | 2026-11-12 09:15:47 | 0,00 |
| del Litoral SA, Tecnologias | CUIT 30715559991 | compras@tecnolitoral.com.ar | 1 | 1.108.050,00 | 2026-11-12 09:15:47 | 670.370,25 |
| Sosa, Lucia | DNI 27998877 | lsosa@mail.com | 2 | 1.049.850,00 | 2026-11-12 09:15:47 | 0,00 |
| Contable RS, Estudio | CUIT 20345678901 | admin@estudiors.com.ar | 1 | 962.300,00 | 2026-09-15 10:12:00 | 698.629,80 |
| Paredes, Nicolas | DNI 40112233 | nparedes@mail.com | 1 | 342.000,00 | 2026-09-24 12:45:00 | 213.820,00 |
| Kowalski, Helena | PAS AB1234567 | hkowalski@mail.com | 1 | 58.000,00 | 2026-09-29 19:02:00 | 40.180,00 |
| Benitez, Rocio | DNI 35111222 | (sin email) | 0 | 0,00 | NULL | 0,00 |

# 12. Programación en la base de datos

## 12.1. Criterios de diseño

- **Una regla, un lugar.** Cada objeto programado resuelve una regla de negocio que el DDL no puede expresar (Tabla 8) o una operación que debe ser atómica.
- **Procedimientos como única vía de escritura de las operaciones de negocio.** La aplicación no inserta pedidos ni facturas directamente: invoca procedimientos que validan el contexto y ejecutan todos los pasos en una transacción.
- **Triggers como red de seguridad.** Las reglas críticas (stock, saldo de facturas, estados, historial) se verifican también en triggers, de modo que rigen aunque alguien ejecute un INSERT o UPDATE manual fuera de la aplicación.
- **Errores identificables.** Los procedimientos informan con THROW códigos 50001–50099 y los triggers 50100–50199, con mensajes en lenguaje natural que la aplicación muestra al usuario. Las pruebas verifican el código exacto, no solo que la operación falle.
- **Orientación a conjuntos.** Los triggers operan sobre las tablas `inserted` y `deleted` completas, por lo que funcionan con sentencias que afectan muchas filas.

## 12.2. Funciones

**Tabla 19.** *Funciones escalares.*

| Función | Parámetros → retorno | Propósito | Utilizada en |
|---|---|---|---|
| fn_PrecioVigente | @ProductoId INT, @Fecha DATE → DECIMAL(12,2) | Precio de venta vigente en una fecha (NULL = hoy). Devuelve NULL si no había precio. | sp_RegistrarPedido, C14, pruebas F1 |
| fn_StockDisponible | @ProductoId INT, @DepositoId INT → DECIMAL(12,3) | Existencia en un depósito o total (depósito NULL). Nunca devuelve NULL. | vw_CatalogoProductos, C10, C15 |
| fn_TotalCompradoCliente | @ClienteId INT → DECIMAL(16,2) | Importe neto comprado, con descuentos y sin pedidos cancelados. | vw_ResumenVentasCliente |

```
CREATE OR ALTER FUNCTION dbo.fn_PrecioVigente (@ProductoId INT, @Fecha DATE)
RETURNS DECIMAL(12,2)
AS
BEGIN
    DECLARE @Precio DECIMAL(12,2);
    DECLARE @F DATE = ISNULL(@Fecha, CAST(SYSDATETIME() AS DATE));

    SELECT TOP (1) @Precio = pp.PrecioVenta
    FROM dbo.ProductoPrecio AS pp
    WHERE pp.ProductoId = @ProductoId
      AND pp.FechaDesde <= @F
      AND (pp.FechaHasta IS NULL OR pp.FechaHasta >= @F)
    ORDER BY pp.FechaDesde DESC;

    RETURN @Precio;
END;
```

## 12.3. Procedimientos almacenados

Los cinco procedimientos siguen la misma estructura: (1) controles de parámetros y de contexto antes de abrir la transacción, para no bloquear recursos en operaciones que se rechazarán; (2) `SET XACT_ABORT ON`, que revierte la transacción ante cualquier error en tiempo de ejecución; (3) bloque `BEGIN TRY … BEGIN TRANSACTION … COMMIT` y, en el `CATCH`, `ROLLBACK` y relanzamiento del error con `THROW` para que el invocante conozca la causa.

**Tabla 20.** *Procedimientos almacenados, parámetros y controles.*

| Procedimiento | Parámetros | Controles (código de error) | Efecto |
|---|---|---|---|
| sp_RegistrarPedido | @ClienteId, @EmpleadoId, @SucursalId, @DepositoId, @Canal, @NroDireccion, @ItemsJson; salida @PedidoId, @NroPedido | Cliente activo (50001); rol de vendedor (50002); vendedor de la sucursal (50003); depósito de la sucursal (50004); canal y dirección (50005); JSON y al menos un renglón (50006); cantidades, descuentos y productos no repetidos (50007); producto activo (50008); precio vigente (50009); stock suficiente con bloqueo (50010) | Pedido CONFIRMADO con numeración PED-AAAA-NNNN, renglones con precio vigente y un EGRESO de kardex por renglón |
| sp_FacturarPedido | @PedidoId, @PuntoVenta, @TipoComprobante, @AlicuotaIva; salida @FacturaId | Pedido existente (50021); estado CONFIRMADO o ENTREGADO (50022); sin factura previa (50023); tipo válido (50024); con renglones (50025) | Factura con neto = total del detalle; tipo A si el cliente tiene CUIT, B en otro caso; número correlativo |
| sp_RegistrarPago | @FacturaId, @Monto, @MedioPagoId, @Referencia; salida @NroPago | Factura existente (50031); medio activo (50032); monto positivo (50033); saldo suficiente (T4, 50131) | Pago con número correlativo dentro de la factura |
| sp_ActualizarPrecioProducto | @ProductoId, @PrecioVenta, @PrecioCosto, @FechaDesde | Producto activo (50041); precios válidos (50042); sin vigencia futura (50043); precio distinto del vigente (50044); período vigente de al menos dos días (50045) | Cierra el período vigente y abre uno nuevo en una transacción |
| sp_CancelarPedido | @PedidoId, @EmpleadoId, @Motivo | Pedido existente (50051); estado PENDIENTE o CONFIRMADO (50052); sin factura (50053); empleado activo (50054) | INGRESO compensatorio por cada egreso y estado CANCELADO |

El procedimiento principal es `sp_RegistrarPedido`. Recibe el detalle como un arreglo JSON —el formato que envía la aplicación web— y lo procesa con OPENJSON. El fragmento siguiente corresponde a la parte transaccional; los controles previos se encuentran en el script.

```
    /* ---- Operacion atomica ---- */
    BEGIN TRY
        BEGIN TRANSACTION;

        /* Control de stock dentro de la transaccion y con bloqueo de
           actualizacion: dos ventas concurrentes no pueden vender la misma
           unidad. T1 vuelve a controlar como segunda linea de defensa. */
        SELECT TOP (1) @Msg = CONCAT('Stock insuficiente de ', p.Sku, ' en el deposito: disponible ',
                                     CAST(ISNULL(s.Cantidad, 0) AS DECIMAL(12,0)),
                                     ', solicitado ', CAST(i.Cantidad AS DECIMAL(12,0)), '.')
        FROM @Items AS i
        JOIN dbo.Producto AS p ON p.ProductoId = i.ProductoId
        LEFT JOIN dbo.Stock AS s WITH (UPDLOCK, HOLDLOCK)
               ON s.ProductoId = i.ProductoId AND s.DepositoId = @DepositoId
        WHERE ISNULL(s.Cantidad, 0) < i.Cantidad;
        IF @Msg IS NOT NULL THROW 50010, @Msg, 1;

        /* Numeracion correlativa por anio, serializada con UPDLOCK/HOLDLOCK. */
        DECLARE @Anio CHAR(4) = CAST(YEAR(@Hoy) AS CHAR(4));
        DECLARE @Siguiente INT = 1 + ISNULL((
            SELECT MAX(CAST(RIGHT(NroPedido, 4) AS INT))
            FROM dbo.Pedido WITH (UPDLOCK, HOLDLOCK)
            WHERE NroPedido LIKE 'PED-' + @Anio + '-%'), 0);
        SET @NroPedido = CONCAT('PED-', @Anio, '-', RIGHT(CONCAT('0000', @Siguiente), 4));
```

```
        INSERT INTO dbo.Pedido (NroPedido, ClienteId, NroDireccion, EmpleadoId, SucursalId, Estado, Canal)
        VALUES (@NroPedido, @ClienteId, @NroDireccion, @EmpleadoId, @SucursalId, 'CONFIRMADO', @Canal);
        SET @PedidoId = SCOPE_IDENTITY();

        INSERT INTO dbo.PedidoDetalle (PedidoId, ProductoId, Cantidad, PrecioUnitarioHistorico,
DescuentoPct)
        SELECT @PedidoId, ProductoId, Cantidad, Precio, DescuentoPct
        FROM @Items;

        INSERT INTO dbo.MovimientoStock (ProductoId, DepositoId, TipoMovimiento, Cantidad, EmpleadoId,
                                         Motivo, DocumentoTipo, DocumentoId)
        SELECT ProductoId, @DepositoId, 'EGRESO', Cantidad, @EmpleadoId,
               CONCAT('Venta ', @NroPedido), 'PEDIDO', @PedidoId
        FROM @Items;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
        SET @PedidoId = NULL;
        SET @NroPedido = NULL;
        THROW;
    END CATCH;
END;
```

Dos aspectos merecen destacarse. El control de stock se repite dentro de la transacción con las sugerencias de bloqueo `UPDLOCK, HOLDLOCK`, que mantienen bloqueadas las filas leídas hasta el COMMIT: dos ventas simultáneas no pueden vender la misma unidad. La misma técnica serializa la generación del número de pedido. Además, el procedimiento no actualiza Stock: inserta movimientos de kardex y es el trigger T1 el que descuenta las existencias, de modo que existe un único camino para modificar el stock.

## 12.4. Triggers

**Tabla 21.** *Triggers, evento que los dispara y regla que implementan.*

| Trigger | Tabla y evento | Tipo | Regla y comportamiento |
|---|---|---|---|
| T1 trg_MovimientoStock_AplicarKardex | MovimientoStock AFTER INSERT, UPDATE, DELETE | Automatización y validación | RN11: aplica cada movimiento sobre Stock (INGRESO suma; EGRESO, TRANSFER y AJUSTE restan). RN05: rechaza egresos sin existencia (50102). RN12: rechaza modificaciones y eliminaciones del kardex (50101). |
| T2 trg_Stock_ControlarExistencias | Stock AFTER INSERT, UPDATE | Control | RN11: rechaza cambios directos de Cantidad que no provengan de T1, detectados con TRIGGER_NESTLEVEL() (50111). RN07: rechaza superar la capacidad del depósito (50112). |
| T3 trg_Pedido_ControlarEstado | Pedido AFTER INSERT, UPDATE | Validación y auditoría | RN13: transiciones permitidas (50121). RN14: entregar exige factura (50122); no se cancela un pedido facturado (50123) ni con stock sin reponer (50124). RN15: registra alta y cambios en PedidoEstadoHistorial. |
| T4 trg_Pago_ControlarSaldo | Pago AFTER INSERT, UPDATE | Validación | RN16: la suma de pagos no puede superar el total de la factura (50131). |
| T5 trg_ProductoPrecio_ProtegerHistorial | ProductoPrecio AFTER UPDATE, DELETE | Integridad | RN10: impide borrar precios (50141) y modificar precios o períodos cerrados (50142); solo admite cerrar el período vigente. |

El trigger T1 es el núcleo de la coherencia entre stock y kardex. Agrupa los movimientos insertados por par producto-depósito, verifica que ningún saldo resulte negativo y aplica la variación neta con una única sentencia UPDATE:

```
CREATE OR ALTER TRIGGER dbo.trg_MovimientoStock_AplicarKardex
ON dbo.MovimientoStock
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    IF NOT EXISTS (SELECT 1 FROM inserted) AND NOT EXISTS (SELECT 1 FROM deleted)
        RETURN;

    IF EXISTS (SELECT 1 FROM deleted)
        THROW 50101, 'El kardex es inalterable: los movimientos de stock no se modifican ni se eliminan.
Registre un movimiento compensatorio.', 1;

    DECLARE @Delta TABLE (
        ProductoId INT NOT NULL,
        DepositoId INT NOT NULL,
        Delta      DECIMAL(12,3) NOT NULL,
        PRIMARY KEY (ProductoId, DepositoId));

    INSERT INTO @Delta (ProductoId, DepositoId, Delta)
    SELECT i.ProductoId, i.DepositoId,
           SUM(CASE WHEN i.TipoMovimiento = 'INGRESO' THEN i.Cantidad ELSE -i.Cantidad END)
    FROM inserted AS i
    GROUP BY i.ProductoId, i.DepositoId;

    DECLARE @Msg NVARCHAR(400);
    SELECT TOP (1) @Msg = CONCAT('Stock insuficiente: producto ', p.Sku, ' en ', dep.Nombre,
                                 ' tiene ', CAST(s.Cantidad AS DECIMAL(12,0)),
                                 ' unidades y el movimiento requiere ', CAST(-d.Delta AS DECIMAL(12,0)),
'.')
    FROM @Delta AS d
    JOIN dbo.Stock    AS s   ON s.ProductoId = d.ProductoId AND s.DepositoId = d.DepositoId
    JOIN dbo.Producto AS p   ON p.ProductoId = d.ProductoId
    JOIN dbo.Deposito AS dep ON dep.DepositoId = d.DepositoId
    WHERE s.Cantidad + d.Delta < 0;

    IF @Msg IS NOT NULL
        THROW 50102, @Msg, 1;

    UPDATE s
       SET s.Cantidad = s.Cantidad + d.Delta
    FROM dbo.Stock AS s
    JOIN @Delta AS d ON d.ProductoId = s.ProductoId AND d.DepositoId = s.DepositoId;
END;
```

El trigger T3 combina validación y auditoría. Como en SQL Server un THROW dentro de un trigger revierte la transacción completa, un cambio de estado inválido deshace también la sentencia que lo provocó; si es válido, la inserción en el historial forma parte de la misma transacción y no puede quedar un cambio sin auditar.

```
    /* 4. Auditoria: alta (EstadoAnterior NULL) o cambio efectivo de estado. */
    INSERT INTO dbo.PedidoEstadoHistorial (PedidoId, EstadoAnterior, EstadoNuevo)
    SELECT i.PedidoId, d.Estado, i.Estado
    FROM inserted AS i
    LEFT JOIN deleted AS d ON d.PedidoId = i.PedidoId
    WHERE d.PedidoId IS NULL OR d.Estado <> i.Estado;
END;
```

## 12.5. Transacciones

La consigna requiere al menos una transacción; el proyecto las utiliza en todas las operaciones que modifican más de una fila o tabla:

- Los cinco procedimientos almacenados (transacción explícita con TRY/CATCH y XACT_ABORT).

- Las transacciones explícitas M2 (recepción de orden de compra) y M3 (transferencia entre depósitos) del módulo 06.
- Los triggers, que se ejecutan dentro de la transacción de la sentencia que los dispara y la revierten con THROW.
- Las pruebas positivas, que se ejecutan dentro de una transacción que se revierte para no alterar los datos.

La atomicidad se verificó con la prueba P1-12: se envió un pedido cuyo primer renglón era válido y el segundo carecía de stock. El procedimiento rechazó la operación con el error 50010 y, después del rechazo, la cantidad de pedidos, de movimientos de kardex y la existencia del primer producto eran idénticas a las previas: no quedó ningún efecto parcial.

# 13. Pruebas y evidencias de funcionamiento

## 13.1. Estrategia

Las pruebas se automatizaron para que sean repetibles y su resultado no dependa de la interpretación de quien las ejecuta. Cada prueba negativa registra el código de error **esperado** y el **obtenido**: solo se considera correcta si la operación falla por la regla que se pretende probar. Este criterio surge de una lección de la AE2, donde una prueba de documento duplicado «pasaba» porque fallaba por un correo duplicado: una prueba que pasa no alcanza, hay que verificar por qué pasa. Las pruebas positivas que modifican datos se ejecutan dentro de una transacción que se revierte al final.

## 13.2. Ejecución del script integral

El script integral se ejecutó con sqlcmd sobre una instancia SQL Server 2022 Express en la que se eliminó previamente la base. La ejecución finalizó con código de salida 0, sin mensajes de error del motor, y produjo el registro `AE4/evidencias/ejecucion_script_integral.log` (474 líneas), del que se extraen todas las tablas de resultados de este informe. Su comienzo muestra la creación de la base y la carga de datos:

```
Changed database context to 'master'.
Changed database context to 'StockFlow'.
DDL aplicado correctamente: 23 tablas.
Changed database context to 'StockFlow'.
--- Sucursales ---
--- Empleados ---
--- Categorias (jerarquicas) ---
--- Marcas ---
--- Productos ---
--- Historial de precios (uno vigente por producto) ---
--- Depositos ---
--- Stock por producto y deposito (relacion N:M) ---
…
```

## 13.3. Pruebas de integridad declarativa

Las 21 pruebas negativas de la AE2 se reejecutaron sobre el modelo final antes de crear los triggers. Todas fueron rechazadas por la restricción prevista:

**Tabla 22.** *Pruebas negativas de restricciones declarativas (módulo 03).*

| Prueba | Operación intentada | Restricción que la rechazó | Resultado |
|---|---|---|---|
| T01 | PK compuesta duplicada rechazada | PK_Stock | OK |
| T02 | FK a Categoria inexistente rechazada | FK_Producto_Categoria | OK |
| T03 | FK compuesta a Stock rechazada | FK_Movimiento_Stock | OK |
| T04 | Direccion de otro cliente rechazada | FK_Pedido_ClienteDireccion | OK |
| T05 | Estado invalido rechazado | CK_Pedido_Estado | OK |
| T06 | Stock negativo rechazado | CK_Stock_Cantidad | OK |
| T07 | CantidadRecibida > Cantidad rechazada | CK_OCD_Recibida | OK |
| T08 | FechaHasta < FechaDesde rechazada | CK_ProductoPrecio_Periodo | OK |
| T09 | Mostrador con direccion rechazado | CK_Pedido_Envio | OK |
| T10 | Pedido web sin direccion rechazado | CK_Pedido_Envio | OK |
| T11 | SKU duplicado rechazado | UQ_Producto_Sku | OK |

| Prueba | Operación intentada | Restricción que la rechazó | Resultado |
|---|---|---|---|
| T12 | Documento duplicado rechazado | UQ_Cliente_Doc | OK |
| T13 | Segunda factura del mismo pedido rechazada | UQ_Factura_Pedido | OK |
| T14 | Numeracion fiscal repetida rechazada | UQ_Factura_Numeracion | OK |
| T15 | Segundo precio vigente rechazado | UX_ProductoPrecio_Vigente | OK |
| T16 | Segundo proveedor preferido rechazado | UX_ProveedorProducto_Preferido | OK |
| T17 | NOT NULL rechazado | NOT NULL (Nombre) | OK |
| T18 | Autosupervision rechazada | CK_Empleado_NoAutoSupervision | OK |
| T19 | CUIT invalido rechazado | CK_Proveedor_Cuit | OK |
| T20 | DELETE restringido por FK | FK_ProveedorProducto_Producto | OK |
| T21 | Alicuota invalida rechazada | CK_Factura_Iva | OK |

Las pruebas positivas T22 a T26 verificaron la eliminación en cascada de renglones y de datos de contacto, el recálculo de la columna calculada TotalFactura, el alta completa de una venta y la aceptación de varios clientes sin correo gracias al índice único filtrado; las cinco finalizaron correctamente.

## 13.4. Pruebas de funciones, procedimientos y triggers

El módulo 08 ejecuta 48 pruebas: 3 de funciones, 21 de procedimientos (incluida la atomicidad) y 24 de triggers. Para cada una se registra el código esperado (0 = operación aceptada), el obtenido y el mensaje del motor. El resumen final fue:

| Pruebas | Correctas | Fallidas |
|---|---|---|
| 48 | 48 | 0 |

**Tabla 23.** *Resultado de las pruebas de los objetos programados (módulo 08).*

| Prueba | Objeto | Caso | Esp. | Obt. | Res. | Mensaje del motor / valores verificados |
|---|---|---|---|---|---|---|
| F1-01 | fn_PrecioVigente | Precio del producto 1 en dos periodos distintos y de un producto sin precio | 0 | 0 | OK | 15/03: 890000.00; 01/07: 1050000.00; producto 14: NULL |
| F2-01 | fn_StockDisponible | Stock total coincide con SUM(Stock) y par inexistente devuelve 0 (no NULL) | 0 | 0 | OK | total producto 3: 242.000; SUM directo: 242.000; producto 2 en deposito 4: 0.000 |
| F3-01 | fn_TotalCompradoCliente | Suma pedidos no cancelados; cliente sin pedidos devuelve 0 | 0 | 0 | OK | cliente 1: 2700900.00; cliente 3 (con un pedido cancelado): 962300.00; cliente 6: 0.00 |
| P1-01 | sp_RegistrarPedido | Cliente inexistente | 50001 | 50001 | OK | El cliente no existe o esta inactivo. |
| P1-02 | sp_RegistrarPedido | Empleado de deposito intenta registrar una venta | 50002 | 50002 | OK | El empleado no existe, esta inactivo o su rol no le permite registrar ventas. |
| P1-03 | sp_RegistrarPedido | Vendedor de Sucursal Norte vende en Casa Central | 50003 | 50003 | OK | El empleado no pertenece a la sucursal indicada. |
| P1-04 | sp_RegistrarPedido | Despacho desde un deposito de otra sucursal | 50004 | 50004 | OK | El deposito de despacho no pertenece a la sucursal indicada. |
| P1-05 | sp_RegistrarPedido | Pedido WEB con la direccion de otro cliente | 50005 | 50005 | OK | Los pedidos WEB y TELEFONO requieren una direccion de envio registrada del cliente. |
| P1-06 | sp_RegistrarPedido | Detalle del pedido vacio | 50006 | 50006 | OK | Todo pedido debe contener al menos un renglon (RN08). |
| P1-07 | sp_RegistrarPedido | Renglon con cantidad cero | 50007 | 50007 | OK | Cada renglon requiere ProductoId, Cantidad > 0 y un descuento entre 0 y 100. |
| P1-08 | sp_RegistrarPedido | Producto repetido en el mismo pedido | 50007 | 50007 | OK | Un producto no puede repetirse en el mismo pedido: unifique las cantidades. |
| P1-09 | sp_RegistrarPedido | Venta de un producto discontinuado | 50008 | 50008 | OK | El producto 13 no existe o esta discontinuado. |
| P1-10 | sp_RegistrarPedido | Venta de un producto sin precio vigente | 50009 | 50009 | OK | El producto WC-LOG-004 no tiene precio vigente. |
| P1-11 | sp_RegistrarPedido | Stock insuficiente en el segundo renglon | 50010 | 50010 | OK | Stock insuficiente de AA-BGH-001 en el deposito: disponible 9, solicitado 50. |

| Prueba | Objeto | Caso | Esp. | Obt. | Res. | Mensaje del motor / valores verificados |
|---|---|---|---|---|---|---|
| P1-12 | sp_RegistrarPedido | Atomicidad: tras el rechazo no quedan pedido, renglones ni movimientos parciales | 0 | 0 | OK | pedidos 9->9; stock mouse 178.000->178.000; movimientos 26->26 |
| P1-13 | sp_RegistrarPedido | Alta valida: descuenta stock, fija precio vigente, audita y genera kardex | 0 | 0 | OK | PED-2026-0010: stock 44.000->42.000; precio 86000.00; historial 1; movimientos 1 |
| P2-01 | sp_FacturarPedido | Facturar un pedido inexistente | 50021 | 50021 | OK | El pedido no existe. |
| P2-02 | sp_FacturarPedido | Facturar un pedido cancelado | 50022 | 50022 | OK | Solo se facturan pedidos CONFIRMADOS o ENTREGADOS. |
| P2-03 | sp_FacturarPedido | Segunda factura para el mismo pedido | 50023 | 50023 | OK | El pedido ya tiene factura emitida (relacion 1:1). |
| P2-04 | sp_FacturarPedido | Neto de todas las facturas = total del detalle; cliente con CUIT recibe factura A | 0 | 0 | OK | facturas con diferencia: 0; tipo emitido a cliente CUIT: A |
| P3-01 | sp_RegistrarPago | Pago con monto cero | 50033 | 50033 | OK | El monto del pago debe ser mayor que cero. |
| P3-02 | sp_RegistrarPago | Pago que excede el saldo de la factura | 50131 | 50131 | OK | El pago excede el saldo de la factura A-0001-00001004: total 1340740.50, cobrado 1370370.25. |
| T4-01 | trg_Pago_ControlarSaldo | INSERT directo (sin procedimiento) sobre una factura ya cancelada | 50131 | 50131 | OK | El pago excede el saldo de la factura A-0001-00001001: total 2768601.00, cobrado 2768602.00. |
| P3-03 | sp_RegistrarPago | Pago exacto del saldo: numeracion correlativa y saldo cero | 0 | 0 | OK | pago nro 2 por 670370.25; saldo final 0.0000 |
| P4-01 | sp_ActualizarPrecioProducto | Precio igual al vigente | 50044 | 50044 | OK | El precio informado es igual al vigente: no hay cambio que registrar. |
| P4-02 | sp_ActualizarPrecioProducto | Precio con vigencia futura | 50043 | 50043 | OK | No se admiten precios con vigencia futura. |
| P4-03 | sp_ActualizarPrecioProducto | Segundo cambio de precio el mismo dia | 50045 | 50045 | OK | La nueva vigencia debe comenzar al menos dos dias despues del inicio del precio vigente. |
| T5-01 | trg_ProductoPrecio_Proteger | UPDATE directo del precio vigente (pisar el valor) | 50142 | 50142 | OK | El historial de precios es inalterable: solo puede cerrarse el periodo vigente. Use dbo.sp_ActualizarPrecioProducto. |
| T5-02 | trg_ProductoPrecio_Proteger | Modificar un periodo de precio ya cerrado | 50142 | 50142 | OK | El historial de precios es inalterable: solo puede cerrarse el periodo vigente. Use dbo.sp_ActualizarPrecioProducto. |
| T5-03 | trg_ProductoPrecio_Proteger | Eliminar el historial de precios de un producto | 50141 | 50141 | OK | El historial de precios no admite eliminaciones. Para discontinuar un producto use Producto.Activo = 0. |
| P4-04 | sp_ActualizarPrecioProducto | Historial consistente: un solo precio vigente y periodos contiguos sin huecos | 0 | 0 | OK | productos con 2+ vigentes: 0; periodos no contiguos: 0 |
| P5-01 | sp_CancelarPedido | Cancelar un pedido facturado | 50053 | 50053 | OK | El pedido esta facturado: requiere nota de credito, no cancelacion. |
| P5-02 | sp_CancelarPedido | Cancelar un pedido ya cancelado | 50052 | 50052 | OK | Solo pueden cancelarse pedidos PENDIENTES o CONFIRMADOS. |
| P5-03 | sp_CancelarPedido | El pedido anulado deja saldo neto cero en el kardex (egresos revertidos) | 0 | 0 | OK | movimientos del pedido: 6; pares con saldo distinto de cero: 0 |
| T1-01 | trg_MovimientoStock_Kardex | Modificar un movimiento de kardex | 50101 | 50101 | OK | El kardex es inalterable: los movimientos de stock no se modifican ni se eliminan. Registre un movimiento compensatorio. |
| T1-02 | trg_MovimientoStock_Kardex | Eliminar un movimiento de kardex | 50101 | 50101 | OK | El kardex es inalterable: los movimientos de stock no se modifican ni se eliminan. Registre un movimiento compensatorio. |
| T1-03 | trg_MovimientoStock_Kardex | Egreso mayor que la existencia del deposito | 50102 | 50102 | OK | Stock insuficiente: producto AA-BGH-001 en Deposito Central tiene 9 unidades y el movimiento requiere 1000. |
| T1-04 | trg_MovimientoStock_Kardex | INGRESO suma y AJUSTE resta automaticamente sobre Stock | 0 | 0 | OK | stock 178.000 -> INGRESO 5 -> 183.000 -> AJUSTE 2 -> 181.000 |
| T2-01 | trg_Stock_ControlarExistencias | UPDATE directo de Stock.Cantidad (sin kardex) | 50111 | 50111 | OK | Stock.Cantidad no se modifica directamente: registre un movimiento en MovimientoStock (INGRESO, EGRESO, TRANSFER o AJUSTE). |
| T2-02 | trg_Stock_ControlarExistencias | Alta de stock con cantidad inicial distinta de cero | 50111 | 50111 | OK | Stock.Cantidad no se modifica directamente: registre un movimiento en MovimientoStock (INGRESO, EGRESO, TRANSFER o AJUSTE). |
| T2-03 | trg_Stock_ControlarExistencias | Ingreso que supera la capacidad del deposito (RN07 de AE1) | 50112 | 50112 | OK | Capacidad excedida en Deposito Salon CC: ocupacion resultante 557 sobre una capacidad maxima de 400 unidades. |
| T2-04 | trg_Stock_ControlarExistencias | Tras el rechazo por capacidad el movimiento y el stock se revierten | 0 | 0 | OK | stock mouse en salon: 25.000 -> 25.000 |
| T2-05 | trg_Stock_ControlarExistencias | Alta de un producto en un deposito con cantidad 0 | 0 | 0 | OK | alta del par (producto 2, deposito 4) con cantidad 0 aceptada |

| Prueba | Objeto | Caso | Esp. | Obt. | Res. | Mensaje del motor / valores verificados |
|---|---|---|---|---|---|---|
| T3-01 | trg_Pedido_ControlarEstado | Retroceder un pedido ENTREGADO a PENDIENTE | 50121 | 50121 | OK | Transicion de estado no permitida en PED-2026-0001: ENTREGADO -> PENDIENTE. |
| T3-02 | trg_Pedido_ControlarEstado | Entregar un pedido sin factura | 50122 | 50122 | OK | El pedido PED-2026-0010 no puede entregarse sin factura emitida. |
| T3-03 | trg_Pedido_ControlarEstado | Cancelar por UPDATE un pedido facturado | 50123 | 50123 | OK | El pedido PED-2026-0007 esta facturado y no puede cancelarse. |
| T3-04 | trg_Pedido_ControlarEstado | Cancelar por UPDATE sin revertir el stock egresado | 50124 | 50124 | OK | El pedido PED-2026-0010 tiene stock egresado sin revertir: anulelo con dbo.sp_CancelarPedido. |
| T3-05 | trg_Pedido_ControlarEstado | Auditoria: el historial registra el alta y cada cambio de estado | 0 | 0 | OK | alta->CONFIRMADO ; CONFIRMADO->CANCELADO |

Entre los resultados conviene destacar: P1-12 (atomicidad: ningún efecto parcial tras un rechazo), T2-03 y T2-04 (el ingreso que excede la capacidad del depósito se rechaza y el movimiento no queda registrado), T3-04 (un pedido no puede cancelarse por UPDATE sin reponer el stock: debe usarse sp_CancelarPedido), P4-04 (después de todos los cambios de precio, ningún producto tiene dos precios vigentes ni períodos con huecos) y T3-05 (el historial de un pedido anulado registra el alta y la cancelación).

## 13.5. Defectos detectados durante las pruebas

Las ejecuciones sucesivas revelaron defectos que la revisión estática del código no había detectado. Se documentan porque forman parte del proceso de verificación:

**Tabla 24.** *Defectos detectados al ejecutar y corrección aplicada.*

| Etapa | Síntoma | Causa | Corrección |
|---|---|---|---|
| AE2 | T12 y T19 fallaban por correo duplicado en lugar de por la restricción que debían probar. | UNIQUE sobre columna nulable: SQL Server considera iguales los NULL. | Índices únicos filtrados WHERE Email IS NOT NULL. |
| AE4 | Error Msg 1934 al crear índices filtrados desde sqlcmd. | sqlcmd ejecuta con QUOTED_IDENTIFIER OFF; SSMS lo activa por defecto. | SET ANSI_NULLS ON y SET QUOTED_IDENTIFIER ON al comienzo de cada módulo. |
| AE4 | Operaciones que buscaban pedidos por identificador fijo afectaban filas equivocadas. | Las pruebas rechazadas consumen valores IDENTITY y dejan huecos en la numeración. | Búsqueda por clave natural (NroPedido, NroOrden) y captura de identificadores con parámetros OUTPUT. |
| AE4 | C14 mostraba como diferencia de precio una venta realizada al precio correcto. | El precio se actualizaba el mismo día después de la venta y fn_PrecioVigente trabaja con granularidad de día. | Las actualizaciones de precio (M4) se ejecutan antes de las ventas del día (M5), en orden cronológico. |
| AE4 | La consulta de productos bajo mínimo no devolvía filas después de las ventas. | Los datos no generaban ningún faltante con las operaciones de la AE4. | Venta de mostrador M5.3 con cantidad suficiente para dejar un producto bajo el mínimo. |
| AE4 | Filas del registro de pruebas con columnas desplazadas. | Los mensajes usaban el carácter \| , que sqlcmd emplea como separador de columnas. | Separador «;» en los mensajes de las pruebas. |

# 14. Matriz de trazabilidad

La matriz vincula cada regla de negocio con su origen, el elemento del modelo conceptual que la representa, las tablas del esquema relacional, el objeto SQL que la hace cumplir, las consultas o vistas que la ponen de manifiesto y las pruebas que verifican su funcionamiento. Permite comprobar que ninguna regla carece de implementación ni de prueba.

**Tabla 25.** *Matriz de trazabilidad de las reglas de negocio.*

| RN | Origen | Modelo conceptual | Tablas | Objeto SQL | Consultas / vistas | Pruebas |
|---|---|---|---|---|---|---|
| RN01 | AE1 | CLASIFICA (1:N) | Producto, Categoria | FK_Producto_Categoria | C01, C07 | T02 |
| RN02 | AE1→AE2 | ALMACENA (N:M) / STOCK | Stock | PK_Stock | C15, vw_StockPorDeposito | T01 |
| RN03 | AE1 | REALIZA (1:N) | Pedido, Cliente | FK_Pedido_Cliente; sp_RegistrarPedido | C03, C12 | P1-01 |
| RN04 | AE1 | COMPONE (N:M) | PedidoDetalle | PK_PedidoDetalle; sp_RegistrarPedido | C03 | P1-07, P1-08 |
| RN05 | AE1 | Atributo Cantidad de ALMACENA | Stock, MovimientoStock | CK_Stock_Cantidad; T1; sp_RegistrarPedido | C04, C15 | T06, T1-03, P1-11 |
| RN06 | AE1 | Atributo PrecioUnitarioHistorico | PedidoDetalle, ProductoPrecio | sp_RegistrarPedido; fn_PrecioVigente | C14 | P1-13, F1-01 |
| RN07 | AE1 | Atributo CapacidadMaxima | Deposito, Stock | T2 trg_Stock_ControlarExistencias | vw_DepositoOcupacion | T2-03, T2-04 |
| RN08 | AE1 | Participación total en COMPONE | Pedido, PedidoDetalle | sp_RegistrarPedido | C03 | P1-06 |
| RN09 | AE2 | PRECIO_PRODUCTO (débil) | ProductoPrecio | UX_ProductoPrecio_Vigente; CK_ProductoPrecio_Periodo; sp_ActualizarPrecioProducto | vw_ProductoPrecioVigente, C09 | T08, T15, P4-01–P4-04 |
| RN10 | AE4 | PRECIO_PRODUCTO | ProductoPrecio | T5 trg_ProductoPrecio_ProtegerHistorial | M4 | T5-01–T5-03 |
| RN11 | AE2→AE4 | STOCK ↔ MOVIMIENTO_STOCK | Stock, MovimientoStock | T1, T2 | C15, C17 | T1-04, T2-01, T2-02, T2-05 |
| RN12 | AE4 | MOVIMIENTO_STOCK | MovimientoStock | T1 | C17 | T1-01, T1-02 |
| RN13 | AE4 | Atributo Estado de PEDIDO | Pedido | CK_Pedido_Estado; T3 | C12 | T05, T3-01 |
| RN14 | AE2→AE4 | GENERA (1:1) | Pedido, Factura, MovimientoStock | T3; sp_CancelarPedido; sp_FacturarPedido | C17 | T3-02–T3-04, P2-02, P5-01–P5-03 |
| RN15 | AE4 | PEDIDO_ESTADO_HISTORIAL | PedidoEstadoHistorial | T3 | C17 | T3-05, P1-13 |
| RN16 | AE2→AE4 | PAGO (débil de FACTURA) | Pago, Factura | T4; sp_RegistrarPago | C13, vw_FacturaSaldo | P3-01–P3-03, T4-01 |
| RN17 | AE2→AE4 | GENERA (1:1) | Factura | UQ_Factura_Pedido; UQ_Factura_Numeracion; sp_FacturarPedido | vw_PedidoTotal | T13, T14, P2-01–P2-04 |
| RN18 | AE2 | DIRECCION_CLIENTE | Pedido, ClienteDireccion | CK_Pedido_Envio; FK_Pedido_ClienteDireccion; sp_RegistrarPedido | C03 | T04, T09, T10, P1-05 |
| RN19 | AE4 | ATIENDE / TRABAJA_EN | Pedido, Empleado, Deposito | sp_RegistrarPedido | C06 | P1-02–P1-04 |

| RN | Origen | Modelo conceptual | Tablas | Objeto SQL | Consultas / vistas | Pruebas |
|---|---|---|---|---|---|---|
| RN20 | AE1→AE2 | Identificadores alternativos | Producto, Proveedor, Cliente, Empleado | UQ_*, UX_Cliente_Email, UX_Proveedor_Email | C02 | T11, T12, T19, T26 |

**Tabla 26.** *Matriz de trazabilidad de los requerimientos de información.*

| RI | Modelo conceptual | Tablas | Consultas / vistas | Requerimiento funcional |
|---|---|---|---|---|
| RI01 | CLIENTE, DIRECCION_CLIENTE | Cliente, ClienteTelefono, ClienteDireccion | C02, C13, vw_ResumenVentasCliente | RF10, RF13 |
| RI02 | PRODUCTO, CATEGORIA, MARCA | Producto, Categoria, Marca | C01, C04, C09, C10, vw_CatalogoProductos | RF08 |
| RI03 | DEPOSITO, SUCURSAL | Deposito, Sucursal | vw_DepositoOcupacion, vw_StockPorDeposito | RF06, RF07 |
| RI04 | STOCK (agregación) | Stock | C15, vw_ProductoStock, vw_StockPorDeposito | RF08, RF09 |
| RI05 | PROVEEDOR, SUMINISTRA | Proveedor, ProveedorTelefono, ProveedorProducto | C11, C20 | RF09, RF13 |
| RI06 | PEDIDO | Pedido | C03, C06, C12 | RF01 |
| RI07 | COMPONE | PedidoDetalle | C03, C10, C14 | RF01 |
| RI08 | STOCK (StockMinimo) | Stock | C11, vw_StockBajoMinimo | RF09 |
| RI09 | PRECIO_PRODUCTO | ProductoPrecio | C09, C14, vw_ProductoPrecioVigente | RF04 |
| RI10 | MOVIMIENTO_STOCK | MovimientoStock | C17 | RF06, RF07, RF12 |
| RI11 | ORDEN_COMPRA, DETALLA | OrdenCompra, OrdenCompraDetalle | C19, M2 | RF06 |
| RI12 | FACTURA, PAGO, MEDIO_PAGO | Factura, Pago, MedioPago | C13, vw_FacturaSaldo | RF02, RF03, RF10 |
| RI13 | SUCURSAL, EMPLEADO | Sucursal, Empleado | C03, C06 | RF01 |
| RI14 | PEDIDO_ESTADO_HISTORIAL | PedidoEstadoHistorial | C17 | RF12 |
| RI15 | — | Pedido, PedidoDetalle, Categoria | C05–C08, C16 | RF11 |

# 15. Conclusiones

El trabajo integró las tres etapas del proyecto en una solución coherente y verificable. El modelo conceptual de la AE1, corregido y ampliado en la AE2, se implementó sin pérdida de semántica: cada entidad, relación y atributo del DER tiene su correspondencia en el esquema relacional, cada dependencia funcional se preserva en una única tabla y cada regla de negocio tiene un mecanismo de verificación y al menos una prueba que lo ejercita, como muestra la matriz de trazabilidad.

La principal lección del proceso fue que la calidad del modelo conceptual condiciona todo lo posterior. Las dos cardinalidades mal definidas en la AE1 (un producto en un único depósito y con un único proveedor) no se habrían detectado en una revisión de notación; habrían sobrevivido a la transformación como claves foráneas y recién se habrían manifestado al incorporar un segundo depósito. El borrador inicial de la AE4 confirmó el riesgo: al partir nuevamente del modelo de la AE1 reapareció cada uno de esos problemas.

En la implementación se comprobó que el motor puede garantizar mucho más de lo que habitualmente se le delega. Claves foráneas compuestas, restricciones CHECK multicolumna e índices únicos filtrados hacen imposibles por construcción varias reglas que suelen quedar como validaciones en la aplicación. Para las reglas restantes, la combinación de procedimientos almacenados transaccionales y triggers permitió que la coherencia entre stock y kardex, el ciclo de vida del pedido, el control de cobranza y la inalterabilidad del historial se cumplan cualquiera sea el origen de la operación. Las 48 pruebas de programación y las 26 de integridad declarativa, ejecutadas sobre un script que se construye desde una base vacía, constituyen la evidencia de ese funcionamiento.

Como líneas de trabajo futuro se identifican la incorporación de notas de crédito para anular ventas facturadas, la gestión de devoluciones a proveedores, la definición de usuarios y permisos del SGBD por rol —de modo que la aplicación solo pueda ejecutar procedimientos y no escribir directamente en las tablas— y la evaluación de vistas indexadas para los indicadores de mayor costo si el volumen de datos lo justificara.

# 16. Bitácora del equipo

La bitácora registra, para cada tarea, la fecha, el responsable, la decisión adoptada, la principal dificultad, la prueba realizada y la corrección aplicada.

**Tabla 27.** *Bitácora del proyecto.*

| Fecha | Tarea | Responsable | Decisión | Dificultad | Prueba realizada | Corrección aplicada |
|---|---|---|---|---|---|---|
| 05/09/2026 | AE1: relevamiento del dominio, RI, RN y DER | Ambos | Claves sustitutas; DETALLE_PEDIDO como entidad débil asociativa; precio pactado en el detalle | Representar el precio histórico sin duplicar el de catálogo | Matriz RI–componentes | Se documentaron las limitaciones (RN02, RN07, RI08) para iteraciones futuras |
| 01/10/2026 | AE2: auditoría del DER, modelo ampliado, normalización y DDL | Nahuel Villegas | ALMACENA y SUMINISTRA N:M; historial de precios; kardex; factura 1:1; SQL Server | Conectar MOVIMIENTO_STOCK con una relación (agregación); cascadas múltiples (error 1785) | 21 pruebas negativas, 5 positivas, 8 consultas de verificación | UNIQUE nulables reemplazados por índices filtrados (T12, T19) |
| 11/11/2026 | AE4 (borrador): DDL, carga y normalización en MySQL | Leandro Acosta | Modelo simplificado de 10 tablas; tabla AUDITORIA_PRECIOS | Reunir la documentación de AE1 y AE2 | Revisión de consultas de ejemplo | Reemplazado por el modelo de la AE2 (Tabla 3) |
| 11/11/2026 | AE4 (borrador): función, procedimiento, triggers y vista en MySQL | Nahuel Villegas | sp_RegistrarPedido con EXIT HANDLER; triggers BEFORE INSERT y AFTER UPDATE | El procedimiento creaba pedidos sin renglones | Llamadas CALL de ejemplo | Rediseño completo de la capa de programación sobre el modelo final |
| 12/11/2026 | Contraste del borrador con la AE2 y decisión de consolidación | Ambos | Adoptar el modelo de la AE2 en SQL Server; nueva base StockFlow | El borrador reintroducía M1, M3, M4 y M7 | Comparación tabla por tabla con la AE2 | Tabla de correspondencia borrador → versión final |
| 12/11/2026 | Catálogo consolidado de RN y RI; requerimientos funcionales | Leandro Acosta | Conservar la numeración de la AE1 (RN01–RN07, RI01–RI08) y agregar las nuevas | Reglas del borrador numeradas distinto que en la AE1 | Cada RN con mecanismo y prueba | Renumeración documentada en la Tabla 8 |
| 12/11/2026 | DDL final: tabla PedidoEstadoHistorial e índices | Nahuel Villegas | Clave sustituta en el historial | Msg 1934 al ejecutar con sqlcmd | Ejecución de 01 a 03 desde base vacía | SET QUOTED_IDENTIFIER ON en cada módulo |
| 12/11/2026 | Funciones, triggers y procedimientos | Nahuel Villegas | Stock solo vía kardex; THROW con rangos de códigos; detalle en JSON | Distinguir cambios directos de Stock de los hechos por T1 | 48 pruebas automatizadas | TRIGGER_NESTLEVEL() en T2 |
| 12/11/2026 | Datos de prueba para casos límite y operaciones DML | Leandro Acosta | Producto discontinuado, producto sin precio, clienta sin datos de contacto | Huecos de IDENTITY por pruebas rechazadas | Recuento de filas y consultas C01–C20 | Búsqueda por número de pedido y parámetros OUTPUT |
| 12/11/2026 | Consultas y vistas | Leandro Acosta | Una consulta por requerimiento; vistas para la aplicación | C14 mostraba diferencias espurias de precio | Revisión de resultados de C01–C20 | Cambios de precio antes de las ventas del día |
| 12/11/2026 | Script integral y evidencia de ejecución | Nahuel Villegas | Generar el integral concatenando módulos | Separador \| en los mensajes del registro | Ejecución completa: 48/48 pruebas correctas | Separador «;» en los mensajes |
| 12/11/2026 | Adaptación de la aplicación web a SQL Server | Nahuel Villegas | PDO_SQLSRV; la lógica de negocio queda en los procedimientos | Traducir los errores THROW a mensajes de la interfaz | Consultas de los endpoints ejecutadas en sqlcmd | Función respuesta_error() en conexion.php |
| 12/11/2026 | Diccionario de datos, trazabilidad e informe final | Leandro Acosta | Diccionario generado desde el catálogo del sistema | Mantener coherencia entre informe y script | Revisión cruzada informe–registro de ejecución | Informe con formato académico |

# 17. Reflexiones individuales

## 17.1. Leandro Acosta

Mi aporte se centró en el modelado y la documentación: el relevamiento y el DER de la AE1, la normalización, el catálogo de requerimientos y reglas de negocio, los datos de prueba, las consultas y la matriz de trazabilidad. El aprendizaje más importante fue entender que una cardinalidad mal definida en el modelo conceptual no es un detalle de notación: se convierte en una clave foránea y limita lo que el sistema puede representar. Comparar nuestro borrador de la AE4 con la AE2 me mostró que habíamos vuelto a cometer los mismos errores al simplificar el modelo, y que la documentación de las decisiones es lo que permite detectarlo. Como equipo acordamos que cada regla de negocio debía tener un mecanismo de verificación y una prueba asociados, y ese acuerdo ordenó todo el trabajo final.

## 17.2. Nahuel Villegas

Me ocupé principalmente de la implementación: el DDL y las pruebas de la AE2, la programación de funciones, procedimientos y triggers, la batería de pruebas automatizadas, el script integral y la adaptación de la aplicación web. Aprendí que una prueba que pasa no alcanza si no se verifica por qué pasa, y que conviene que la base de datos garantice sus propias reglas en lugar de confiar en la aplicación: decidir que el stock solo cambie a través del kardex simplificó los procedimientos y eliminó una fuente entera de inconsistencias. También aprendí a valorar los detalles del motor —el comportamiento de los NULL en UNIQUE, las opciones de sesión de sqlcmd, los triggers por sentencia— que solo aparecen al ejecutar. La división del trabajo funcionó porque acordamos desde el principio un catálogo único de reglas sobre el que ambos trabajamos.

# Referencias


Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9–36.

Date, C. J. (2019). *Database design and relational theory: Normal forms and all that jazz* (2.ª ed.). Apress.

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Microsoft. (2024). *Transact-SQL reference (Database Engine)*. Microsoft Learn. https://learn.microsoft.com/sql/t-sql/language-reference

Microsoft. (2024). *CREATE TRIGGER (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/sql/t-sql/statements/create-trigger-transact-sql

Microsoft. (2024). *Create filtered indexes*. Microsoft Learn. https://learn.microsoft.com/sql/relational-databases/indexes/create-filtered-indexes

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2019). *Database system concepts* (7.ª ed.). McGraw-Hill.

Universidad de la Cuenca del Plata. (2026). *Consigna de la Actividad de Evaluación N.º 4: Base de Datos (ISI17BAD28V26AE4V26)*. Facultad de Ingeniería y Tecnología.

# Anexo A. Aplicación web de demostración

El repositorio incluye una aplicación web (`stockflow/`) que demuestra el uso de la base desde una capa de presentación. Se compone de una página HTML con JavaScript y un conjunto de servicios PHP que se conectan a SQL Server mediante la extensión PDO_SQLSRV. En la AE4 se adaptó al modelo final: los servicios no contienen lógica de negocio, sino que consultan las vistas e invocan los procedimientos almacenados; cuando un procedimiento rechaza una operación, el mensaje del THROW se muestra al usuario.

**Tabla 28.** *Servicios de la aplicación web y objetos de la base que utilizan.*

| Servicio | Método | Objeto de la base | Función |
|---|---|---|---|
| api/productos.php | GET | vw_CatalogoProductos | Catálogo con precio vigente y stock |
| api/stock.php | GET | vw_StockPorDeposito, vw_DepositoOcupacion | Existencias por depósito y ocupación |
| api/maestros.php | GET | Cliente, ClienteDireccion, Empleado, Deposito | Datos para el formulario de venta |
| api/registrar_pedido.php | POST | sp_RegistrarPedido | Registra una venta desde el carrito |
| api/actualizar_precio.php | POST | sp_ActualizarPrecioProducto | Cambia un precio conservando el historial |
| api/historial_precios.php | GET | ProductoPrecio | Últimos períodos de precio |
| api/resumen_ventas.php | GET | vw_ResumenVentasCliente | Cartera de clientes |

**Ejecución.** La aplicación se ejecutó con PHP 8.3 y el controlador PDO_SQLSRV 5.12 de Microsoft, mediante el servidor web integrado de PHP, sin necesidad de instalar un servidor web completo:

```
php -S localhost:8000 -t stockflow
```

La página se abre en http://localhost:8000. La dirección de los servicios es relativa a la página (`api/`), por lo que la aplicación también funciona publicada en XAMPP o IIS. La conexión utiliza autenticación de Windows y sus parámetros se configuran en `stockflow/conexion.php`.

**Pruebas realizadas.** Se verificaron los siete servicios contra la base generada por el script integral. La tabla siguiente resume los casos; los rechazos llegan a la interfaz con el código y el mensaje del THROW correspondiente.

**Tabla 29.** *Pruebas de la aplicación web.*

| Servicio | Caso | Respuesta obtenida |
|---|---|---|
| productos, stock, maestros, historial_precios, resumen_ventas | Consulta de datos | HTTP 200 con los datos de las vistas |
| registrar_pedido | Venta web válida | HTTP 200: «Pedido PED-2026-0010 registrado y stock descontado.» |
| registrar_pedido | Cantidad mayor al stock | HTTP 422, código 50010: «Stock insuficiente de SS-KIN-001 en el deposito…» |
| registrar_pedido | Venta web sin dirección | HTTP 422, código 50005: «Los pedidos WEB y TELEFONO requieren una direccion de envio…» |

| Servicio | Caso | Respuesta obtenida |
|---|---|---|
| actualizar_precio | Precio igual al vigente | HTTP 422, código 50044: «El precio informado es igual al vigente…» |
| actualizar_precio | Precio nuevo | HTTP 200: «Precio actualizado; el periodo anterior quedo cerrado en el historial.» |

**Interfaz y evidencias.** La interfaz se diseñó para que la utilicen empleados sin conocimientos técnicos: emplea el vocabulario del negocio (Productos, Depósitos, Nueva venta, Precios y Clientes), traduce los estados de la base a expresiones simples y presenta los rechazos de los procedimientos como mensajes en lenguaje llano. Un interruptor «Detalles técnicos» muestra en cada pantalla los objetos de la base que intervienen. Las Figuras 7 a 17 documentan las pantallas y las pruebas de la Tabla 29; debajo de cada figura se indica qué objetos de la base participan y qué regla queda evidenciada.

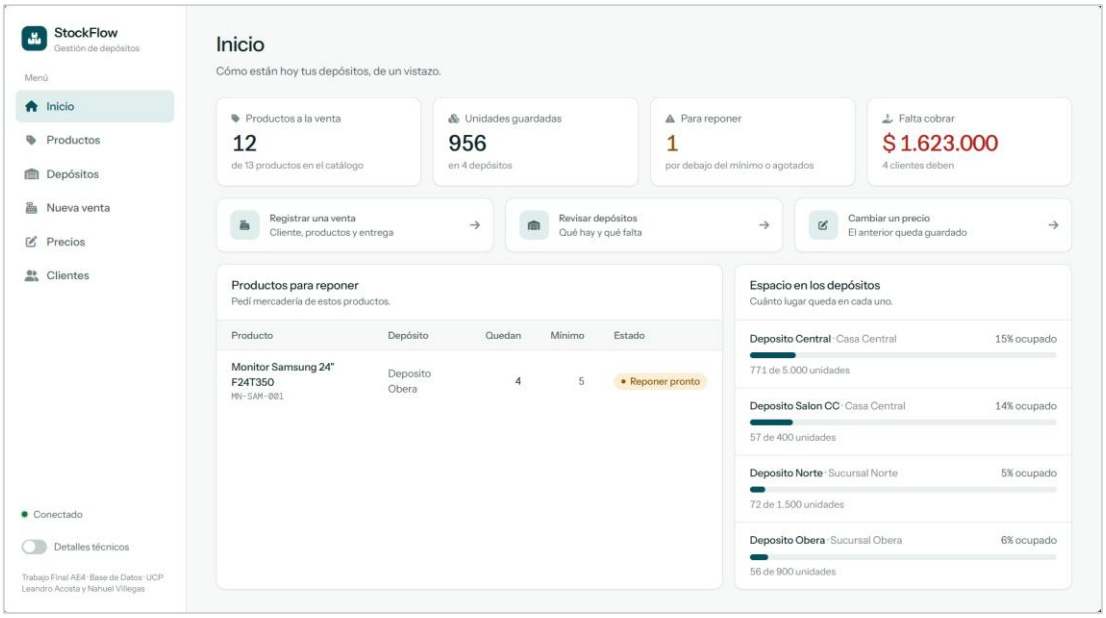



**Figura 7.** *Pantalla Inicio: resumen del estado de los depósitos.*

**Detalle técnico.** Los indicadores se calculan sobre vistas de la base: los productos a la venta provienen de vw_CatalogoProductos (precio vigente y stock mayor que cero); las unidades guardadas y los productos para reponer, de vw_StockPorDeposito, cuyo atributo EstadoStock compara la cantidad con el stock mínimo (OK, BAJO MINIMO o SIN STOCK, que la interfaz muestra como «Bien», «Reponer pronto» y «Agotado»); el espacio ocupado, de vw_DepositoOcupacion frente a la capacidad máxima de cada depósito; y el saldo a cobrar, de vw_ResumenVentasCliente.

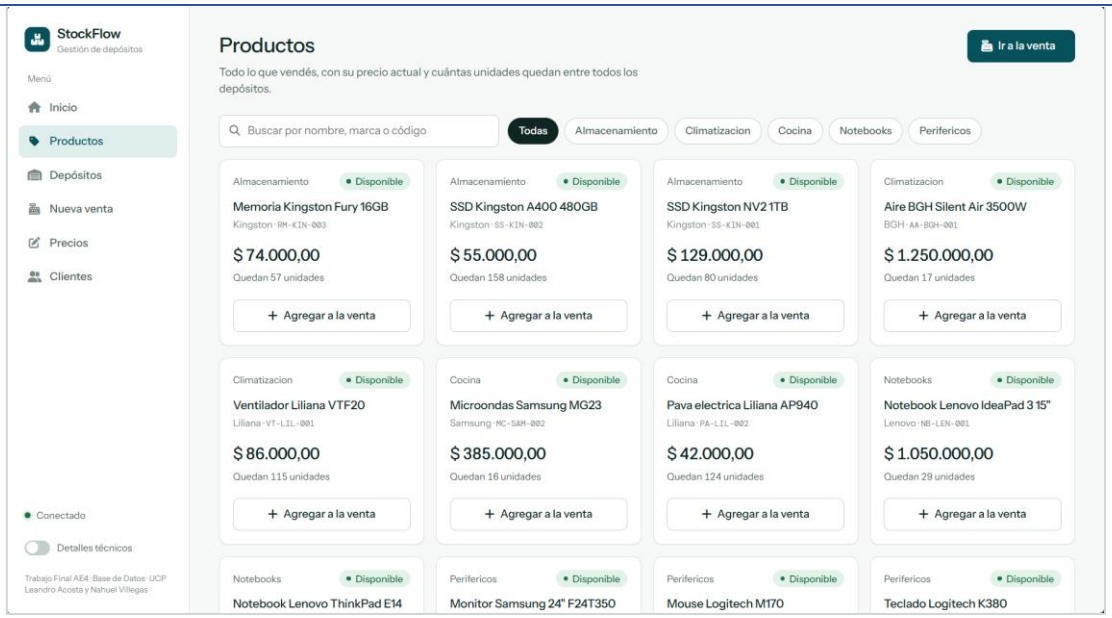



**Figura 8.** *Pantalla Productos: catálogo con precio vigente y existencias totales.*

**Detalle técnico.** El servicio api/productos.php consulta vw_CatalogoProductos. El precio mostrado es el del período vigente de ProductoPrecio (FechaHasta nula) y las existencias son la suma de todos los depósitos, calculada con fn_StockDisponible. Un producto sin precio vigente o sin existencias se presenta como no disponible y no puede agregarse a una venta.

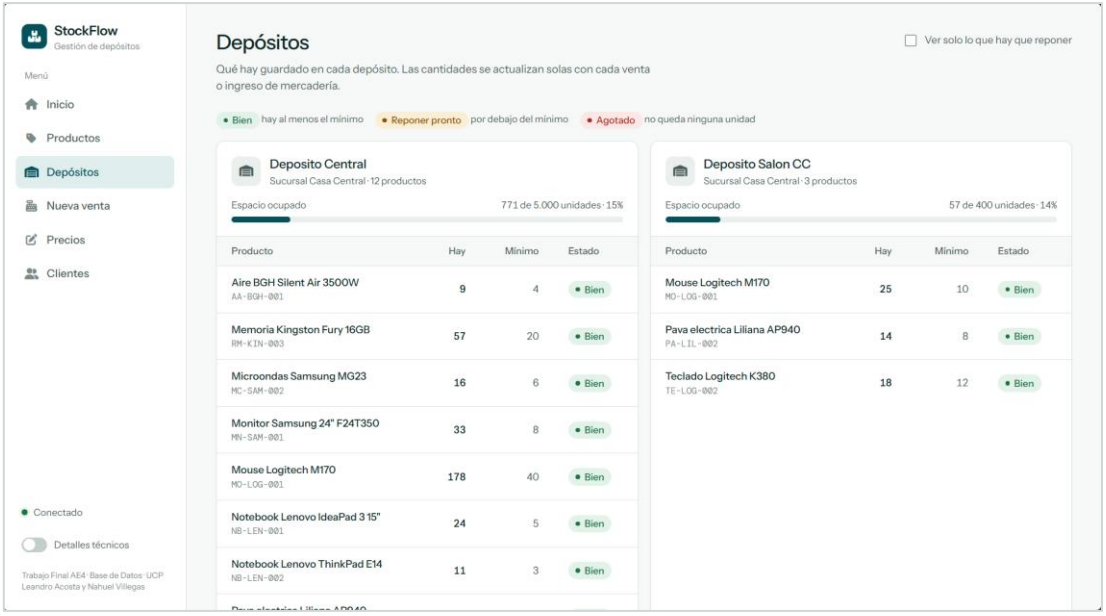



**Figura 9.** *Pantalla Depósitos: existencias por depósito y estado de reposición.*

**Detalle técnico.** El servicio api/stock.php consulta vw_StockPorDeposito y vw_DepositoOcupacion. Cada fila corresponde a una tupla de Stock, la relación N:M entre Producto y Deposito. La aplicación no edita cantidades: solo cambian mediante movimientos de kardex, que trg_MovimientoStock_AplicarKardex aplica sobre Stock, mientras trg_Stock_ControlarExistencias rechaza la modificación directa de Stock.Cantidad y controla que la ocupación no supere la capacidad máxima del depósito.

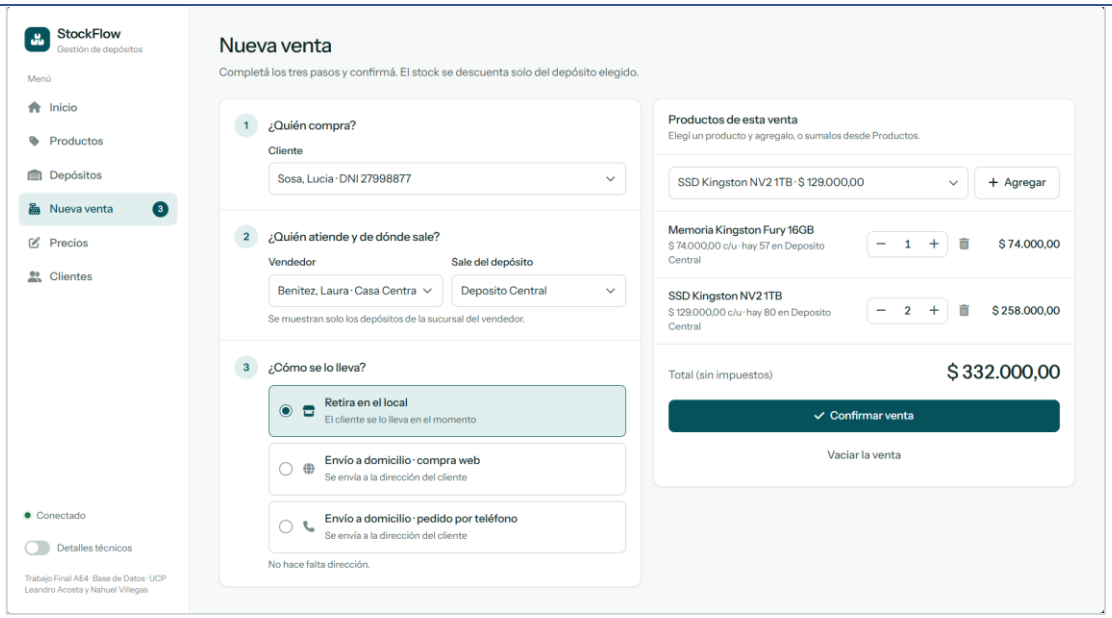



**Figura 10.** *Nueva venta: cliente, vendedor, depósito de despacho, forma de entrega y renglones.*

**Detalle técnico.** Los selectores se cargan desde api/maestros.php: clientes activos con su dirección principal (ClienteDireccion), empleados activos con un rol que habilita la venta y los depósitos de la sucursal del vendedor elegido. La interfaz advierte si una cantidad supera la existencia del depósito, pero la validación definitiva la realiza el procedimiento almacenado: la aplicación no reemplaza los controles de la base.

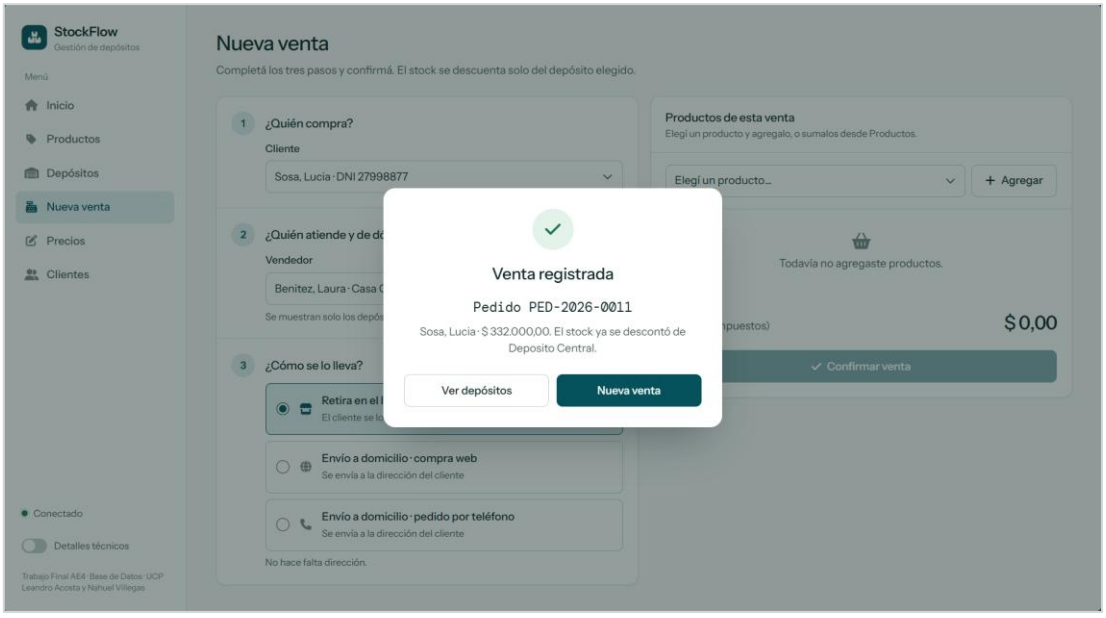



**Figura 11.** *Venta confirmada: el procedimiento devuelve el número de pedido PED-2026-0011.*

**Detalle técnico.** El servicio api/registrar_pedido.php invoca sp_RegistrarPedido con los renglones en formato JSON. Dentro de una única transacción, el procedimiento valida cliente, vendedor, sucursal, depósito, canal y existencias; inserta el Pedido, sus renglones en PedidoDetalle con el precio obtenido por fn_PrecioVigente y un movimiento de EGRESO por renglón en MovimientoStock, que trg_MovimientoStock_AplicarKardex descuenta de Stock. El número de pedido se devuelve en el parámetro de salida @NroPedido.

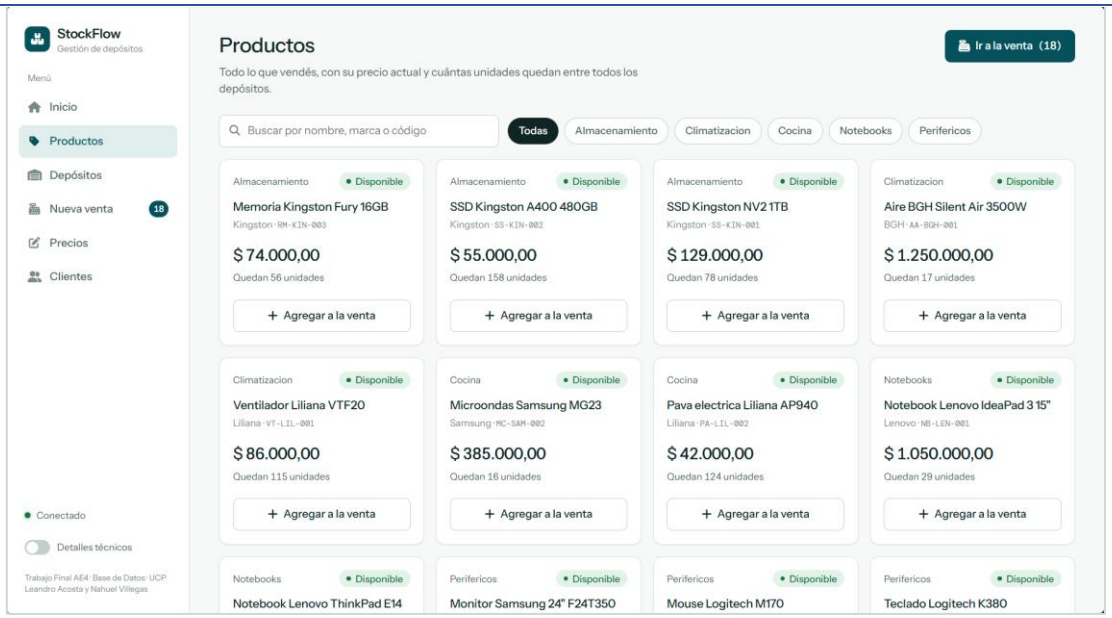



**Figura 12.** *Catálogo después de la venta de la Figura 11.*

**Detalle técnico.** Respecto de la Figura 8, la existencia de la Memoria Kingston Fury 16GB bajó de 57 a 56 unidades y la del SSD Kingston NV2 1TB de 80 a 78, exactamente las cantidades vendidas. El descuento no lo calcula la aplicación: resulta de los movimientos de EGRESO que aplicó el trigger de kardex.

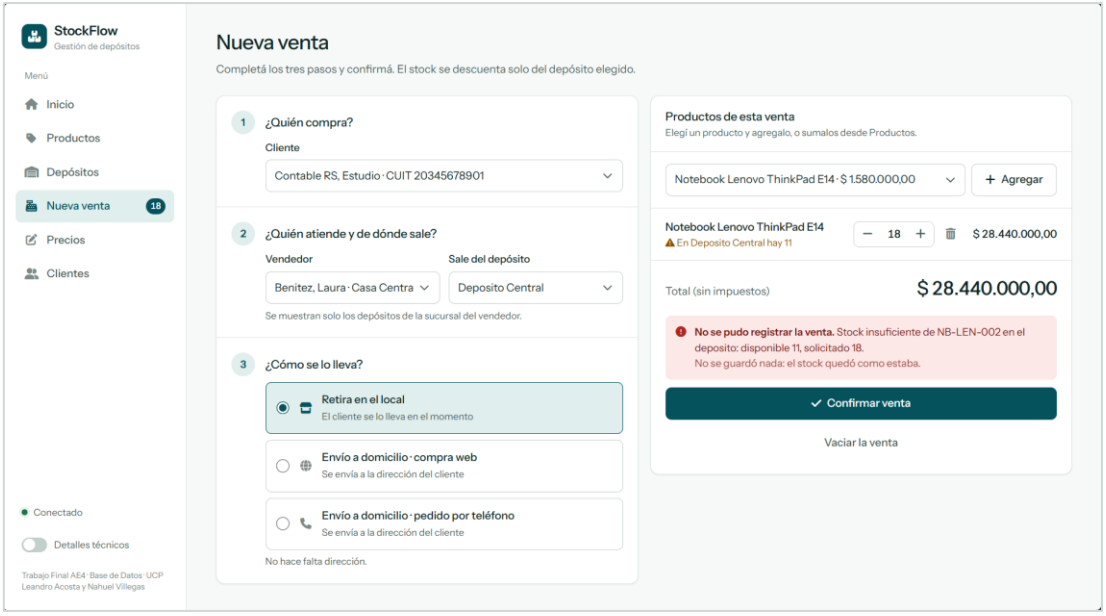



**Figura 13.** *Venta rechazada por existencias insuficientes.*

**Detalle técnico.** Se solicitaron 18 unidades de la Notebook Lenovo ThinkPad E14, con 11 disponibles en el Depósito Central. sp_RegistrarPedido detecta la falta de existencias, revierte la transacción (ROLLBACK) y lanza THROW 50010; el servicio responde HTTP 422 y la interfaz muestra el mensaje del procedimiento. No queda registrado ningún pedido ni movimiento de stock.

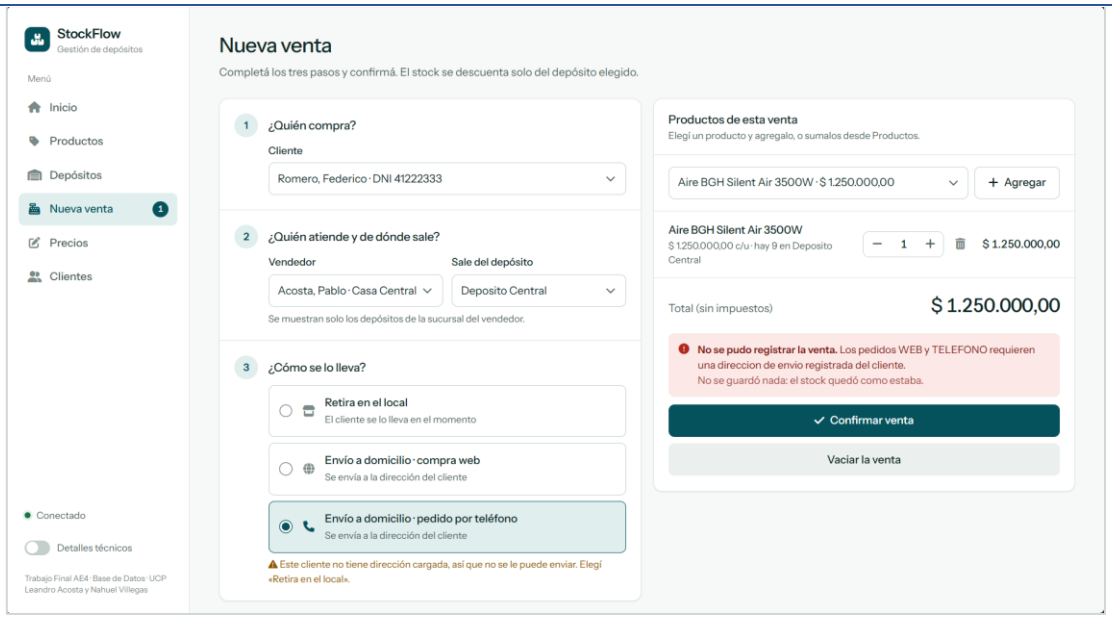



**Figura 14.** *Venta con envío a domicilio rechazada por falta de dirección del cliente.*

**Detalle técnico.** El cliente Romero, Federico, incorporado para esta prueba, no tiene dirección registrada y se eligió un canal con envío (TELEFONO). sp_RegistrarPedido rechaza la operación con THROW 50005. La misma regla está protegida en forma declarativa por la restricción CK_Pedido_Envio de la tabla Pedido, que exige dirección para los canales WEB y TELEFONO y la prohíbe para MOSTRADOR.

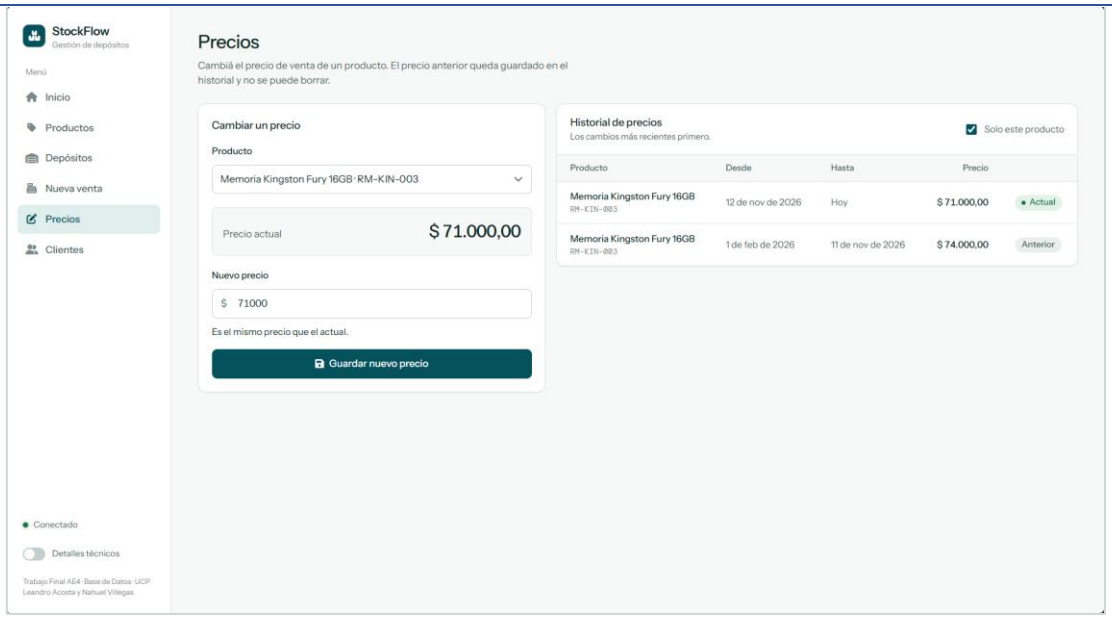



**Figura 15.** *Precios después de cambiar el precio de la Memoria Kingston Fury 16GB de $ 74.000 a $ 71.000.*

**Detalle técnico.** El servicio api/actualizar_precio.php invoca sp_ActualizarPrecioProducto, que cierra el período vigente (asigna a FechaHasta el día anterior al inicio del nuevo precio) e inserta un nuevo período en ProductoPrecio, sin sobrescribir el anterior. El historial conserva así ambos períodos: $ 74.000, vigente del 01/02/2026 al 11/11/2026, y $ 71.000, vigente desde el 12/11/2026. trg_ProductoPrecio_ProtegerHistorial impide modificar o eliminar períodos ya registrados (THROW 50141 y 50142), de modo que siempre puede reconstruirse el precio vigente en cualquier fecha.

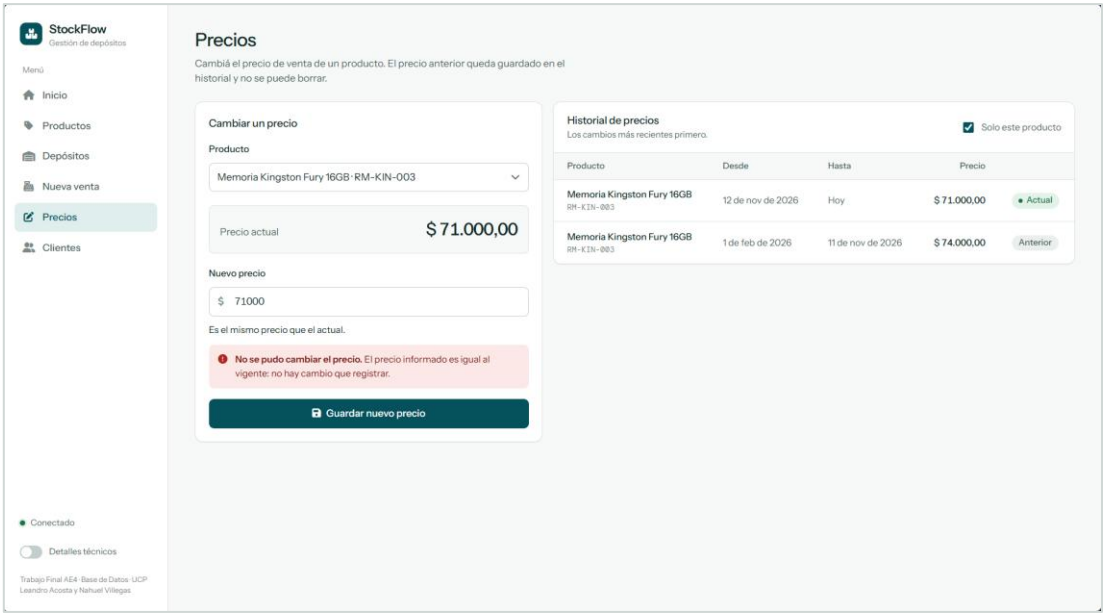



**Figura 16.** *Cambio de precio rechazado por ser igual al vigente.*

**Detalle técnico.** Se intentó registrar como nuevo precio el mismo valor vigente. sp_ActualizarPrecioProducto lo rechaza con THROW 50044 porque no hay cambio que registrar, y el historial queda sin modificaciones.

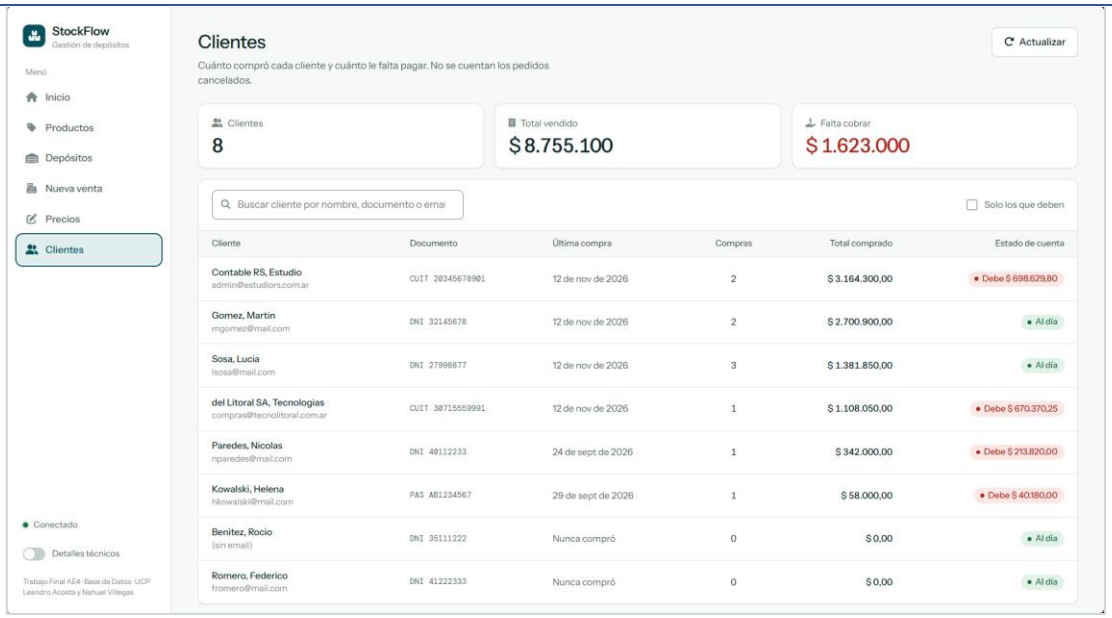



**Figura 17.** *Pantalla Clientes: compras y saldo pendiente de cada cliente.*

**Detalle técnico.** El servicio api/resumen_ventas.php consulta vw_ResumenVentasCliente. El total comprado se obtiene con fn_TotalCompradoCliente, que excluye los pedidos cancelados; la vista incluye a los clientes sin compras mediante LEFT JOIN y suma el saldo pendiente de sus facturas a partir de vw_FacturaSaldo. El estado «Debe» o «Al día» se deriva de ese saldo.

Las pruebas de escritura y las capturas de las Figuras 7 a 17 se realizaron sobre los datos de prueba, por lo que algunos valores (el pedido PED-2026-0011, precios y saldos) difieren del estado documentado en las secciones 10 a 13; luego se volvió a ejecutar el script integral para restituir ese estado. La aplicación es complementaria y no forma parte de los requisitos de la consigna.

# Anexo B. Consultas: código y resultados

Para cada consulta del módulo 07 se presenta el código y el resultado obtenido en la ejecución del script integral. Los resultados extensos se truncan; el resultado completo figura en el registro de ejecución.

## *C01. Catálogo de periféricos activos*

```
/* C01 - Catalogo de perifericos activos, ordenado por nombre. */
SELECT 'C01' AS Consulta, Sku, Nombre, UnidadMedida, PesoKg
FROM dbo.Producto
WHERE CategoriaId = (SELECT CategoriaId FROM dbo.Categoria WHERE Nombre = N'Perifericos')
  AND Activo = 1
ORDER BY Nombre;
```

| Sku | Nombre | UnidadMedida | PesoKg |
|---|---|---|---|
| MN-SAM-001 | Monitor Samsung 24" F24T350 | UN | 3,20 |
| MO-LOG-001 | Mouse Logitech M170 | UN | 0,08 |
| TE-LOG-002 | Teclado Logitech K380 | UN | 0,42 |
| WC-LOG-004 | Webcam Logitech C920 | UN | NULL |

## *C02. Clientes sin correo electrónico*

```
/* C02 - Clientes sin email registrado: no pueden recibir la factura
   electronica. ISNULL presenta el dato faltante de forma legible. */
SELECT 'C02' AS Consulta, ClienteId, CONCAT(Apellido, N', ', Nombre) AS Cliente,
       ISNULL(Email, N'(sin email)') AS Email, FechaAlta
FROM dbo.Cliente
WHERE Email IS NULL
ORDER BY Apellido;
```

| ClienteId | Cliente | Email | FechaAlta |
|---|---|---|---|
| 6 | Benitez, Rocio | (sin email) | 2026-10-20 |

## *C03. Detalle de ventas: quién compró, qué, quién vendió y dónde*

```
/* C03 - Detalle de ventas: quien compro que, quien lo vendio y donde
   (pedidos no cancelados). */
SELECT 'C03' AS Consulta, p.NroPedido, CAST(p.FechaPedido AS DATE) AS Fecha,
       CONCAT(c.Apellido, N', ', c.Nombre) AS Cliente,
       e.Apellido AS Vendedor, su.Nombre AS Sucursal,
       pr.Sku, pd.Cantidad, pd.PrecioUnitarioHistorico, pd.DescuentoPct,
       CAST(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0) AS DECIMAL(14,2)) AS Subtotal
FROM dbo.Pedido AS p
INNER JOIN dbo.Cliente       AS c  ON c.ClienteId   = p.ClienteId
INNER JOIN dbo.Empleado      AS e  ON e.EmpleadoId  = p.EmpleadoId
INNER JOIN dbo.Sucursal      AS su ON su.SucursalId = p.SucursalId
INNER JOIN dbo.PedidoDetalle AS pd ON pd.PedidoId   = p.PedidoId
INNER JOIN dbo.Producto      AS pr ON pr.ProductoId = pd.ProductoId
WHERE p.Estado <> 'CANCELADO'
ORDER BY p.NroPedido, pr.Sku;
```

| NroPedido | Fecha | Cliente | Vendedor | Sucursal | Sku | Cantidad | PrecioUnitarioHistorico | DescuentoPct | Subtotal |
|---|---|---|---|---|---|---|---|---|---|
| PED-2026-0001 | 2026-09-05 | Gomez, Martin | Ramirez | Casa Central | MN-SAM-001 | 1 | 198.000,00 | 5,00 | 188.100,00 |
| PED-2026-0001 | 2026-09-05 | Gomez, Martin | Ramirez | Casa Central | NB-LEN-001 | 2 | 1.050.000,00 | 0,00 | 2.100.000,00 |

| NroPedido | Fecha | Cliente | Vendedor | Sucursal | Sku | Cantidad | PrecioUnitarioHistorico | DescuentoPct | Subtotal |
|---|---|---|---|---|---|---|---|---|---|
| PED-2026-0002 | 2026-09-08 | Sosa, Lucia | Ojeda | Sucursal Norte | MO-LOG-001 | 2 | 11.500,00 | 0,00 | 23.000,00 |
| PED-2026-0002 | 2026-09-08 | Sosa, Lucia | Ojeda | Sucursal Norte | TE-LOG-002 | 1 | 64.500,00 | 10,00 | 58.050,00 |
| PED-2026-0003 | 2026-09-15 | Contable RS, Estudio | Ramirez | Casa Central | MN-SAM-001 | 2 | 198.000,00 | 7,50 | 366.300,00 |
| PED-2026-0003 | 2026-09-15 | Contable RS, Estudio | Ramirez | Casa Central | RM-KIN-003 | 2 | 74.000,00 | 0,00 | 148.000,00 |
| PED-2026-0003 | 2026-09-15 | Contable RS, Estudio | Ramirez | Casa Central | SS-KIN-001 | 4 | 112.000,00 | 0,00 | 448.000,00 |
| PED-2026-0004 | 2026-09-24 | Paredes, Nicolas | Maidana | Sucursal Obera | PA-LIL-002 | 2 | 42.000,00 | 0,00 | 84.000,00 |

*(... 8 filas más en el registro de ejecución)*

## C04. Productos sin precio vigente o sin stock

```
/* C04 - Productos con datos incompletos para la venta: sin precio vigente
   o sin existencias en ningun deposito. El LEFT JOIN conserva los productos
   sin coincidencia y COALESCE reemplaza los NULL resultantes. */
SELECT 'C04' AS Consulta, p.Sku, p.Nombre,
       COALESCE(CONVERT(NVARCHAR(20), pv.PrecioVenta), N'SIN PRECIO') AS PrecioVigente,
       COALESCE(SUM(s.Cantidad), 0) AS StockTotal,
       CASE WHEN p.Activo = 1 THEN 'ACTIVO' ELSE 'DISCONTINUADO' END AS Situacion
FROM dbo.Producto AS p
LEFT JOIN dbo.vw_ProductoPrecioVigente AS pv ON pv.ProductoId = p.ProductoId
LEFT JOIN dbo.Stock AS s ON s.ProductoId = p.ProductoId
GROUP BY p.Sku, p.Nombre, pv.PrecioVenta, p.Activo
HAVING pv.PrecioVenta IS NULL OR COALESCE(SUM(s.Cantidad), 0) = 0;
```

| Sku | Nombre | PrecioVigente | StockTotal | Situacion |
|---|---|---|---|---|
| WC-LOG-004 | Webcam Logitech C920 | SIN PRECIO | 0 | ACTIVO |
| IM-SAM-003 | Impresora Samsung Xpress M2020 | 310.000,00 | 0 | DISCONTINUADO |

## C05. Indicadores globales de ventas

```
/* C05 - Indicadores globales de ventas (pedidos no cancelados). */
SELECT 'C05' AS Consulta,
       COUNT(*)          AS Pedidos,
       SUM(t.Total)      AS VentasNetas,
       AVG(t.Total)      AS TicketPromedio,
       MIN(t.Total)      AS TicketMinimo,
       MAX(t.Total)      AS TicketMaximo,
       (SELECT SUM(pd.Cantidad) FROM dbo.PedidoDetalle pd
        JOIN dbo.Pedido p2 ON p2.PedidoId = pd.PedidoId
        WHERE p2.Estado <> 'CANCELADO') AS UnidadesVendidas
FROM dbo.vw_PedidoTotal AS t
JOIN dbo.Pedido AS p ON p.PedidoId = t.PedidoId
WHERE p.Estado <> 'CANCELADO';
```

| Pedidos | VentasNetas | TicketPromedio | TicketMinimo | TicketMaximo | UnidadesVendidas |
|---|---|---|---|---|---|
| 8 | 6.221.100,00 | 777.637,50 | 58.000,00 | 2.288.100,00 | 38 |

## C06. Ventas por sucursal y canal

```
/* C06 - Ventas por sucursal y canal. */
SELECT 'C06' AS Consulta, su.Nombre AS Sucursal, p.Canal,
       COUNT(*) AS Pedidos, SUM(t.Total) AS VentasNetas, AVG(t.Total) AS TicketPromedio
FROM dbo.Pedido AS p
JOIN dbo.Sucursal AS su ON su.SucursalId = p.SucursalId
```

```
JOIN dbo.vw_PedidoTotal AS t ON t.PedidoId = p.PedidoId
WHERE p.Estado <> 'CANCELADO'
GROUP BY su.Nombre, p.Canal
ORDER BY su.Nombre, VentasNetas DESC;
```

| Sucursal | Canal | Pedidos | VentasNetas | TicketPromedio |
|---|---|---|---|---|
| Casa Central | WEB | 2 | 2.700.900,00 | 1.350.450 |
| Casa Central | TELEFONO | 1 | 962.300,00 | 962.300 |
| Sucursal Norte | TELEFONO | 1 | 1.108.050,00 | 1.108.050 |
| Sucursal Norte | MOSTRADOR | 2 | 139.050,00 | 69.525 |
| Sucursal Obera | MOSTRADOR | 1 | 968.800,00 | 968.800 |
| Sucursal Obera | WEB | 1 | 342.000,00 | 342.000 |

## *C07. Categorías con venta neta superior a $1.000.000*

```
/* C07 - Categorias cuya venta neta supera $1.000.000 (HAVING filtra
   grupos; WHERE filtraria filas). Se informa la categoria padre. */
SELECT 'C07' AS Consulta, ISNULL(cp.Nombre, c.Nombre) AS Rubro, c.Nombre AS Categoria,
       COUNT(DISTINCT pd.PedidoId) AS Pedidos, SUM(pd.Cantidad) AS Unidades,
       SUM(CAST(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0) AS DECIMAL(16,2))) AS
VentaNeta
FROM dbo.PedidoDetalle AS pd
JOIN dbo.Pedido    AS p  ON p.PedidoId    = pd.PedidoId
JOIN dbo.Producto  AS pr ON pr.ProductoId = pd.ProductoId
JOIN dbo.Categoria AS c  ON c.CategoriaId = pr.CategoriaId
LEFT JOIN dbo.Categoria AS cp ON cp.CategoriaId = c.CategoriaPadreId
WHERE p.Estado <> 'CANCELADO'
GROUP BY cp.Nombre, c.Nombre
HAVING SUM(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0)) > 1000000
ORDER BY VentaNeta DESC;
```

| Rubro | Categoria | Pedidos | Unidades | VentaNeta |
|---|---|---|---|---|
| Informatica | Notebooks | 2 | 3 | 3.150.000,00 |
| Informatica | Perifericos | 6 | 16 | 1.361.300,00 |
| Informatica | Almacenamiento | 3 | 10 | 1.041.000,00 |

## *C08. Clientes con más de un pedido*

```
/* C08 - Clientes recurrentes: mas de un pedido no cancelado. */
SELECT 'C08' AS Consulta, c.ClienteId, CONCAT(c.Apellido, N', ', c.Nombre) AS Cliente,
       COUNT(*) AS Pedidos, SUM(t.Total) AS TotalComprado
FROM dbo.Cliente AS c
JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId
JOIN dbo.vw_PedidoTotal AS t ON t.PedidoId = p.PedidoId
WHERE p.Estado <> 'CANCELADO'
GROUP BY c.ClienteId, c.Apellido, c.Nombre
HAVING COUNT(*) > 1
ORDER BY TotalComprado DESC;
```

| ClienteId | Cliente | Pedidos | TotalComprado |
|---|---|---|---|
| 1 | Gomez, Martin | 2 | 2.700.900,00 |
| 2 | Sosa, Lucia | 2 | 1.049.850,00 |

## *C09. Productos con precio superior al promedio*

```
/* C09 - Productos cuyo precio vigente supera el promedio del catalogo
   (subconsulta escalar no correlacionada). */
SELECT 'C09' AS Consulta, p.Sku, p.Nombre, pv.PrecioVenta,
       (SELECT CAST(AVG(PrecioVenta) AS DECIMAL(12,2)) FROM dbo.vw_ProductoPrecioVigente) AS PromedioCatalogo
FROM dbo.Producto AS p
JOIN dbo.vw_ProductoPrecioVigente AS pv ON pv.ProductoId = p.ProductoId
WHERE pv.PrecioVenta > (SELECT AVG(PrecioVenta) FROM dbo.vw_ProductoPrecioVigente)
```

```
ORDER BY pv.PrecioVenta DESC;
```

| Sku | Nombre | PrecioVenta | PromedioCatalogo |
|---|---|---|---|
| NB-LEN-002 | Notebook Lenovo ThinkPad E14 | 1.580.000,00 | 404.261,54 |
| AA-BGH-001 | Aire BGH Silent Air 3500W | 1.250.000,00 | 404.261,54 |
| NB-LEN-001 | Notebook Lenovo IdeaPad 3 15" | 1.050.000,00 | 404.261,54 |

## C10. Productos activos nunca vendidos

```
/* C10 - Productos activos que nunca se vendieron (NOT EXISTS). */
SELECT 'C10' AS Consulta, p.Sku, p.Nombre, dbo.fn_StockDisponible(p.ProductoId, NULL) AS StockTotal
FROM dbo.Producto AS p
WHERE p.Activo = 1
  AND NOT EXISTS (SELECT 1 FROM dbo.PedidoDetalle AS pd
                  JOIN dbo.Pedido AS pe ON pe.PedidoId = pd.PedidoId
                  WHERE pd.ProductoId = p.ProductoId AND pe.Estado <> 'CANCELADO')
ORDER BY p.Sku;
```

| Sku | Nombre | StockTotal |
|---|---|---|
| AA-BGH-001 | Aire BGH Silent Air 3500W | 17 |
| MC-SAM-002 | Microondas Samsung MG23 | 16 |
| NB-LEN-002 | Notebook Lenovo ThinkPad E14 | 11 |
| WC-LOG-004 | Webcam Logitech C920 | 0 |

## C11. Proveedor preferido de los productos bajo mínimo

```
/* C11 - A quien reponer: proveedor preferido de cada producto que esta
   bajo minimo en algun deposito (subconsulta IN sobre una vista). */
SELECT 'C11' AS Consulta, pr.RazonSocial, p.Sku, pp.PrecioCompra, pp.PlazoEntregaDias
FROM dbo.ProveedorProducto AS pp
JOIN dbo.Proveedor AS pr ON pr.ProveedorId = pp.ProveedorId
JOIN dbo.Producto  AS p  ON p.ProductoId   = pp.ProductoId
WHERE pp.EsPreferido = 1
  AND pp.ProductoId IN (SELECT ProductoId FROM dbo.vw_StockBajoMinimo)
ORDER BY pr.RazonSocial;
```

| RazonSocial | Sku | PrecioCompra | PlazoEntregaDias |
|---|---|---|---|
| TecnoMayorista SA | MN-SAM-001 | 142.000,00 | 8 |

## C12. Último pedido de cada cliente

```
/* C12 - Ultimo pedido de cada cliente (subconsulta correlacionada: se
   evalua una vez por cada fila externa). */
SELECT 'C12' AS Consulta, CONCAT(c.Apellido, N', ', c.Nombre) AS Cliente, p.NroPedido, p.FechaPedido, p.Estado
FROM dbo.Pedido AS p
JOIN dbo.Cliente AS c ON c.ClienteId = p.ClienteId
WHERE p.FechaPedido = (SELECT MAX(p2.FechaPedido) FROM dbo.Pedido AS p2
                       WHERE p2.ClienteId = p.ClienteId)
ORDER BY p.FechaPedido DESC;
```

| Cliente | NroPedido | FechaPedido | Estado |
|---|---|---|---|
| del Litoral SA, Tecnologias | PED-2026-0007 | 2026-11-12 09:15:47 | CONFIRMADO |
| Contable RS, Estudio | PED-2026-0009 | 2026-11-12 09:15:47 | CANCELADO |
| Sosa, Lucia | PED-2026-0008 | 2026-11-12 09:15:47 | ENTREGADO |
| Gomez, Martin | PED-2026-0006 | 2026-11-12 09:15:47 | ENTREGADO |
| Kowalski, Helena | PED-2026-0005 | 2026-09-29 19:02:00 | CONFIRMADO |
| Paredes, Nicolas | PED-2026-0004 | 2026-09-24 12:45:00 | ENTREGADO |

## C13. Cuentas a cobrar con teléfono de contacto

```
/* C13 - Cuentas a cobrar: clientes con al menos una factura con saldo y
   con telefono de contacto para gestionar la cobranza (EXISTS). */
SELECT 'C13' AS Consulta, c.ClienteId, CONCAT(c.Apellido, N', ', c.Nombre) AS Cliente,
       CAST(SUM(s.SaldoPendiente) AS DECIMAL(16,2)) AS Deuda, COUNT(*) AS FacturasConSaldo
FROM dbo.Cliente AS c
JOIN dbo.Pedido AS p ON p.ClienteId = c.ClienteId
JOIN dbo.Factura AS f ON f.PedidoId = p.PedidoId
JOIN dbo.vw_FacturaSaldo AS s ON s.FacturaId = f.FacturaId
WHERE s.SaldoPendiente > 0
  AND EXISTS (SELECT 1 FROM dbo.ClienteTelefono ct WHERE ct.ClienteId = c.ClienteId)
GROUP BY c.ClienteId, c.Apellido, c.Nombre
ORDER BY Deuda DESC;
```

| ClienteId | Cliente | Deuda | FacturasConSaldo |
|---|---|---|---|
| 3 | Contable RS, Estudio | 698.629,80 | 1 |
| 7 | del Litoral SA, Tecnologias | 670.370,25 | 1 |
| 4 | Paredes, Nicolas | 213.820,00 | 1 |
| 5 | Kowalski, Helena | 40.180,00 | 1 |

## C14. Precio cobrado frente a precio de lista

```
/* C14 - Precio cobrado frente al precio de lista de la fecha de venta y
   frente al precio vigente hoy (fn_PrecioVigente). Evidencia de RN06: el
   precio historico del renglon no cambia aunque cambie la lista. */
SELECT 'C14' AS Consulta, p.NroPedido, CAST(p.FechaPedido AS DATE) AS Fecha, pr.Sku,
       pd.PrecioUnitarioHistorico AS PrecioCobrado,
       dbo.fn_PrecioVigente(pd.ProductoId, CAST(p.FechaPedido AS DATE)) AS PrecioListaEseDia,
       dbo.fn_PrecioVigente(pd.ProductoId, NULL) AS PrecioListaHoy
FROM dbo.PedidoDetalle AS pd
JOIN dbo.Pedido   AS p  ON p.PedidoId    = pd.PedidoId
JOIN dbo.Producto AS pr ON pr.ProductoId = pd.ProductoId
WHERE pd.PrecioUnitarioHistorico <> dbo.fn_PrecioVigente(pd.ProductoId, NULL)
ORDER BY p.NroPedido;
```

| NroPedido | Fecha | Sku | PrecioCobrado | PrecioListaEseDia | PrecioListaHoy |
|---|---|---|---|---|---|
| PED-2026-0001 | 2026-09-05 | MN-SAM-001 | 198.000,00 | 198.000,00 | 214.000,00 |
| PED-2026-0002 | 2026-09-08 | MO-LOG-001 | 11.500,00 | 11.500,00 | 12.900,00 |
| PED-2026-0003 | 2026-09-15 | MN-SAM-001 | 198.000,00 | 198.000,00 | 214.000,00 |
| PED-2026-0003 | 2026-09-15 | SS-KIN-001 | 112.000,00 | 112.000,00 | 129.000,00 |

## C15. Disponibilidad por depósito

```
/* C15 - Disponibilidad de los productos de Informatica por deposito y
   total (fn_StockDisponible). */
SELECT 'C15' AS Consulta, p.Sku,
       dbo.fn_StockDisponible(p.ProductoId, 1) AS Central,
       dbo.fn_StockDisponible(p.ProductoId, 2) AS SalonCC,
       dbo.fn_StockDisponible(p.ProductoId, 3) AS Norte,
       dbo.fn_StockDisponible(p.ProductoId, 4) AS Obera,
       dbo.fn_StockDisponible(p.ProductoId, NULL) AS Total
FROM dbo.Producto AS p
JOIN dbo.Categoria AS c ON c.CategoriaId = p.CategoriaId
WHERE c.CategoriaPadreId = (SELECT CategoriaId FROM dbo.Categoria WHERE Nombre = N'Informatica')
  AND p.Activo = 1
ORDER BY p.Sku;
```

| Sku | Central | SalonCC | Norte | Obera | Total |
|---|---|---|---|---|---|
| MN-SAM-001 | 33 | 0 | 0 | 4 | 37 |
| MO-LOG-001 | 178 | 25 | 39 | 0 | 242 |
| NB-LEN-001 | 24 | 0 | 5 | 0 | 29 |

| Sku | Central | SalonCC | Norte | Obera | Total |
|---|---|---|---|---|---|
| NB-LEN-002 | 11 | 0 | 0 | 0 | 11 |
| RM-KIN-003 | 58 | 0 | 0 | 0 | 58 |
| SS-KIN-001 | 92 | 0 | 0 | 0 | 92 |
| SS-KIN-002 | 140 | 0 | 28 | 0 | 168 |
| TE-LOG-002 | 52 | 18 | 0 | 0 | 70 |

*(... 1 filas más en el registro de ejecución)*

## C16. Ranking de productos por venta neta

```
/* C16 - Ranking de productos por venta neta (RANK) y participacion
   porcentual sobre el total (SUM OVER). */
SELECT 'C16' AS Consulta,
       RANK() OVER (ORDER BY v.VentaNeta DESC) AS Puesto,
       v.Sku, v.Unidades, v.VentaNeta,
       CAST(100.0 * v.VentaNeta / SUM(v.VentaNeta) OVER () AS DECIMAL(5,2)) AS PorcentajeDelTotal
FROM (SELECT pr.Sku, SUM(pd.Cantidad) AS Unidades,
             SUM(CAST(pd.Cantidad * pd.PrecioUnitarioHistorico * (1 - pd.DescuentoPct / 100.0) AS DECIMAL(16,2)))
AS VentaNeta
      FROM dbo.PedidoDetalle AS pd
      JOIN dbo.Pedido   AS p  ON p.PedidoId    = pd.PedidoId
      JOIN dbo.Producto AS pr ON pr.ProductoId = pd.ProductoId
      WHERE p.Estado <> 'CANCELADO'
      GROUP BY pr.Sku) AS v
ORDER BY Puesto;
```

| Puesto | Sku | Unidades | VentaNeta | PorcentajeDelTotal |
|---|---|---|---|---|
| 1 | NB-LEN-001 | 3 | 3.150.000,00 | 50,63 |
| 2 | MN-SAM-001 | 6 | 1.196.400,00 | 19,23 |
| 3 | SS-KIN-001 | 7 | 835.000,00 | 13,42 |
| 4 | VT-LIL-001 | 7 | 584.800,00 | 9,40 |
| 5 | RM-KIN-003 | 2 | 148.000,00 | 2,38 |
| 6 | MO-LOG-001 | 9 | 106.850,00 | 1,72 |
| 7 | PA-LIL-002 | 2 | 84.000,00 | 1,35 |
| 8 | TE-LOG-002 | 1 | 58.050,00 | 0,93 |

*(... 1 filas más en el registro de ejecución)*

## C17. Trazabilidad de un pedido anulado

```
/* C17 - Trazabilidad de un pedido anulado: estados, momentos y usuario,
   junto con los movimientos de stock que genero y revirtio. */
SELECT 'C17 estados' AS Consulta, p.NroPedido, h.EstadoAnterior, h.EstadoNuevo, h.FechaHora
FROM dbo.PedidoEstadoHistorial AS h
JOIN dbo.Pedido AS p ON p.PedidoId = h.PedidoId
WHERE p.Estado = 'CANCELADO'
ORDER BY h.HistorialId;


SELECT 'C17 kardex' AS Consulta, p.NroPedido, m.TipoMovimiento, pr.Sku, m.Cantidad, m.Motivo
FROM dbo.MovimientoStock AS m
JOIN dbo.Pedido   AS p  ON p.PedidoId    = m.DocumentoId AND m.DocumentoTipo = 'PEDIDO'
JOIN dbo.Producto AS pr ON pr.ProductoId = m.ProductoId
WHERE p.Estado = 'CANCELADO'
ORDER BY m.MovimientoId;
```

| NroPedido | EstadoAnterior | EstadoNuevo | FechaHora |
|---|---|---|---|
| PED-2026-0009 | NULL | CONFIRMADO | 2026-11-12 09:15:47 |
| PED-2026-0009 | CONFIRMADO | CANCELADO | 2026-11-12 09:15:47 |

| NroPedido | TipoMovimiento | Sku | Cantidad | Motivo |
|---|---|---|---|---|
| PED-2026-0009 | EGRESO | NB-LEN-002 | 1 | Venta PED-2026-0009 |
| PED-2026-0009 | EGRESO | RM-KIN-003 | 4 | Venta PED-2026-0009 |
| PED-2026-0009 | EGRESO | SS-KIN-002 | 10 | Venta PED-2026-0009 |

| NroPedido | TipoMovimiento | Sku | Cantidad | Motivo |
|---|---|---|---|---|
| PED-2026-0009 | INGRESO | NB-LEN-002 | 1 | Anulacion PED-2026-0009: El cliente desiste de la compra |
| PED-2026-0009 | INGRESO | SS-KIN-002 | 10 | Anulacion PED-2026-0009: El cliente desiste de la compra |
| PED-2026-0009 | INGRESO | RM-KIN-003 | 4 | Anulacion PED-2026-0009: El cliente desiste de la compra |

## C18. Catálogo y cartera de clientes

```
/* C18 - Consultas sobre las vistas del sistema. */
SELECT 'C18 catalogo' AS Consulta, Sku, Producto, Categoria, Marca,
       ISNULL(CONVERT(NVARCHAR(20), PrecioVigente), N'-') AS PrecioVigente, StockTotal, ProveedorPreferido
FROM dbo.vw_CatalogoProductos
WHERE Activo = 1
ORDER BY Categoria, Producto;

SELECT 'C18 cartera' AS Consulta, Cliente, Documento, Email, CantidadPedidos,
       TotalComprado, UltimaCompra, CAST(SaldoPendiente AS DECIMAL(16,2)) AS SaldoPendiente
FROM dbo.vw_ResumenVentasCliente
ORDER BY TotalComprado DESC;
```

| Sku | Producto | Categoria | Marca | PrecioVigente | StockTotal | ProveedorPreferido |
|---|---|---|---|---|---|---|
| RM-KIN-003 | Memoria Kingston Fury 16GB | Almacenamiento | Kingston | 74.000,00 | 58 | TecnoMayorista SA |
| SS-KIN-002 | SSD Kingston A400 480GB | Almacenamiento | Kingston | 58.000,00 | 168 | TecnoMayorista SA |
| SS-KIN-001 | SSD Kingston NV2 1TB | Almacenamiento | Kingston | 129.000,00 | 92 | TecnoMayorista SA |
| AA-BGH-001 | Aire BGH Silent Air 3500W | Climatizacion | BGH | 1.250.000,00 | 17 | Electro Litoral SRL |
| VT-LIL-001 | Ventilador Liliana VTF20 | Climatizacion | Liliana | 86.000,00 | 115 | Electro Litoral SRL |
| MC-SAM-002 | Microondas Samsung MG23 | Cocina | Samsung | 385.000,00 | 16 | Importadora Guarani SA |
| PA-LIL-002 | Pava electrica Liliana AP940 | Cocina | Liliana | 42.000,00 | 124 | Electro Litoral SRL |
| NB-LEN-001 | Notebook Lenovo IdeaPad 3 15" | Notebooks | Lenovo | 1.050.000,00 | 29 | TecnoMayorista SA |

*(... 5 filas más en el registro de ejecución)*

| Cliente | Documento | Email | CantidadPedidos | TotalComprado | UltimaCompra | SaldoPendiente |
|---|---|---|---|---|---|---|
| Gomez, Martin | DNI 32145678 | mgomez@mail.com | 2 | 2.700.900,00 | 2026-11-12 09:15:47 | 0,00 |
| del Litoral SA, Tecnologias | CUIT 30715559991 | compras@tecnolitoral.com.ar | 1 | 1.108.050,00 | 2026-11-12 09:15:47 | 670.370,25 |
| Sosa, Lucia | DNI 27998877 | lsosa@mail.com | 2 | 1.049.850,00 | 2026-11-12 09:15:47 | 0,00 |
| Contable RS, Estudio | CUIT 20345678901 | admin@estudiors.com.ar | 1 | 962.300,00 | 2026-09-15 10:12:00 | 698.629,80 |
| Paredes, Nicolas | DNI 40112233 | nparedes@mail.com | 1 | 342.000,00 | 2026-09-24 12:45:00 | 213.820,00 |
| Kowalski, Helena | PAS AB1234567 | hkowalski@mail.com | 1 | 58.000,00 | 2026-09-29 19:02:00 | 40.180,00 |
| Benitez, Rocio | DNI 35111222 | (sin email) | 0 | 0,00 | NULL | 0,00 |

## C19. Anular órdenes en borrador con más de 30 días

```
/* C19 - UPDATE con subconsulta: anular ordenes de compra en BORRADOR con
   mas de 30 dias de antiguedad y sin mercaderia recibida. */
```

```
UPDATE dbo.OrdenCompra
   SET Estado = 'ANULADA'
 WHERE Estado = 'BORRADOR'
   AND FechaEmision < DATEADD(DAY, -30, CAST(SYSDATETIME() AS DATE))
   AND NOT EXISTS (SELECT 1 FROM dbo.OrdenCompraDetalle d
                   WHERE d.OrdenCompraId = dbo.OrdenCompra.OrdenCompraId AND d.CantidadRecibida > 0);
PRINT CONCAT('C19 ordenes de compra anuladas: ', @@ROWCOUNT);
SELECT 'C19' AS Consulta, NroOrden, FechaEmision, Estado FROM dbo.OrdenCompra ORDER BY NroOrden;
```

| NroOrden | FechaEmision | Estado |
|---|---|---|
| OC-2026-0001 | 2026-08-04 | RECIBIDA |
| OC-2026-0002 | 2026-08-20 | RECIBIDA |
| OC-2026-0003 | 2026-09-10 | RECIBIDA |
| OC-2026-0004 | 2026-09-22 | ANULADA |

Mensaje de ejecución: «C19 ordenes de compra anuladas: 1».

## *C20. Eliminar faxes de proveedores con teléfono principal*

```
/* C20 - DELETE con subconsulta: dar de baja los numeros de FAX de los
   proveedores que conservan un telefono PRINCIPAL. */
DELETE FROM dbo.ProveedorTelefono
 WHERE Tipo = 'FAX'
   AND ProveedorId IN (SELECT ProveedorId FROM dbo.ProveedorTelefono WHERE Tipo = 'PRINCIPAL');
PRINT CONCAT('C20 telefonos de fax eliminados: ', @@ROWCOUNT);
SELECT 'C20' AS Consulta, ProveedorId, NroTelefono, Telefono, Tipo
FROM dbo.ProveedorTelefono ORDER BY ProveedorId, NroTelefono;
```

| ProveedorId | NroTelefono | Telefono | Tipo |
|---|---|---|---|
| 1 | 1 | 0376-4451122 | PRINCIPAL |
| 1 | 2 | 0376-154778899 | WHATSAPP |
| 2 | 1 | 011-47883322 | PRINCIPAL |
| 3 | 1 | 0342-4559988 | PRINCIPAL |
| 4 | 1 | 03757-422110 | PRINCIPAL |
| 4 | 2 | 03757-422111 | ALTERNATIVO |

Mensaje de ejecución: «C20 telefonos de fax eliminados: 1».